

Heap Thrust Reference

ICPC Reference

Team Members:

Enrique Job Calderón (ksobrenat32)
Luis Daniel Salazar (luisdakan)
Gustavo Valenzuela (GusTimeTraveler)

Institution:

Facultad de Ingeniería, UNAM - CPCFI

August 10, 2026

Contents

1	Python stuff	4
1.1	Regular input	4
1.2	Fast I/O	4
1.3	List comprehensions	4
1.4	Strings	4
1.5	Regex	5
1.5.1	Regex notes	5
1.6	Itertools	6
1.7	Scripts	7
1.7.1	Combinatorics calculator	7
1.7.2	Random array generator	7
1.7.3	Generate all subsets	7
2	C++ utils	7
2.1	Common STL Algorithms	7
2.2	Random number generator	9
2.2.1	Graph generator	9
3	Binary Search in the Answer	9
4	Bitwise Techniques	10
4.1	All Subsets	10
4.2	FWHT	10
4.3	Gray Code	11
4.4	XOR Basis	11
4.5	Hopcroft-Karp	11
4.6	Min-Cost Max-Flow	12
4.7	Push-Relabel	13
4.8	0-1 BFS	14
4.9	Biconnected Components	14
4.10	Johnson's Algorithm	15
5	Graph Theory	16
5.1	Bellman-Ford	16
5.2	Dijkstra	16
5.3	Dinic	16
5.4	Directed Cycles	18
5.5	Edmonds-Karp	18
5.6	Floyd-Warshall	19
5.7	Kahn Topological Sort	19
5.8	Kruskal	19

5.9	Lava Flow	20
5.10	Negative Cycles	21
5.11	Prim Algorithm	21
5.12	Strongly Connected Components	22
5.13	Topological Sort	23
5.14	2-SAT	23
5.15	Undirected Cycles	24
5.16	Articulation Points	24
5.17	Bridges	25
5.18	Bridges Online	25
5.19	Bridge Tree	26
5.20	Dynamic Connectivity	27
5.21	Eulerian Path	28
5.22	Gale-Shapley	28
5.23	Hungarian Algorithm	29
5.24	SPFA	29
5.25	Tarjan	30
5.26	Sqrt Decomposition	30
6	Data Structures	31
6.1	2D Segment Tree	31
6.2	Bit Trie	32
6.3	Disjoint Set Union	33
6.4	Fenwick Tree	33
6.5	GP Hash Table	33
6.6	Implicit Treap	34
6.7	Iterative Segment Tree	35
6.8	Lazy Segment Tree (Max)	35
6.9	Lazy Segment Tree (Sum)	36
6.10	Lazy Treap	37
6.11	Merge Sort Tree	38
6.12	Ordered Set	39
6.13	Recursive Segment Tree	39
6.14	Treap	40
6.15	Cycle Detection Online	41
6.16	DSU with Rollbacks	41
6.17	Fenwick Lazy	42
6.18	Implicit Treap 2	42
6.19	Online Bipartiteness	43
6.20	Path Copy Persistent Array	43

6.21	Persistent Array	44	8.26	Mod Struct	59
6.22	Persistent Segment Tree	44	8.27	Factorials	59
6.23	Prefix Suffix Segment Tree	45	8.28	Number of Divisors	59
6.24	Wicho Lazy Segment Tree	45	8.29	Modular Inverse	59
6.25	Li Chao Tree	47	8.30	Discrete Logarithm	60
6.26	Mo's Algorithm	47	8.31	Segmented Sieve	60
7	Range Queries	48	8.32	Sieve of Eratosthenes	60
7.1	Sparse Table	48	8.33	Simpson Method	61
8	Mathematics	48	8.34	Precompute Pascal Triangle	61
8.1	Formula Sheet (Contest)	48	8.35	Sqrt and Inverse Sqrt	61
8.1.1	Combinatorics	48	8.36	Sum of Divisors	61
8.2	Cayley's Formula	49	8.37	Ternary Search	62
8.3	Geometric series	49	8.38	Trivial Factorization	62
8.4	Divisors	49	8.39	Chinese Remainder Theorem	62
8.5	Number of primes between 1 and n	49	8.40	Garner's Algorithm	63
8.6	Pythagorean triplets	49	8.41	Polynomial Operations	64
8.7	Derangements	49	8.42	Linear Recurrence	66
8.7.1	Sums	50	8.43	Matrix Exponentiation	67
8.7.2	Geometry	50	8.44	Modular Square Root	68
8.7.3	Probability	50	8.45	Simplex	68
8.7.4	Number Theory and Counting Essentials	50	9	String Algorithms	71
8.8	Manhattan Distance on a Grid	51	9.1	Aho-Corasick	71
8.9	Binary Exponentiation	51	9.2	DP on Trie	72
8.10	Binomial Coefficients	51	9.3	KMP	72
8.11	Extended Euclidean	51	9.4	Manacher Algorithm	73
8.12	FFT	52	9.5	Palindromic Tree	73
8.13	Big Integer	53	9.6	Polynomial Hash	74
8.14	NTT	53	9.7	Rabin-Karp	75
8.15	Graph Paths	54	9.8	Suffix Array + LCP	75
8.16	Graph Paths Min-Plus	55	9.9	Suffix Automaton	76
8.17	Log Fibonacci	55	9.10	Suffix Automata 2	77
8.18	Matrix Determinant	56	9.11	String Stream	78
8.19	Matrix Rank	56	9.12	XOR Hash	78
8.20	Gauss-Jordan	56	9.13	Z Algorithm	78
8.21	Miller-Rabin (Primality Test)	57	10	Computational Geometry	79
8.22	Linear Sieve	57	10.1	Convex Hull	79
8.23	Euler Totient Function	58	10.2	Complex Point	80
8.24	Fast Totient Function	58	10.3	Line Segment Intersection	80
8.25	Diophantine Equations	58			

10.4 Minimum Euclidean Distance	81	11.12 SOS DP	92
10.5 Minimum Manhattan Distance	82	11.13 Convex Hull Trick	92
10.6 Point In Polygon	82	11.14 Knuth's Optimization	93
10.7 Point Location Test	83	11.15 Divide and Conquer DP	94
10.8 Polar Sort	83	12 Trees	94
10.9 Polygon Area	84	12.1 Diameter	94
10.10 Closest Pair of Points	84	12.2 Euler Tour	95
10.11 Half-Plane Intersection	85	12.3 Tree Isomorphism	96
10.12 Smallest Enclosing Circle	87	12.4 Max Distance	96
10.13 Rotating Calipers	87	12.5 Binary Lifting	97
10.14 Pick's Theorem	88	12.6 Centroid Decomposition	97
11 Dynamic Programming	88	12.7 Heavy-Light Decomposition	98
11.1 Coin Problem	88	12.8 Lowest Common Ancestor	99
11.2 Coin Problem (Min Coins)	89	12.9 Successor DP	100
11.3 Coin Problem (Sorted)	89	13 Miscellaneous	100
11.4 Knapsack	89	13.1 2D Prefix Sum	100
11.5 LCS	90	13.2 Kadane	100
11.6 LIS	90	13.3 Index Compression	101
11.7 Two Sets	90	13.4 Berlekamp-Massey	101
11.8 Bitmask DP	90	13.5 Moore's Voting	101
11.9 Digit DP	91	13.6 Compiler Optimizations	102
11.10 Edit Distance	91	13.7 Sprague-Grundy	102
11.11 Retrieve LIS	92		

1 Python stuff

1.1 Regular input

```

1 s = input() # This is a string
2 n = int(input()) # This is an integer
3
4 arr = input().split() # This is an array of strings
5
6 # Array of integers (list comprehension)
7 a = [int(x) for x in input().split()]
8 x,y,z = [int(i) for i in input().split()] # Multiple integers

```

1.2 Fast I/O

```

1 import sys
2 input = sys.stdin.readline
3 n = 220
4 sys.stdout.write(str(n) + '\n')

```

1.3 List comprehensions

```

1 # 1 to N
2 nums = [i for i in range(1, N+1)]
3
4 # Filter values
5 evens = [x for x in arr if x%2 == 0]
6
7 # Filter primes
8 primes = [x for x in arr if all(x%d for d in range(2,int(x**0.5)+1))
9           ]
10
11 # Square every element
12 sqr = [x*x for x in arr]
13
14 # Convert string digits to int
15 digits = [int(c) for c in s]
16
17 # Nested loops
18 pairs = [(i,j) for i in range(N) for j in range(M)]
19
20 # Generate NxM matrix
21 matrix = [[i*j for j in range(M)] for i in range(N)]
22
23 # Generate all combinations of two lists
24 combs = [(a,b) for a in list1 for b in list2]

```

```

24
25 # Prefix Sum
26 from itertools import accumulate
27 pref = list(accumulate(arr))
28
29 # Count occurrences of a value (without count)
30 cnt = sum(1 for x in arr if x == target)

```

1.4 Strings

```

1 s = "  Hola reference  "
2 # Remove trailing/whitespaces
3 print(s.strip()) # also .lstrip and .rstrip
4 print(s.split()) # Separates by spaces on list
5
6 s = "hola,reference,icpc,2026"
7 print(s.split(',')) # Separate by character
8
9 strs = ['vamos', 'a', 'ganar', 'icpc']
10
11 # Join a list of strings
12 joined = ' '.join(strs)
13 print(joined)
14
15 print(s.replace('hola','adios')) # Replace substrings
16 s.find("reference") # Finds the first index of the substring
17 s.rfind("reference") # Finds the last index of substring
18
19 s.startswith('hola') # Checks for a prefix
20 s.endswith('2026') # Checks for a suffix
21
22 s.count('icpc') # Counts the number of non overlapping occurrences
23 s.lower() # to lower
24 s.upper() # to upper
25
26 s.capitalize() # Capitalizes the word
27
28 c = 'a'
29 c.isalpha() # Is from alphabet
30
31 c='2'
32 c.isdigit() # Checks if it's a digit
33
34 c = '69'
35 c.isnumeric() # Checks if it is a number
36
37 # Slice a string (substring)
38
39 subs = s[2:5] # Substring from indices 2 to 5
40 rev = s[::-1] # Reverse a substring

```

```

41 step = s[::2] # Transform into every second character
42
43 c = 'z'
44 ord(c) # Unicode code point of the character
45 chr(65) # Reverse of ord

```

1.5 Regex

We must be careful with these functions since they could blow into $O(2^n)$ complexity.

```

1 import re
2
3 # 1. Basic matching
4 re.match(r'foo', 'foobar')           # <re.Match object>
5 re.search(r'bar', 'foobar')          # <re.Match object>
6 re.fullmatch(r'foo', 'foo')          # <re.Match object>
7
8 # 2. Find all / finditer
9 re.findall(r'\d+', 'a12b34')          # ['12', '34']
10 re.finditer(r'\d+', 'a12b34')         # iterator over <Match> objects
11
12 # 3. Replace
13 re.sub(r'\d+', '#', 'a12b34')         # 'a#b#'
14 re.subn(r'\d+', '#', 'a12b34')        # ('a#b#', 2)
15
16 # 4. Split
17 re.split(r'\W+', 'a,b;c')             # ['a', 'b', 'c']
18
19 # 5. Escape a literal
20 re.escape(r'foo.bar*')                # 'foo\\.bar\\*'

```

1.5.1 Regex notes

The "must-know" Python `re` functions (and their runtimes).

`re.match(pattern, string, flags=0)` Test start of `string`. $O(n)$ avg, exponential worst case.

`re.search(pattern, string, flags=0)` Find first match anywhere in `string`. $O(n)$ avg, exponential worst case.

`re.fullmatch(pattern, string, flags=0)` Match entire string. $O(n)$ avg, exponential worst case.

`re.compile(pattern, flags=0)` Pre-compile pattern. $O(m)$ pattern length.

`re.findall(pattern, string, flags=0)` All non-overlapping matches. $O(n)$ avg.

`re.finditer(pattern, string, flags=0)` Iterator of matches. $O(n)$ avg.

`re.split(pattern, string, maxsplit=0, flags=0)` Split at matches. $O(n)$ avg.

`re.sub(pattern, repl, string, count=0, flags=0)` Replace matches. $O(n)$ avg.

`re.subn(pattern, repl, string, count=0, flags=0)` Replace + count. $O(n)$ avg.

`re.escape(string)` Escape metacharacters. $O(m)$ m = input length.

`re.finditer + match.group()` Extract captured groups.

`re.Scanner (rare)` Multi-pattern scanner for tokenizers.

General rule of thumb: If your pattern does not contain nested quantifiers or back-references, the engine runs in **linear time** in the length of the input string. If you do have nested `*`, `+`, `{m,n}` or back-references, the worst-case becomes **exponential** - this is the "catastrophic backtracking" phenomenon.

Why the exponential blow-up happens

Python's regex engine is a **back-tracking NFA**. When the engine tries a branch (e.g., a `.` that can match any character) it keeps a stack of alternatives. If the branch fails later, it **pops** to the previous branch and tries the next alternative. If a pattern has k nested alternations, the number of states explored can be as high as 2^k .

Typical examples:

```

1 pattern = r'^{(a)+}$'           # matches only "a", but the engine
                                # explores 2^(len(s)) possibilities

```

Tip: If you ever hit a "too slow" regex, try to rewrite it so that the number of back-tracking branches is bounded or replace it with a deterministic automaton (e.g., use `re.fullmatch(r'[a-z]+')` instead of `re.match(r'.*')`).

Pre-compilation matters

```

1 pat = re.compile(r'\bfoo\b')
2 for line in sys.stdin:
3     if pat.search(line):
4         ...

```

Compiling once keeps the $O(m)$ cost to a single place, then every call to `search`, `match`, or `sub` runs in $O(n)$.

Quick-reference cheat-sheet

```

1 import re
2
3 # 1. Basic matching
4 re.match(r'foo', 'foobar')           # <re.Match object>
5 re.search(r'bar', 'foobar')          # <re.Match object>
6 re.fullmatch(r'foo', 'foo')          # <re.Match object>
7
8 # 2. Find all / finditer
9 re.findall(r'\d+', 'a12b34')          # ['12', '34']
10 re.finditer(r'\d+', 'a12b34')         # iterator over <Match> objects
11
12 # 3. Replace
13 re.sub(r'\d+', '#', 'a12b34')         # 'a#b#'
14 re.subn(r'\d+', '#', 'a12b34')        # ('a#b#', 2)
15
16 # 4. Split
17 re.split(r'\W+', 'a,b;c')             # ['a', 'b', 'c']
18
19 # 5. Escape a literal
20 re.escape(r'foo.bar*')                # 'foo\\.bar\\*'
21
22 # 6. Pre-compile
23 pat = re.compile(r'\bfoo\b')
24 for line in sys.stdin:
25     if pat.search(line):
26         ...

```

Summary of complexities

Complexity summary (all worst cases are 2^k where k is the number of nested quantifiers):

- `match`, `search`, `fullmatch`: $O(n)$ avg. Watch for `.*`, `.+`, or back-references in the pattern.

- `findall`, `finditer`: $O(n)$ avg. Non-overlapping matches.
- `split`: $O(n)$ avg. Avoid `.+?` inside the separator.
- `sub`, `subn`: $O(n)$ avg. Replacement string or callable.
- `escape`: $O(m)$ avg and worst. Any literal.
- `compile`: $O(m)$ avg. Pre-compile once per program run.

Bottom line: For the vast majority of CP problems you will never hit the exponential worst-case. Just avoid nested quantifiers or back-references, and you're safely in linear-time territory. When you do need them, test the regex on a few worst-case inputs to be sure it runs in time.

1.6 Itertools

```

1 import itertools
2
3 a = [1,2,3,4]
4 b = [5,6,7,8]
5
6 # All 2 digit numbers composed of digits 1,2,3
7 # Cartesian product
8 for a, b in itertools.product('123', repeat=2):
9     print(a+b)
10
11 # All k-subsets (combinations)
12 for team in itertools.combinations(['A','B','C','D','E'],3):
13     print(team)
14
15 # Subsets with repetitions (combinations_with_replacement)
16 # All ways to choose 3 balls from 2 colours where each colour can be
17 # used any number of times
18 for bag in itertools.combinations_with_replacement(['red','blue'],3):
19     print(bag)
20
21 # All ways to order 3 letters O(n!)
22 for order in itertools.permutations('abc'):
23     print(''.join(order))
24
25 # Prefix XOR
26 pref_xor = list(itertools.accumulate(a, func=lambda x,y : x^y))
27
28 # Group consecutive equal keys
29 s = "AAABBBCCDA"
30 for key, group in itertools.groupby(s):

```

```

31     print(key, list(group))
32 # A ['A','A','A']
33 # B ['B','B','B']
34 # C ['C','C']
35 # D ['D']
36 # A ['A']

```

1.7 Scripts

1.7.1 Combinatorics calculator

```

1 a,b = [int(x) for x in input.split()]
2
3 MAX = 1000001
4 MOD = 1000000007
5
6 fact = [0] * MAX
7 i_fact = [0] * MAX
8
9 def inv(x : int):
10     return pow(x, MOD-2,MOD)
11
12 def factorial():
13     fact[0] = fact[1] = 1
14     for i in range(2,MAX):
15         fact[i] = (fact[i-1] * i) % MOD
16     i_fact[MAX-1] = inv(fact[MAX-1])
17     for i in range(MAX-2, -1, -1):
18         i_fact[i] = (i_fact[i+1]*(i+1))%MOD
19
20 def comb(n : int, k: int):
21     return fact[n] * i_fact[n-k]%MOD * i_fact[k] % MOD
22
23 factorial()
24 print(comb(a,b))

```

1.7.2 Random array generator

```

1 import random
2
3 with open("in.txt", 'w') as f:
4     n = random.randint(1,100000)
5     f.write(str(n)+'\n')
6     for i in range(n):
7         f.write(str(random.randint(1,100))+' ')

```

1.7.3 Generate all subsets

```

1 k = int(input())
2 S = [int(x) for x in input().split()]
3 subsets = [[S[j] for j in range(k) if (i >> j) & 1] for i in range(1
4     << k)]
5 print(subsets)

```

2 C++ utils

2.1 Common STL Algorithms

Sorting Algorithms

Function	Parameters	Description
sort	begin, end, [comp]	Standard unstable sort ($O(n \log n)$)
stable_sort	begin, end, [comp]	Stable sort preserves element order
is_sorted	begin, end, [comp]	Checks if range is sorted (returns bool)
nth_element	begin, nth, end, [comp]	Partitions around nth element

Searching Functions

Function	Parameters	Description
lower_bound	begin, end, val, [comp]	First element $\leq$ value
upper_bound	begin, end, val, [comp]	First element $>$ value
binary_search	begin, end, val, [comp]	Existence check in sorted range
find	begin, end, val	Linear search for value
find_if	begin, end, pred	Find first matching predicate

Sequence Operations

Function	Parameters	Description
reverse	begin, end	Reverse elements in-place
rotate	begin, mid, end	Rotate elements left
next_permutation	begin, end	Generate next permutation
unique	begin, end, [pred]	Remove consecutive duplicates
remove	begin, end, val	Remove elements equal to value

Numerical Functions

Function	Parameters	Description
accumulate	begin, end, init, [op]	Sum/accumulate elements
partial_sum	begin, end, dest, [op]	Compute prefix sums
__gcd	a, b	Greatest common divisor (GCC ext.)
lcm	a, b	Least common multiple (C++17)
iota	begin, end, val	Fill with consecutive values

Memory/Array Operations

Function	Parameters	Description
memset	ptr, value, count	Fill memory with byte value
fill	begin, end, value	Fill range with value
fill_n	begin, count, value	Fill N elements with value
copy	src_b, src_e, dest	Copy range to destination
copy_if	src_b, src_e, dest, pred	Copy elements matching predicate

Utility Functions

Function	Parameters	Description
swap	a, b	Swap two values
max_element	begin, end, [comp]	Find maximum element
min_element	begin, end, [comp]	Find minimum element
count	begin, end, val	Count element occurrences
all_of	begin, end, pred	Check all elements satisfy condition

Mathematical / Bitwise Builtins

Function	Parameters	Description
__builtin_popcount	x (int)	Count number of 1-bits
__builtin_popcountll	x (long long)	Count number of 1-bits (64-bit)
__builtin_clz	x (unsigned int)	Count leading zeros
__builtin_clzll	x (unsigned long long)	Leading zeros (64-bit)
__builtin_ctz	x (unsigned int)	Count trailing zeros
__builtin_ctzll	x (unsigned long long)	Trailing zeros (64-bit)
__builtin_parity	x	Return 1 if #bits is odd
__builtin_ffs	x	Position of least significant 1-bit (1-indexed)
__lg	x	Floor of $\log_2(x)$ (index of highest bit)

Priority Queues and Heaps

Function	Parameters	Description
priority_queue	[type], [container], [comp]	Max-heap by default
make_heap	begin, end, [comp]	Turn range into heap
push_heap	begin, end, [comp]	Push element into heap
pop_heap	begin, end, [comp]	Pop max element into end
sort_heap	begin, end, [comp]	Heap sort

Set / Map Utilities

Operation	Usage	Description
s.lower_bound(x)	set/map	First element $\geq x$
s.upper_bound(x)	set/map	First element $> x$
s.equal_range(x)	multiset/map	Pair of lower/upper bound
s.erase(it)	iterator	Erase element at iterator
s.find(x)	key	Iterator to key or end

String Functions

Function	Parameters	Description
stoi, stol, stoll	string, [pos], [base]	Convert string → int/long/ll
stoul, stoull	string, [pos], [base]	Convert string → unsigned
stod, stof, stold	string	Convert string → double/float/long double
to_string	value	Convert number → string
substr	pos, len	Substring
find	str, pos	Find first occurrence
rfind	str, pos	Find last occurrence

Random Number Utilities

Type / Function	Usage	Description
mt19937 rng	chrono::steady_clock::now()	Fast random generator
uniform_int_distribution	dist(a,b)(rng)	Random int in [a,b]
shuffle	begin, end, rng	Random shuffle

Other Useful Utilities

Function	Parameters	Description
chrono::high_resolution_clock	now()	Get precise current time
_int128	value	128-bit integer (manual I/O needed)
bitset < N >	ops: &, —, ; ii, ll	Fixed-size bitset manipulation
tuple	get< i >(t)	Store and access heterogeneous data
pair	first, second	Store pair of values

2.2 Random number generator

```
1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
```

2.2.1 Graph generator

```
1 import random
2 n,m = [int(x) for x in input().split()]
3
4 class Edge:
5     def __init__(self, u:int, v:int):
6         self.u = u
7         self.v = v
8
9     def __eq__(self, o):
10         return o.u == self.u and o.v == self.v
11
12     def __hash__(self):
13         return hash((self.u, self.v))
14
15
16 with open("in.txt", 'w') as f:
17     f.write(str(n)+" "+str(m)+"\n")
18     edges = set()
19     for i in range(m):
20         u,v = [0,0]
21         while u == v or Edge(u,v) in edges:
22             u = random.randint(1,n)
23             v = random.randint(1,n)
24         edges.add(Edge(u,v))
25         f.write(str(u)+" "+str(v)+"\n")
26 print(edges)
```

3 Binary Search in the Answer

```
1 // Standard binary search (iterative)
2 int binary_search(vector<int>& arr, int target) {
3     int left = 0, right = arr.size() - 1;
4     while (left <= right) {
5         int mid = left + (right - left) / 2;
6         if (arr[mid] == target) return mid;
7         if (arr[mid] < target) left = mid + 1;
8         else right = mid - 1;
9     }
10    return -1;
11 }
12
13 // Lower bound (first element >= target)
14 int lower_bound(vector<int>& arr, int target) {
15     int left = 0, right = arr.size();
16     while (left < right) {
17         int mid = left + (right - left) / 2;
18         arr[mid] < target ? left = mid + 1
19                          : right = mid;
20    }
```

```

21     return left;
22 }
23
24 // Upper bound (first element > target)
25 int upper_bound(vector<int>& arr, int target) {
26     int left = 0, right = arr.size();
27     while (left < right) {
28         int mid = left + (right - left) / 2;
29         arr[mid] <= target ? left = mid + 1
30                           : right = mid;
31     }
32     return left;
33 }
34
35 // Binary search on real numbers (e.g. sqrt)
36 double sqrt_precision(double n, double eps=1e-6) {
37     double left = 0, right = n;
38     for (int i = 0; i < 100; ++i) { // or while (right-left > eps)
39         double mid = (left + right) / 2;
40         if (mid*mid < n) left = mid;
41         else right = mid;
42     }
43     return left;
44 }
45
46 // Binary search on answer space (monotonic condition)
47 int find_min_valid(vector<int>& nums, int k) {
48     auto is_valid = [&](int x) {
49         /* condition check */
50     };
51
52     int left = 0, right = 1e9; // adjust bounds
53     while (left < right) {
54         int mid = left + (right - left) / 2;
55         is_valid(mid) ? right = mid
56                     : left = mid + 1;
57     }
58     return left;
59 }
60
61 // Binary search using for loop
62
63 int binary_search_for_loop(vector<int>& arr, int target) {
64     int left = 0, right = arr.size() - 1;
65     for (; left <= right; ) {
66         int mid = left + (right - left) / 2;
67         if (arr[mid] == target) return mid;
68         if (arr[mid] < target) left = mid + 1;
69         else right = mid - 1;
70     }
71     return -1;
72 }

```

4 Bitwise Techniques

4.1 All Subsets

```

1 void minSumSubset(vector<int> &a, ll total_zum){
2     int n = (int)a.size();
3     ll min_diff = LLONG_MAX;
4
5     for(int i=1; i<(1<< n); i++){
6         ll zum = 0;
7         for(int j=0; j<n; j++){
8             if((i & (1<<j)) != 0){
9                 zum+=a[j];
10            }
11        }
12        ll diff = abs(total_zum - 2LL * zum);
13        min_diff = min(diff,min_diff);
14    }
15
16 }

```

4.2 FWHT

```

1 const ll MOD = 1e9+7;
2
3 // MAX should be a power of 2 and >= 2*10^5
4 // FFT works with sizes that are powers of 2
5 // 2^18 = 262144 > 200000
6 const int MAX = 1 << 20;
7
8 typedef complex<double> cd;
9 double PI = acos(-1);
10
11 // Transformada de Wicho Cachondo
12 vector<ll> fwht(vector<ll> a, int d = 1) {
13     int n = a.size();
14     vector<ll> r = a;
15
16     for (int m = 2; m <= n; m *= 2) {
17         for (int k = 0; k < n; k += m) {
18             for (int j = 0; j < m / 2; j++) {
19                 ll u = r[k + j];
20                 ll t = r[k + j + m / 2];
21
22                 r[k + j] = u + t;
23                 r[k + j + m / 2] = u - t;
24             }
25         }
26     }

```

```

27     }
28
29     if (d == -1) {
30         for (int i = 0; i < n; i++)
31             r[i] /= n;
32     }
33
34     return r;
35 }

```

4.3 Gray Code

```

1 vector<string> grayCode(int n){
2     vector<string> arns;
3     for(int i=0; i<(1<<n); i++){
4         int g = i^(i>>1);
5         bitset<32> r(g);
6         string arnsito = r.to_string().substr(32-n, 32);
7         arns.push_back(arnsito);
8     }
9     return arns;
10 }

```

4.4 XOR Basis

```

1 const int D = 32;
2 ll basis[D];
3 ll bs_len = 0;
4
5 void add(int x){
6     for(int i=0; i<D; i++){
7         if((x & 1 << i) == 0) continue;
8
9         if(!basis[i]){
10             basis[i] = x;
11             bs_len++;
12             return;
13         }
14
15         x ^= basis[i];
16     }
17 }
18
19 void add2(ll x){
20     for(int i=D-1; i>=0; i--){
21         if((x>>i) & 1){
22             if(!basis[i]){

```

```

23         basis[i] = x;
24         return;
25     }
26     x ^= basis[i];
27 }
28 }
29 }
30
31 inline void solve(){
32     int n; cin>>n;
33     vector<int> st(n);
34     for(int &x : st) cin>>x;
35
36     for(int vec : st){
37         add2(vec);
38     }
39
40     ll max_xor = 0;
41
42     for(int i=D-1; i>=0; i--){
43         max_xor = max(max_xor, max_xor^basis[i]);
44     }
45
46     cout<<max_xor<<endl;
47 }

```

4.5 Hopcroft-Karp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Hopcroft-Karp: Maximum bipartite matching in O(E * sqrt(V))
5 // adj[i] contains right-side neighbors of left vertex i (0-indexed)
6 // Returns size of maximum matching.
7 // Left vertices: 0..n_l-1, Right vertices: 0..n_r-1
8
9 int hopcroft_karp(int n_l, int n_r, const vector<vector<int>>& adj)
10 {
11     const int INF = 1e9;
12
13     vector<int> pair_l(n_l, -1), pair_r(n_r, -1), dist(n_l);
14
15     auto bfs = [&]() -> bool {
16         queue<int> q;
17         for (int u = 0; u < n_l; ++u) {
18             if (pair_l[u] == -1) {
19                 dist[u] = 0;
20                 q.push(u);
21             } else {
22                 dist[u] = INF;

```

```

22     }
23 }
24
25 bool reachable_free = false;
26 while (!q.empty()) {
27     int u = q.front(); q.pop();
28     for (int v : adj[u]) {
29         int pu = pair_r[v];
30         if (pu != -1 && dist[pu] == INF) {
31             dist[pu] = dist[u] + 1;
32             q.push(pu);
33         }
34         if (pu == -1) {
35             reachable_free = true;
36         }
37     }
38 }
39 return reachable_free;
40 };
41
42 function<bool(int)> dfs = [&](int u) -> bool {
43     for (int v : adj[u]) {
44         int pu = pair_r[v];
45         if (pu == -1 || (dist[pu] == dist[u] + 1 && dfs(pu))) {
46             pair_l[u] = v;
47             pair_r[v] = u;
48             return true;
49         }
50     }
51     dist[u] = INF;
52     return false;
53 };
54
55 int matching = 0;
56 while (bfs()) {
57     for (int u = 0; u < n_1; ++u) {
58         if (pair_l[u] == -1 && dfs(u))
59             ++matching;
60     }
61 }
62 return matching;
63 }

```

4.6 Min-Cost Max-Flow

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 const ll LINF = numeric_limits<ll>::max() / 4;

```

```

6
7 struct MCMFEdge {
8     int to, rev;
9     ll cap, cost;
10 };
11
12 struct MinCostMaxFlow {
13     int n, s, t;
14     ll flow_ = 0, cost_ = 0;
15     vector<vector<MCMFEdge>> adj;
16     vector<ll> pot, dist;
17     vector<int> prevnode, prevedge;
18
19     MinCostMaxFlow(int n, int s, int t) : n(n), s(s), t(t) {
20         adj.assign(n, {});
21         pot.resize(n, 0);
22         dist.resize(n);
23         prevnode.resize(n);
24         prevedge.resize(n);
25     }
26
27     void add_edge(int from, int to, ll cap, ll cost) {
28         MCMFEdge a{to, (int)adj[to].size(), cap, cost};
29         MCMFEdge b{from, (int)adj[from].size(), 0, -cost};
30         adj[from].push_back(a);
31         adj[to].push_back(b);
32     }
33
34     bool dijkstra() {
35         fill(dist.begin(), dist.end(), LINF);
36         fill(prevnode.begin(), prevnode.end(), -1);
37         priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
38
39         dist[s] = 0;
40         pq.emplace(0, s);
41
42         while (!pq.empty()) {
43             auto [d, v] = pq.top();
44             pq.pop();
45             if (d != dist[v]) continue;
46             for (int i = 0; i < (int)adj[v].size(); i++) {
47                 MCMFEdge &e = adj[v][i];
48                 if (e.cap <= 0) continue;
49                 ll ndist = dist[v] + e.cost + pot[v] - pot[e.to];
50                 if (ndist < dist[e.to]) {
51                     dist[e.to] = ndist;
52                     prevnode[e.to] = v;
53                     prevedge[e.to] = i;
54                     pq.emplace(ndist, e.to);
55                 }
56             }
57         }

```

```

58     if (dist[t] == LINF) return false;
59
60     for (int i = 0; i < n; i++) {
61         if (dist[i] < LINF) pot[i] += dist[i];
62     }
63
64     return true;
65 }
66
67 void augment() {
68     ll addflow = LINF;
69     for (int v = t; v != s; v = prevnode[v]) {
70         MCMFEdge &e = adj[prevnode[v]][prevedge[v]];
71         addflow = min(addflow, e.cap);
72     }
73
74     for (int v = t; v != s; v = prevnode[v]) {
75         MCMFEdge &e = adj[prevnode[v]][prevedge[v]];
76         e.cap -= addflow;
77         adj[e.to][e.rev].cap += addflow;
78     }
79
80     flow_ += addflow;
81     cost_ += addflow * pot[t];
82 }
83
84 pair<ll, ll> min_cost_flow(int s, int t) {
85     this->s = s;
86     this->t = t;
87     flow_ = 0;
88     cost_ = 0;
89     fill(pot.begin(), pot.end(), 0);
90
91     while (dijkstra()) {
92         augment();
93     }
94     return {flow_, cost_};
95 }
96
97 ll flow() const { return flow_; }
98 ll cost() const { return cost_; }
99
100 };

```

4.7 Push-Relabel

```

1 /**
2  * Push-Relabel Max-Flow (Highest-Label + Gap Relabeling)
3  * Time:  $O(V^2 \sqrt{E})$  worst case, faster than Dinic on dense
           graphs

```

```

4  * Based on KACTL PushRelabel.h
5  */
6
7 #include <bits/stdc++.h>
8 using namespace std;
9
10 using ll = long long;
11 const ll INF = LLONG_MAX;
12
13 struct PushRelabel {
14     struct Edge {
15         int dest, back;
16         ll f, c;
17     };
18
19     int n;
20     vector<vector<Edge>> g;
21     vector<ll> ec; // excess capacity
22     vector<Edge*> cur; // current edge pointer
23     vector<vector<int>> hs;
24     vector<int> H;
25
26     PushRelabel(int n) : n(n), g(n), ec(n), cur(n), hs(2 * n), H(n) {}
27
28     void add_edge(int u, int v, ll cap) {
29         if (u == v) return;
30         g[u].push_back({v, (int)g[v].size(), 0, cap});
31         g[v].push_back({u, (int)g[u].size() - 1, 0, 0});
32     }
33
34     void addFlow(Edge& e, ll f) {
35         Edge& back = g[e.dest][e.back];
36         if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
37         e.f += f;
38         e.c -= f;
39         ec[e.dest] += f;
40         back.f -= f;
41         back.c += f;
42         ec[back.dest] -= f;
43     }
44
45     ll max_flow(int s, int t) {
46         int v = n;
47         H[s] = v;
48         ec[t] = 1;
49         vector<int> co(2 * v);
50         co[0] = v - 1;
51         for (int i = 0; i < v; i++) cur[i] = g[i].data();
52
53         for (Edge& e : g[s]) addFlow(e, e.c);
54
55         for (int hi = 0;;) {

```

```

56 while (hs[hi].empty()) if (hi-- == 0) return -ec[s];
57 int u = hs[hi].back();
58 hs[hi].pop_back();
59
60 while (ec[u] > 0) { // discharge u
61     if (cur[u] == g[u].data() + (int)g[u].size()) {
62         H[u] = INT_MAX;
63         for (Edge& e : g[u])
64             if (e.c && H[u] > H[e.dest] + 1)
65                 H[u] = H[e.dest] + 1, cur[u] = &e;
66         if (++co[H[u]], !--co[hi] && hi < v)
67             for (int i = 0; i < v; i++)
68                 if (hi < H[i] && H[i] < v)
69                     --co[H[i]], H[i] = v + 1;
70         hi = H[u];
71     } else if (cur[u]->c && H[u] == H[cur[u]->dest] + 1)
72         addFlow(*cur[u], min(ec[u], cur[u]->c));
73     else
74         ++cur[u];
75 }
76 }
77 }
78
79 // Returns true if vertex is on the source side of min cut
80 bool left_of_min_cut(int a) { return H[a] >= n; }
81 };

```

4.8 0-1 BFS

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 vector<ll> zero_one_bfs(int n, int src, const vector<vector<pair<int
6     ,int>>>& adj) {
7     const ll INF = (ll)1e17;
8     vector<ll> dist(n, INF);
9     deque<int> dq;
10    dist[src] = 0;
11    dq.push_back(src);
12
13    while (!dq.empty()) {
14        int u = dq.front();
15        dq.pop_front();
16        for (auto [v, w] : adj[u]) {
17            if (dist[u] + w < dist[v]) {
18                dist[v] = dist[u] + w;
19                if (w == 0) {
20                    dq.push_front(v);
21                } else {

```

```

21                dq.push_back(v);
22            }
23        }
24    }
25    return dist;
26 }
27 }

```

4.9 Biconnected Components

```

1 // Biconnected Components (vertex-biconnected components)
2 // Uses DFS with edge stack; articulation points split components
3
4 #include <algorithm>
5 #include <vector>
6 #include <utility>
7
8 using namespace std;
9
10 int dfsTime = 0;
11 vector<int> disc, low;
12 vector<vector<int>> comps;
13 vector<pair<int,int>> edgeStack;
14 vector<vector<int>> adj;
15
16 // disc[u] = discovery time of u
17 // low[u] = 'low' node of u (lowest discovery time reachable from u)
18
19 void dfsBCC(int u, int parent) {
20     disc[u] = low[u] = ++dfsTime;
21     for (int v : adj[u]) {
22         if (v == parent) continue;
23         if (!disc[v]) {
24             edgeStack.emplace_back(u, v);
25             dfsBCC(v, u);
26             low[u] = min(low[u], low[v]);
27             if (disc[u] <= low[v]) {
28                 // u is an articulation point; pop edges to form a component
29                 comps.push_back({});
30                 while (edgeStack.back() != make_pair(u, v)) {
31                     int a = edgeStack.back().first, b = edgeStack.back().
32                         second;
33                     comps.back().push_back(a);
34                     comps.back().push_back(b);
35                     edgeStack.pop_back();
36                 }
37                 // pop the (u,v) edge
38                 comps.back().push_back(u);
39                 comps.back().push_back(v);
40                 edgeStack.pop_back();

```

```

40     }
41     } else if (disc[v] < disc[u]) {
42         // back edge to ancestor (avoid duplicates in undirected graph)
43         low[u] = min(low[u], disc[v]);
44         edgeStack.emplace_back(u, v);
45     }
46 }
47 }
48
49 vector<vector<int>> biconnected_components(int n, const vector<
    vector<int>>& adj_) {
50     adj = adj_;
51     disc.assign(n, 0);
52     low.assign(n, 0);
53     comps.clear();
54     edgeStack.clear();
55     dfsTime = 0;
56
57     for (int u = 0; u < n; u++) {
58         if (!disc[u]) {
59             dfsBCC(u, u);
60             // remaining edges on stack form a component
61             if (!edgeStack.empty()) {
62                 comps.push_back({});
63                 while (!edgeStack.empty()) {
64                     comps.back().push_back(edgeStack.back().first);
65                     comps.back().push_back(edgeStack.back().second);
66                     edgeStack.pop_back();
67                 }
68             }
69         }
70     }
71
72     // remove duplicate vertices in each component
73     for (auto& comp : comps) {
74         sort(comp.begin(), comp.end());
75         comp.erase(unique(comp.begin(), comp.end()), comp.end());
76     }
77
78     return comps;
79 }

```

4.10 Johnson's Algorithm

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 const ll INF = 1e17;
5

```

```

6 vector<ll> dijkstra(int n, int source, const vector<vector<pair<int,
    ll>>>& adj, const vector<ll>& h) {
7     vector<ll> dist(n, INF);
8     priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<
        ll,int>>> pq;
9     dist[source] = 0;
10    pq.push({0, source});
11
12    while (!pq.empty()) {
13        auto [d, u] = pq.top();
14        pq.pop();
15        if (d > dist[u]) continue;
16
17        for (auto [v, w] : adj[u]) {
18            ll reweighted = w + h[u] - h[v];
19            if (dist[u] + reweighted < dist[v]) {
20                dist[v] = dist[u] + reweighted;
21                pq.push({dist[v], v});
22            }
23        }
24    }
25
26    return dist;
27 }
28
29 vector<vector<ll>> johnson(int n, const vector<vector<pair<int,ll
    >>>& adj) {
30     vector<vector<pair<int,ll>>> aug = adj;
31     for (int i = 0; i < n; i++) {
32         aug[i].push_back({n, 0});
33     }
34
35     vector<ll> dist(n + 1, INF);
36     dist[n] = 0;
37     for (int i = 0; i < n; i++) {
38         bool updated = false;
39         for (int u = 0; u <= n; u++) {
40             if (dist[u] == INF) continue;
41             for (auto [v, w] : aug[u]) {
42                 if (dist[u] + w < dist[v]) {
43                     dist[v] = dist[u] + w;
44                     updated = true;
45                 }
46             }
47         }
48         if (!updated) break;
49     }
50
51     vector<ll> h(n);
52     for (int i = 0; i < n; i++) h[i] = (dist[i] == INF) ? 0 : dist[i]
        ];
53
54     vector<vector<ll>> result(n, vector<ll>(n, INF));

```

```

55     for (int s = 0; s < n; s++) {
56         vector<ll> d = dijkstra(n, s, adj, h);
57         for (int t = 0; t < n; t++) {
58             if (d[t] != INF) {
59                 result[s][t] = d[t] - h[s] + h[t];
60             }
61         }
62     }
63     return result;
64 }

```

5 Graph Theory

5.1 Bellman-Ford

```

1  const ll INF = 1e17;
2  const ll MIN_INF = (-1)*INF;
3
4  vector<ll> dist(MAX, INF);
5
6  struct Edge {
7      int u, v;
8      ll w;
9  };
10
11 void bellmanFord(int V, int E, vector<Edge>& edges, int start) {
12     dist[start] = 0;
13     vector<int> prev(V, -1);
14
15     for (int i = 0; i < V-1; i++){
16         for (int j = 0; j < E; j++){
17             int u = edges[j].u;
18             int v = edges[j].v;
19             ll w = edges[j].w;
20             if (dist[u] == INF) continue;
21             dist[v] = min(dist[v], w+dist[u]);
22             if (dist[v] > dist[u] + w){
23                 dist[v] = max(MIN_INF, dist[u] + w);
24             }
25         }
26     }
27
28     for (int i = 0; i < V-1; i++){
29         for (int j = 0; j < E; j++){
30             int u = edges[j].u;
31             int v = edges[j].v;
32             ll w = edges[j].w;
33             if (dist[u] == INF) continue;
34             dist[v] = max(MIN_INF, dist[v]);

```

```

35         if (dist[u] == MIN_INF or dist[v] > dist[u] + w){
36             dist[v] = MIN_INF;
37         }
38     }
39 }
40
41 }

```

5.2 Dijkstra

```

1  // Old implementation, use global variables.
2
3  const int MAX = 2e5+3;
4
5  #define INF LLONG_MAX
6  typedef pair<ll, ll> pll;
7
8  vector<ll> dijkstra(int n, int source, vector<vector<pll>> &adj) {
9      vector<ll> dist(n, INF);
10     priority_queue<pll, vector<pll>, greater<pll>> pq;
11     dist[source] = 0;
12     pq.push({0, source});
13
14     while (!pq.empty()) {
15         ll d = pq.top().first;
16         ll u = pq.top().second;
17         pq.pop();
18
19         if (d > dist[u]) continue;
20
21         for (auto &edge : adj[u]) {
22             ll v = edge.first;
23             ll weight = edge.second;
24
25             if (dist[u] + weight < dist[v]) {
26                 dist[v] = dist[u] + weight;
27                 pq.push({dist[v], v});
28             }
29         }
30     }
31
32     return dist;
33 }

```

5.3 Dinic

```

1  const int MAX = 503;

```

```

2
3 #define INF LLONG_MAX
4 #define pii pair<int,int>
5 #define fi first
6 #define sc second
7 #define all(x) x.begin(),x.end()
8
9 struct FlowEdge{
10     int u,v;
11     ll flow=0, cap;
12     FlowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap){};
13 };
14
15 struct Dinic{
16     vector<FlowEdge> edges;
17     vector<vector<int>> adj;
18     int n, s, t; // Vertices, Source, Sink
19     int id = 0;
20     vector<int> level, next;
21     queue<int> q;
22
23     Dinic(int n, int s, int t) : n(n), s(s), t(t){
24         adj.resize(n);
25         level.resize(n);
26         next.resize(n);
27         fill(all(level), -1);
28         level[s] = 0;
29         q.push(s);
30     }
31
32     void addEdge(int u, int v, ll cap){
33         edges.emplace_back(u,v,cap);
34         edges.emplace_back(v,u,0);
35         adj[u].push_back(id++);
36         adj[v].push_back(id++);
37     }
38
39     bool bfs(){
40         while(!q.empty()){
41             int u = q.front();
42             q.pop();
43             for(auto e : adj[u]){
44                 // Saturated
45                 if(edges[e].cap - edges[e].flow < 1) continue;
46                 // Visited
47                 if(level[edges[e].v] != -1) continue;
48
49                 level[edges[e].v] = level[edges[e].u] + 1;
50                 q.push(edges[e].v);
51             }
52         }
53     }
54

```

```

55     return level[t] != -1;
56 }
57
58 ll dfs(int u, ll flow){
59     if(flow == 0) return 0;
60     if(u == t) return flow;
61     // cid is current id
62     for(int &cid = next[u]; cid<adj[u].size(); cid++){
63         int e = adj[u][cid];
64         int v = edges[e].v;
65
66         if(level[edges[e].u] + 1 != level[v] or edges[e].cap -
67             edges[e].flow < 1)
68             continue;
69         ll f = dfs(v, min(flow, edges[e].cap - edges[e].flow));
70         if(f == 0) continue;
71
72         edges[e].flow += f;
73         edges[e^1].flow -= f;
74         return f;
75     }
76     return 0;
77 }
78
79 ll maxFlow(){
80     ll flow = 0;
81     while(bfs()){ // While there are aug paths
82         fill(all(next), 0);
83         for(ll f = dfs(s, INF); f!=0; f = dfs(s,INF)) flow+=f;
84         fill(all(level), -1);
85         level[s] = 0;
86         q.push(s);
87     }
88     return flow;
89 }
90
91 vector<pii> minCut(){
92     vector<pii> arns;
93     fill(all(level), -1);
94     level[s] = 0;
95     q.push(s);
96     while(!q.empty()){
97         int u = q.front();
98         q.pop();
99         for(int id = 0; id<adj[u].size(); id++){
100             int e = adj[u][id];
101             if(level[edges[e].v] == -1 and edges[e].cap - edges[
102                 e].flow > 0){
103                 q.push(edges[e].v);
104                 level[edges[e].v] = level[edges[e].u] + 1;
105             }
106         }
107     }
108 }

```

```

106     }
107
108     for(int i=0; i<level.size(); i++){
109         if(level[i] != -1){
110             for(int id=0; id<adj[i].size(); id++){
111                 int e = adj[i][id];
112                 if(level[edges[e].v] == -1 and edges[e].cap -
113                     edges[e].flow == 0)
114                     arns.emplace_back(edges[e].u, edges[e].v);
115             }
116         }
117     }
118     return arns;
119 }
120
121 vector<pii> maxMatching(){
122     vector<pii> arns;
123     fill(all(level), -1); // Reset levels.
124     level[s] = 0;
125     q.push(s);
126     while(!q.empty()){
127         int u = q.front();
128         q.pop();
129         for(int id = 0; id < adj[u].size(); id++){
130             int e = adj[u][id];
131             if(level[edges[e].v] == -1 and edges[e].cap - edges[e].flow == 0 and edges[e].flow != 011){
132                 q.push(edges[e].v);
133                 level[edges[e].v] = level[edges[e].u] + 1;
134             }
135         }
136     }
137
138     for(int i=0; i<level.size(); i++){
139         if(level[i] != -1){
140             for(int id=0; id<adj[i].size(); id++){
141                 int e = adj[i][id];
142                 if(edges[e].u != s and edges[e].v != t and edges[e].cap - edges[e].flow == 0 and edges[e].flow != 011)
143                     arns.emplace_back(edges[e].u, edges[e].v);
144             }
145         }
146     }
147     return arns;
148 }
149
150 };

```

5.4 Directed Cycles

```

1 int n; // number of vertices
2 vector<vector<int>> adj(MAX);
3 stack<int> st;
4 vector<bool> visited(MAX);
5 vector<bool> stFlg(MAX);
6 vector<int> arns;
7 int depth = 0;
8
9 bool dfs(int v) {
10     visited[v] = true;
11     st.push(v);
12     stFlg[v] = true;
13     for(auto &z : adj[v]){
14         if(!visited[z])
15             if(dfs(z)) return true;
16
17         if(stFlg[z]){
18             st.push(z);
19             return true;
20         }
21     }
22
23     st.pop();
24     stFlg[v] = false;
25     return false;
26 }
27
28 void printCycle(){
29     int tmp = st.top();
30     while(!st.empty()){
31         arns.push_back(st.top());
32         st.pop();
33         if(arns.back() == tmp and arns.size() != 1) break;
34     }
35     reverse(arns.begin(), arns.end());
36
37     cout<<arns.size()<<endl;
38     for(auto v: arns) cout<<v<<" ";
39     cout<<endl;
40 }

```

5.5 Edmonds-Karp

```

1 vector<vector<int>> adj(MAX);
2 vector<vector<int>> cap(MAX, vector<int>(MAX));
3
4 int bfs(int s, int t, vector<int> &parent){
5     parent[s] = -2;

```

```

6   queue<pii> q;
7   q.push({s, INF});
8
9   while(!q.empty()){
10      int u = q.front().fi;
11      int flow = q.front().sc;
12      q.pop();
13
14      for(int v : adj[u]){
15         if(parent[v] == -1 and cap[u][v]){
16            parent[v] = u;
17            int new_flow = min(flow, cap[u][v]);
18            if(v == t) return new_flow;
19            q.push({v, new_flow});
20        }
21      }
22   }
23   return 0;
24 }
25
26 int edmond_karp(int s, int t, int n){
27   vector<int> parent(n+1, -1);
28   int flow = 0;
29   int new_flow;
30   while(new_flow = bfs(s, t, parent)){
31      flow += new_flow;
32      int cur = t;
33      while(cur != s){
34         int prev = parent[cur];
35         cap[prev][cur] -= new_flow;
36         cap[cur][prev] += new_flow;
37         cur = prev;
38      }
39      fill(parent.begin(), parent.end(), -1);
40   }
41
42   return flow;
43 }

```

5.6 Floyd-Warshall

```

1   void floydWarshall(vector<vector<ll>> &d, int n){
2       for (int k = 0; k < n; ++k) {
3           for (int i = 0; i < n; ++i) {
4               for (int j = 0; j < n; ++j) {
5                   d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
6               }
7           }
8       }
9   }

```

5.7 Kahn Topological Sort

```

1   def kahnTopoSort(self, adj: List[List[int]]) -> List[int]:
2       #print(adj)
3       in_deg = [0] * len(adj)
4       for i in range(len(adj)):
5           for u in adj[i]:
6               in_deg[u] += 1
7
8       q = []
9       for i in range(len(in_deg)):
10          if in_deg[i] == 0:
11              q.append(i)
12
13       arns = []
14       while len(q) > 0:
15          u = q[0]
16          q.pop(0)
17          arns.append(u)
18
19          for v in adj[u]:
20              in_deg[v] -= 1
21              if in_deg[v] == 0:
22                  q.append(v)
23
24       print(str(len(arns)) + " " + str(len(adj)))
25       if (len(arns) != len(adj)):
26           return []
27
28       return arns

```

5.8 Kruskal

```

1   struct Edge {
2       int u, v, w;
3       bool operator<(Edge const& other) {
4           return w < other.w;
5       }
6   };
7
8   class DisjointSets {
9   private:
10      vector<int> parents;
11      vector<int> rank; // Use rank instead of size
12
13   public:
14      DisjointSets(int size) : parents(size), rank(size, 0) {
15          for (int i = 0; i < size; i++) parents[i] = i;
16      }
17

```

```

18 int find(int x) {
19     if (parents[x] != x) parents[x] = find(parents[x]);
20     return parents[x];
21 }
22
23 bool unite(int x, int y) {
24     int x_root = find(x);
25     int y_root = find(y);
26     if (x_root == y_root) return false;
27
28     if (rank[x_root] < rank[y_root]) swap(x_root, y_root);
29     parents[y_root] = x_root;
30     if (rank[x_root] == rank[y_root]) rank[x_root]++;
31     return true;
32 }
33 };
34
35 int kruskal(int n, vector<Edge> &edges, vector<Edge> &ans){
36     ll cost = 0;
37     int ed_cnt = 0;
38     DisjointSets dsu(n);
39     sort(edges.begin(), edges.end());
40
41     for(Edge e : edges){
42         if(dsu.unite(e.u, e.v)){
43             ans.push_back(e);
44             cost += e.w;
45             ed_cnt++;
46         }
47     }
48
49     if (ed_cnt != n - 1) return -1;
50     return cost;
51 }
52 }

```

5.9 Lava Flow

```

1 struct Cell{
2     int x,y,t;
3 };
4
5 const int MAX = 1005;
6 int n,m;
7
8 char maze[MAX][MAX];
9 int vis[MAX][MAX];
10 int player[MAX][MAX];
11 char path[MAX][MAX];
12 set<pii> isExit;

```

```

13 queue<Cell> q;
14 string res;
15
16 bool isValid(int x, int y){
17     if(x < 0 || x >= n || y < 0 || y >= m) return false;
18     if(maze[x][y] == '#') return false;
19     return true;
20 }
21
22 bool isSafe(int x, int y, int u, int v){
23     return player[x][y] == -1 and maze[x][y] != 'M' and (vis[x][y]
24         == -1 or player[u][v] + 1 < vis[x][y]);
25 }
26
27 void restorePath(int u, int v, int x, int y){
28     while (x != u || y != v) {
29         res.push_back(path[u][v]);
30
31         if (res.back() == 'U') u++;
32         if (res.back() == 'D') u--;
33         if (res.back() == 'L') v++;
34         if (res.back() == 'R') v--;
35     }
36 }
37
38 bool lavaFlow(int x,int y){
39     q.push({x,y,1});
40     player[x][y] = 0;
41
42     while(!q.empty()){
43         int u = q.front().x;
44         int v = q.front().y;
45         int t = q.front().t;
46
47         q.pop();
48
49         vector<pii> dir = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
50
51         for(auto it: dir){
52             int i = u+it.fs;
53             int j = v+it.sc;
54
55             if(isValid(i,j)){
56                 if(t == 0){
57                     if(vis[i][j] == -1){
58                         vis[i][j] = vis[u][v]+1;
59                         q.push(Cell{i,j,0});
60                     }
61                 }else{
62                     if(isSafe(i,j,u,v)){
63                         path[i][j] = (it.fs == 1) ? 'D' : (it.fs ==
64                             -1) ? 'U' : (it.sc == 1) ? 'R' : 'L';

```

```

64         player[i][j] = player[u][v]+1;
65         q.push(Cell{i,j,1});
66         if (isExit.find({i,j}) != isExit.end()) {
67             if (player[i][j] < vis[i][j] || vis[i][j]
68                 ] == -1) {
69                 restorePath(i, j, x, y);
70                 return true;
71             }
72         }
73     }
74 }
75
76 }
77 }
78 return false;
79 }
80 }

```

```

30         int curr = x;
31         for (int i = 0; i < V; ++i)
32             curr = prev[curr];
33
34         vector<int> cycle;
35         for (int x = curr; x = prev[x]) {
36             cycle.push_back(x);
37             if (x == curr and cycle.size() > 1) break;
38         }
39
40         reverse(cycle.begin(), cycle.end());
41
42         for(int v : cycle)
43             cout<<v+1<<" ";
44         //cout<<cycle[0]+1<<endl;
45     }
46 }
47
48 }
49 }
50
51
52 void solve(){
53     int n,m; cin>>n>>m;
54     vector<Edge> edges;
55     for(int i=0; i<m; i++){
56         int u,v; ll w; cin>>u>>v>>w;
57         edges.push_back({u-1,v-1,w});
58     }
59
60     int dummyNode = n;
61     for (int i = 0; i < n; ++i) {
62         edges.push_back({dummyNode, i, 0LL});
63     }
64
65     bellmanFord(n + 1, m + n, edges, dummyNode);
66 }
67
68 }

```

5.10 Negative Cycles

```

1 struct Edge {
2     int u, v;
3     ll w;
4 };
5
6 void bellmanFord(int V, int E, vector<Edge>& edges, int start) {
7     vector<ll> dist(V, INF);
8     dist[start] = 0;
9     vector<int> prev(V, -1);
10
11     int x;
12     for (int i = 0; i < V; i++){
13         x = -1;
14         for (int j = 0; j < E; j++){
15             int u = edges[j].u;
16             int v = edges[j].v;
17             ll weight = edges[j].w;
18             if (dist[u] != INF && dist[u] + weight < dist[v]) {
19                 dist[v] = max(-INF, (ll) dist[u] + weight);
20                 prev[v] = u;
21                 x = v;
22             }
23         }
24     }
25
26     if(x == -1){
27         cout<<"NO"<<endl; // No negative Cycle
28     }else{
29         cout<<"YES"<<endl;

```

5.11 Prim Algorithm

```

1 const int MAX = 2e5+3;
2
3 #define INF LLONG_MAX
4 typedef pair<ll, ll> pll;
5
6 ll prim(int V, int E, vector<vector<pll>> &adj) {
7
8     priority_queue<pll, vector<pll>, greater<pll>> pq;

```

```

9
10 vector<bool> visited(V, false);
11
12 ll res = 0;
13
14 pq.push({0, 0});
15
16 while(!pq.empty()){
17     auto p = pq.top();
18     pq.pop();
19
20     int wt = p.first;
21     int u = p.second;
22
23     if(visited[u] == true){
24         continue;
25     }
26
27     res += wt;
28     visited[u] = true;
29
30     for(auto v : adj[u]){
31         if(visited[v.first] == false){
32             pq.push({v.second, v.first});
33         }
34     }
35 }
36
37 for(int i=0; i<V; i++){
38     if(!visited[i])
39         return -1;
40 }
41
42 return res;
43 }

```

```

13 // Calculate vertex order
14 void dfsA(int s){
15     visited[s] = true;
16     for(int u : adj[s]){
17         if(!visited[u])
18             dfsA(u);
19     }
20     order.push_back(s);
21 }
22
23 // Set its component id to every vertex
24 void dfsB(int s, int c){
25     visited[s] = true;
26     for(int u: t_adj[s]){
27         if(!visited[u])
28             dfsB(u,c);
29     }
30     v_id[s] = c;
31 }
32
33
34 void solve(){
35     int n,m; cin>>n>>m;
36
37     for(int i=0; i<m; i++){
38         int u,v; cin>>u>>v;
39         adj[u].push_back(v);
40         t_adj[v].push_back(u);
41     }
42
43     for(int i=1; i<=n; i++){
44         if(!visited[i])
45             dfsA(i);
46     }
47
48     for(int i=1; i<=n; i++)
49         visited[i] = false;
50
51     reverse(order.begin(),order.end());
52
53     int c = 0;
54     for(int i=0; i<order.size(); i++){
55         if(!visited[order[i]]){
56             c++;
57             dfsB(order[i],c);
58         }
59     }
60
61     if(c == 1){
62         cout<<"YES"<<endl;
63     }else{
64         bool flag1 = false, flag2 = false;
65         cout<<"NO"<<endl;

```

5.12 Strongly Connected Components

```

1
2 const int MAX = 2e5+3;
3
4 #define INF LLONG_MAX
5 typedef pair<ll, ll> pll;
6
7 vector<vector<int>> adj(MAX);
8 vector<vector<int>> t_adj(MAX);
9 vector<int> v_id(MAX);
10 bool visited[MAX];
11 vector<int> order; // Append the respective nodes with dfsA order
12

```

```

66     int arns[2];
67     for(int i=1; i<=n; i++){
68         if(v_id[i] == 1 and flag1 == false){
69             flag1 = true;
70             arns[0] = i;
71         }
72
73         if(v_id[i] == 2 and flag2 == false){
74             flag2 = true;
75             arns[1] = i;
76         }
77
78         if(flag1 == true and flag2 ==true) break;
79     }
80     cout<<arns[1]<<" "<<arns[0]<<endl;
81 }
82 }

```

5.13 Topological Sort

```

1 void dfs(int v) {
2     visited[v] = true;
3     for (int u : adj[v]) {
4         if (!visited[u])
5             dfs(u);
6     }
7     ans.push_back(v);
8 }
9
10 void topological_sort() {
11     visited.assign(n+1, false);
12     ans.clear();
13     for (int i = 1; i <= n; ++i) {
14         if (!visited[i]) {
15             dfs(i);
16         }
17     }
18     reverse(ans.begin(), ans.end());
19 }

```

5.14 2-SAT

```

1 // Kosaraju Algorithm
2 struct TwoSAT{
3     int vars;
4     int n;
5     vector<vector<int>> adj, adj_t;

```

```

6     vector<bool> used;
7     vector<int> order, comp;
8     vector<bool> values;
9
10    TwoSAT(int _vars) :
11        vars(_vars),
12        n(2 * vars),
13        adj(n),
14        adj_t(n),
15        used(n),
16        order(),
17        comp(n,-1),
18        values(n){
19        order.reserve(n);
20    }
21
22    void dfsA(int s){
23        used[s] = true;
24        for(int u : adj[s]){
25            if(!used[u]) dfsA(u);
26        }
27        order.push_back(s);
28    }
29
30    void dfsB(int s, int sc){
31        comp[s] = sc;
32        for(int u : adj_t[s]){
33            if(comp[u] == -1) dfsB(u,sc);
34        }
35    }
36
37    bool twoSAT(){
38        order.clear();
39        used.assign(n, false);
40
41        for(int i=0; i<n; i++){
42            if(!used[i]) dfsA(i);
43        }
44
45        comp.assign(n, -1);
46
47        for(int i=0, j=0; i<n; i++){
48            int v = order[n-i-1];
49            if(comp[v] == -1) dfsB(v, j++);
50        }
51
52        values.assign(vars, false);
53
54        for(int i=0; i<n; i+=2){
55            if(comp[i] == comp[i+1]) return false;
56            values[i/2] = comp[i] > comp[i+1];
57        }
58        return true;

```

```

59     }
60
61     void add_disjunction(int a, bool na, int b, bool nb){
62         a = 2* a ^ na;
63         b = 2* b ^ nb;
64
65         int not_a = a^1;
66         int not_b = b^1;
67
68         adj[not_a].push_back(b);
69         adj[not_b].push_back(a);
70         adj_t[b].push_back(not_a);
71         adj_t[a].push_back(not_b);
72     }
73 };

```

5.15 Undirected Cycles

```

1  bool dfs(int v, int par) { // passing vertex and its parent vertex
2      visited[v] = true;
3      for (int u : adj[v]) {
4          if(u == par) continue; // skipping edge to parent vertex
5          if (visited[u]) {
6              cycle_end = v;
7              cycle_start = u;
8              return true;
9          }
10         parent[u] = v;
11         if (dfs(u, parent[u]))
12             return true;
13     }
14     return false;
15 }
16
17 void find_cycle() {
18     visited.assign(n+1, false);
19     parent.assign(n+1, -1);
20     cycle_start = -1;
21
22     for (int v = 0; v < n; v++) {
23         if (!visited[v] && dfs(v, parent[v]))
24             break;
25     }
26
27     if (cycle_start == -1) {
28         cout << "IMPOSSIBLE" << endl;
29     } else {
30         vector<int> cycle;
31         cycle.push_back(cycle_start);
32         for (int v = cycle_end; v != cycle_start; v = parent[v])

```

```

33         cycle.push_back(v);
34         cycle.push_back(cycle_start);
35
36         cout << cycle.size() << endl;
37         for (int v : cycle)
38             cout << v << " ";
39         cout << endl;
40     }
41 }

```

5.16 Articulation Points

```

1  // adj[u] = adjacent nodes of u
2  // ap = AP = articulation points
3  // p = parent
4  // disc[u] = discovery time of u
5  // low[u] = 'low' node of u
6
7  int dfsAP(int u, int p) {
8      int children = 0;
9      low[u] = disc[u] = ++Time;
10     for (int& v : adj[u]) {
11         if (v == p) continue; // we don't want to go back through the
12                                 // same path.
13                                 // if we go back is because we found
14                                 // another way back
15         if (!disc[v]) { // if V has not been discovered before
16             children++;
17             dfsAP(v, u); // recursive DFS call
18             if (disc[u] <= low[v]) // condition #1
19                 ap[u] = 1;
20             low[u] = min(low[u], low[v]); // low[v] might be an ancestor
21                                         // of u
22         } else // if v was already discovered means that we found an
23                 // ancestor
24             low[u] = min(low[u], disc[v]); // finds the ancestor with the
25                                         // least discovery time
26     }
27     return children;
28 }
29
30 void AP() {
31     ap = low = disc = vector<int>(adj.size());
32     Time = 0;
33     for (int u = 0; u < adj.size(); u++)
34         if (!disc[u])
35             ap[u] = dfsAP(u, u) > 1; // condition #2
36 }

```

5.17 Bridges

```

1 // br = bridges, p = parent
2
3 vector<pair<int, int>> br;
4
5 int dfsBR(int u, int p) {
6     low[u] = disc[u] = ++Time;
7     for (int& v : adj[u]) {
8         if (v == p) continue; // we don't want to go back through the
9                               // same path.
10                                // if we go back is because we found
11                                // another way back
12         if (!disc[v]) { // if v has not been discovered before
13             dfsBR(v, u); // recursive DFS call
14             if (low[v] > disc[u]) // condition to find a bridge
15                 br.push_back({u, v});
16             low[u] = min(low[u], low[v]); // low[v] might be an ancestor
17                                         // of u
18         } else // if v was already discovered means that we found an
19             // ancestor
20             low[u] = min(low[u], disc[v]); // finds the ancestor with the
21                                         // least discovery time
22     }
23 }
24
25 void BR() {
26     low = disc = vector<int>(adj.size());
27     Time = 0;
28     for (int u = 0; u < adj.size(); u++)
29         if (!disc[u])
30             dfsBR(u, u);
31 }

```

5.18 Bridges Online

```

1 vector<int> par, dsu_2ecc, dsu_cc, dsu_cc_size;
2 int bridges;
3 int lca_iteration;
4 vector<int> last_visit;
5
6 void init(int n) {
7     par.resize(n);
8     dsu_2ecc.resize(n);
9     dsu_cc.resize(n);
10    dsu_cc_size.resize(n);
11    lca_iteration = 0;
12    last_visit.assign(n, 0);
13    for (int i=0; i<n; ++i) {
14        dsu_2ecc[i] = i;

```

```

15        dsu_cc[i] = i;
16        dsu_cc_size[i] = 1;
17        par[i] = -1;
18    }
19    bridges = 0;
20 }
21
22 int find_2ecc(int v) {
23     if (v == -1)
24         return -1;
25     return dsu_2ecc[v] == v ? v : dsu_2ecc[v] = find_2ecc(dsu_2ecc[v]);
26 }
27
28 int find_cc(int v) {
29     v = find_2ecc(v);
30     return dsu_cc[v] == v ? v : dsu_cc[v] = find_cc(dsu_cc[v]);
31 }
32
33 void make_root(int v) {
34     int root = v;
35     int child = -1;
36     while (v != -1) {
37         int p = find_2ecc(par[v]);
38         par[v] = child;
39         dsu_cc[v] = root;
40         child = v;
41         v = p;
42     }
43     dsu_cc_size[root] = dsu_cc_size[child];
44 }
45
46 void merge_path (int a, int b) {
47     ++lca_iteration;
48     vector<int> path_a, path_b;
49     int lca = -1;
50     while (lca == -1) {
51         if (a != -1) {
52             a = find_2ecc(a);
53             path_a.push_back(a);
54             if (last_visit[a] == lca_iteration){
55                 lca = a;
56                 break;
57             }
58             last_visit[a] = lca_iteration;
59             a = par[a];
60         }
61         if (b != -1) {
62             b = find_2ecc(b);
63             path_b.push_back(b);
64             if (last_visit[b] == lca_iteration){
65                 lca = b;
66                 break;

```

```

67     }
68     last_visit[b] = lca_iteration;
69     b = par[b];
70 }
71
72 }
73
74 for (int v : path_a) {
75     dsu_2ecc[v] = lca;
76     if (v == lca)
77         break;
78     --bridges;
79 }
80 for (int v : path_b) {
81     dsu_2ecc[v] = lca;
82     if (v == lca)
83         break;
84     --bridges;
85 }
86 }
87
88 void add_edge(int a, int b) {
89     a = find_2ecc(a);
90     b = find_2ecc(b);
91     if (a == b)
92         return;
93
94     int ca = find_cc(a);
95     int cb = find_cc(b);
96
97     if (ca != cb) {
98         ++bridges;
99         if (dsu_cc_size[ca] > dsu_cc_size[cb]) {
100             swap(a, b);
101             swap(ca, cb);
102         }
103         make_root(a);
104         par[a] = dsu_cc[a] = b;
105         dsu_cc_size[cb] += dsu_cc_size[a];
106     } else {
107         merge_path(a, b);
108     }
109 }

```

5.19 Bridge Tree

```

1 vector<pair<int, int>> g[MAXN];
2 bool used[MAXN], isBridge[MAXN];
3 int comp[MAXN], tin[MAXN], minAncestor[MAXN];
4

```

```

5 vector<int> tree[MAXN]; // Store 2-edge-connected component tree. (
6     Bridge tree).
7
8 void dfs(int v, int p) {
9     tin[v] = minAncestor[v] = ++timer;
10    used[v] = 1;
11    for(auto &e: g[v]) {
12        int to, id;
13        tie(to, id) = e;
14        if(to == p) continue;
15        if(used[to]) {
16            minAncestor[v] = min(minAncestor[v], tin[to]);
17        } else {
18            dfs(to, v);
19            minAncestor[v] = min(minAncestor[v], minAncestor[to]);
20            if(minAncestor[to] > tin[v]) {
21                isBridge[id] = true;
22            }
23        }
24    }
25
26 void dfs1(int v, int p) {
27     used[v] = 1;
28     comp[v] = compid;
29     for(auto &e: g[v]) {
30         int to, id;
31         tie(to, id) = e;
32
33         if(isBridge[id]) { // avoid traversing from this edge. so we
34             // get full component.
35             continue;
36         }
37         if(used[to]) {
38             continue;
39         }
40         dfs1(to, v);
41     }
42 }
43
44 vector<pair<int, int>> edges;
45
46 void addEdge(int from, int to, int id) {
47     g[from].push_back({to, id});
48     g[to].push_back({from, id});
49     edges[id] = {from, to};
50 }
51
52 void initB() {
53     for(int i = 0; i <= compid; ++i)
54         tree[i].clear();
55     for(int i = 1; i <= N; ++i)

```

```

56     used[i] = false;
57     for(int i = 1; i <= M; ++i)
58         isBridge[i] = false;
59
60     timer = 0;
61     compid = 0;
62 }
63
64 void bridge_tree() {
65
66     initB();
67
68     dfs(1, -1); //Assuming graph is connected.
69
70     for(int i = 1; i <= N; ++i)
71         used[i] = 0;
72
73     for(int i = 1; i <= N; ++i) {
74         if(!used[i]) {
75             dfs1(i, -1);
76             ++compid;
77         }
78     }
79
80     for(int i = 1; i <= M; ++i) {
81         if(isBridge[i]) {
82             int u, v;
83             tie(u, v) = edges[i];
84             // connect two componets using edge.
85             tree[comp[u]].push_back(comp[v]);
86             tree[comp[v]].push_back(comp[u]);
87         }
88     }
89 }
90
91 void init() {
92     edges.clear(); edges.resize(M + 1);
93     for(int i = 1; i <= N; ++i)
94         g[i].clear();
95 }

```

```

7     query(int citya, int cityb, int l, int r, int id): citya(citya),
        cityb(cityb), l(l), r(r),
8         id(id)
9     {}
10 };
11
12 class DSU{
13     private:
14         vector<int> leader;
15         vector<int> sz;
16         int n;
17
18         int getLeader(int a){
19             if(leader[a]==a) return a;
20             leader[a]=getLeader(leader[a]);
21             return leader[a];
22         }
23
24     public:
25         DSU(int n): n(n){
26             leader=vector<int> (n+1);
27             sz=vector<int> (n+1,1);
28             FO(i,n+1){
29                 leader[i]=i;
30             }
31         }
32
33         void clear(){
34             leader=vector<int> (n+1);
35             sz=vector<int> (n+1,1);
36             FO(i,n+1){
37                 leader[i]=i;
38             }
39         }
40
41         void unite(int a,int b){
42             a=getLeader(a);
43             b=getLeader(b);
44             if(a==b) return;
45             if(sz[a]<sz[b]){
46                 swap(a,b);
47             }
48             leader[b]=a;
49             sz[a]+=sz[b];
50         }
51
52         bool isConnected(int a,int b){
53             return getLeader(a)==getLeader(b);
54         }
55     };
56 };
57
58

```

5.20 Dynamic Connectivity

```

1 const int BS=18;
2 //ascii https://elcodigoascii.com.ar/
3
4
5 struct query{
6     int citya,cityb,l,r,id;

```

```

59 inline void solve()
60 {
61     int n,m,q; cin>>n>>m>>q;
62     vector<vector<query>> timeline(m);
63     vector<vector<query>> nxt(m);
64     vector<pair<int,int>> edges(m);
65     vector<int> res(q,-2);
66     DSU dj(n);
67     FO(i,m){
68         int a,b; cin>>a>>b;
69         edges[i]={a,b};
70     }
71
72     FO(i,q){
73         int a,b; cin>>a>>b;
74         if(a==b) res[i]=-1;
75         else timeline[(m-1)/2].push_back({a,b,0,m-1,i});
76     }
77     FO(i,BS){
78         dj.clear();
79         FO(i,m) nxt[i].clear();
80         for(int j=0;j<m;j++)
81         {
82             dj.unite(edges[j].X,edges[j].Y);
83
84             for(query &q:timeline[j])
85             {
86                 if(q.l>q.r) continue;
87                 if(dj.isConnected(q.citya,q.cityb)){
88
89                     res[q.id]=j;
90                     int mid=(j-1+q.l)/2;
91                     nxt[mid].push_back(query(q.citya,q.cityb,q.l,j
92                                     -1,q.id));
93
94                     else{
95                         int mid=(j+1+q.r)/2;
96                         nxt[mid].push_back(query(q.citya,q.cityb,j+1,q.r
97                                     ,q.id));
98                     }
99                 }
100             }
101             timeline.swap(nxt);
102         }
103         for(int r:res){
104             cout<<r+1<<endl;
105         }
106     }

```

5.21 Eulerian Path

```

1 int n, m;
2 vector<vector<int>> g;
3 vector<int> in, out, path;
4
5 // Undirected
6
7 int n, m;
8 vector<vector<pair<int, int>>> g;
9 vector<int> path;
10 vector<bool> seen;
11
12 void dfs(int node) {
13     while (!g[node].empty()) {
14         auto [son, idx] = g[node].back();
15         g[node].pop_back();
16         if (seen[idx]) { continue; }
17         seen[idx] = true;
18         dfs(son);
19     }
20     path.push_back(node);
21 }
22
23 // Directed
24 void dfs(int node) {
25     while (!g[node].empty()) {
26         int son = g[node].back();
27         g[node].pop_back();
28         dfs(son);
29     }
30     path.push_back(node);
31 }

```

5.22 Gale-Shapley

```

1 // Checks if woman 'w' prefers 'm1' over 'm'
2 bool wPrefersM1OverM(vector<vector<int>> &prefer, int w, int m, int
3     m1)
4 {
5     int N = prefer[0].size();
6     for (int i = 0; i < N; i++)
7     {
8         // If m1 comes before m, w prefers
9         // her current engagement
10        if (prefer[w][i] == m1)
11            return true;
12
13        // If m comes before m1, w prefers m
14        if (prefer[w][i] == m)
15            return false;
16    }

```

```

15     }
16 }
17
18 // Implements the stable marriage algorithm
19 vector<int> stableMarriage(vector<vector<int>> &prefer)
20 {
21     int N = prefer[0].size();
22
23     // Stores women's partners
24     vector<int> wPartner(N, -1);
25
26     // Tracks free men
27     vector<bool> mFree(N, false);
28     int freeCount = N;
29
30     while (freeCount > 0)
31     {
32         int m;
33         for (m = 0; m < N; m++)
34             if (!mFree[m])
35                 break;
36
37         // Process each woman in m's preference list
38         for (int i = 0; i < N && !mFree[m]; i++)
39         {
40             int w = prefer[m][i];
41             if (wPartner[w - N] == -1)
42             {
43                 // Engage m and w if w is free
44                 wPartner[w - N] = m;
45                 mFree[m] = true;
46                 freeCount--;
47             }
48             else
49             {
50                 int m1 = wPartner[w - N];
51                 // If w prefers m over her current partner, reassign
52                 if (!wPrefersM1OverM(prefer, w, m, m1))
53                 {
54                     wPartner[w - N] = m;
55                     mFree[m] = true;
56                     mFree[m1] = false;
57                 }
58             }
59         }
60     }
61     return wPartner;
62 }

```

5.23 Hungarian Algorithm

```

1 vector<int> u (n+1), v (m+1), p (m+1), way (m+1);
2 for (int i=1; i<=n; ++i) {
3     p[0] = i;
4     int j0 = 0;
5     vector<int> minv (m+1, INF);
6     vector<bool> used (m+1, false);
7     do {
8         used[j0] = true;
9         int i0 = p[j0], delta = INF, j1;
10        for (int j=1; j<=m; ++j)
11            if (!used[j]) {
12                int cur = A[i0][j]-u[i0]-v[j];
13                if (cur < minv[j])
14                    minv[j] = cur, way[j] = j0;
15                if (minv[j] < delta)
16                    delta = minv[j], j1 = j;
17            }
18        for (int j=0; j<=m; ++j)
19            if (used[j])
20                u[p[j]] += delta, v[j] -= delta;
21        else
22            minv[j] -= delta;
23        j0 = j1;
24    } while (p[j0] != 0);
25    do {
26        int j1 = way[j0];
27        p[j0] = p[j1];
28        j0 = j1;
29    } while (j0);
30 }
31
32 vector<int> ans (n+1);
33 for (int j=1; j<=m; ++j)
34     ans[p[j]] = j;
35
36 int cost = -v[0];

```

5.24 SPFA

```

1 const int INF = 1000000000;
2 vector<vector<pair<int, int>>> adj;
3
4 bool spfa(int s, vector<int>& d) {
5     int n = adj.size();
6     d.assign(n, INF);
7     vector<int> cnt(n, 0);
8     vector<bool> inqueue(n, false);
9     queue<int> q;
10

```

```

11     d[s] = 0;
12     q.push(s);
13     inqueue[s] = true;
14     while (!q.empty()) {
15         int v = q.front();
16         q.pop();
17         inqueue[v] = false;
18
19         for (auto edge : adj[v]) {
20             int to = edge.first;
21             int len = edge.second;
22
23             if (d[v] + len < d[to]) {
24                 d[to] = d[v] + len;
25                 if (!inqueue[to]) {
26                     q.push(to);
27                     inqueue[to] = true;
28                     cnt[to]++;
29                     if (cnt[to] > n)
30                         return false; // negative cycle
31                 }
32             }
33         }
34     }
35     return true;
36 }

```

5.25 Tarjan

```

1  /** Takes in an adjacency list and calculates the SCCs of the graph. */
2  class TarjanSolver {
3  private:
4      vector<vector<int>>> rev_adj;
5      vector<int> post;
6      vector<int> comp;
7
8      vector<bool> visited;
9      int timer = 0;
10     int id = 0;
11
12     void fill_post(int at) {
13         visited[at] = true;
14         for (int n : rev_adj[at]) {
15             if (!visited[n]) { fill_post(n); }
16         }
17         post[at] = timer++;
18     }
19
20     void find_comp(int at) {

```

```

21         visited[at] = true;
22         comp[at] = id;
23         for (int n : adj[at]) {
24             if (!visited[n]) { find_comp(n); }
25         }
26     }
27
28     public:
29     const vector<vector<int>>> &adj;
30
31     TarjanSolver(const vector<vector<int>>> &adj)
32         : adj(adj), rev_adj(adj.size()), post(adj.size()), comp(adj.
33           size()),
34           visited(adj.size()) {
35         vector<int> nodes(adj.size());
36         for (int n = 0; n < adj.size(); n++) {
37             nodes[n] = n;
38             for (int next : adj[n]) { rev_adj[next].push_back(n); }
39         }
40         for (int n = 0; n < adj.size(); n++) {
41             if (!visited[n]) { fill_post(n); }
42         }
43         std::sort(nodes.begin(), nodes.end(),
44                   [&](int n1, int n2) { return post[n1] > post[n2]; });
45
46         visited.assign(adj.size(), false);
47         for (int n : nodes) {
48             if (!visited[n]) {
49                 find_comp(n);
50                 id++;
51             }
52         }
53     }
54
55     int comp_num() const { return id; }
56
57     int get_comp(int n) const { return comp[n]; }
58 };

```

5.26 Sqrt Decomposition

```

1  /** Sqrt Decomposition -- range sum queries & point updates in O((N+Q)*sqrt(N)) */
2  #include <vector>
3  #include <cmath>
4
5  using ll = long long;
6
7  template <typename T>

```

```

8 class SqrtDecomp {
9     int n, sz;
10    std::vector<T> a;
11    std::vector<ll> block;
12
13 public:
14    SqrtDecomp(const std::vector<T>& arr) {
15        n = (int)arr.size();
16        sz = (int)std::sqrt(n) + 1;
17        a = arr;
18        block.assign((n + sz - 1) / sz, 0);
19        for (int i = 0; i < n; i++)
20            block[i / sz] += arr[i];
21    }
22
23    // point update: set a[i] = val
24    void update(int i, T val) {
25        block[i / sz] += val - a[i];
26        a[i] = val;
27    }
28
29    // inclusive range query [l, r]
30    ll query(int l, int r) const {
31        ll res = 0;
32        int start = l / sz, end = r / sz;
33        if (start == end) {
34            for (int i = l; i <= r; i++)
35                res += a[i];
36            return res;
37        }
38        // left partial block
39        for (int i = l; i < (start + 1) * sz; i++)
40            res += a[i];
41        // full blocks
42        for (int b = start + 1; b < end; b++)
43            res += block[b];
44        // right partial block
45        for (int i = end * sz; i <= r; i++)
46            res += a[i];
47        return res;
48    }
49 };

```

```

3    vector<vector<int>> segtree;
4    int n;
5    int m;
6    int d1;
7    int d2;
8
9 public:
10
11    void build(vector<vector<int>> &a){
12        n = a.size();
13        m = a[0].size();
14        d1=1;
15        while(d1<n) d1<<=1;
16
17        d2=1;
18        while(d2<m) d2<<=1;
19
20        segtree.assign(2*d1, vector<int>(2*d2,0));
21
22        // Set leaves
23        for(int i=0; i<n; i++)
24            for(int j=0; j<m; j++)
25                segtree[d1+i][d2+j] = a[i][j];
26
27        // Propagate on columns
28        for(int i=d1; i<2*d1; i++){
29            for(int j=d2-1; j>0; j--){
30                segtree[i][j] = segtree[i][2*j] + segtree[i][2*j+1];
31            }
32
33        // Propagate on rows
34        for(int i=d1-1; i>0; i--){
35            for(int j=1; j<2*d2; j++){
36                segtree[i][j] = segtree[2*i][j] + segtree[2*i+1][j];
37            }
38        }
39
40    void update(int x, int y, int k){
41        int r = x + d1;
42        int c = y + d2;
43
44        segtree[r][c] = k;
45
46        // Update columns for the row corresponding to x
47        for(int j=c>>1; j>0; j>>=1)
48            segtree[r][j] = segtree[r][2*j] + segtree[r][2*j+1];
49
50
51        for(int i=r>>1; i>0; i>>=1){
52            segtree[i][c] = segtree[2*i][c] + segtree[2*i+1][c];
53            for(int j=c>>1; j>0; j>>=1)
54                segtree[i][j] = segtree[i][2*j] + segtree[i][2*j+1];
55        }

```

6 Data Structures

6.1 2D Segment Tree

```

1 class SegmentTree2D{
2 private:

```

```

56     }
57
58
59
60     int query(int x1, int y1, int x2, int y2){
61         int lx = x1+d1, rx = x2+d1+1;
62
63         ll arns = 0;
64         while(lx<rx) {
65             if(lx & 1){
66                 arns += query_y(lx, y1, y2);
67                 ++lx;
68             }
69             if (rx & 1){
70                 --rx;
71                 arns += query_y(rx, y1, y2);
72             }
73             lx >>= 1; rx >>= 1;
74         }
75         return arns;
76     }
77
78     int query_y(int row, int y1, int y2) {
79         int ly = y1 + d2, ry = y2 + 1 + d2;
80         int s = 0;
81         while (ly < ry) {
82             if(ly & 1) s+=segtree[row][ly++];
83             if(ry & 1) s+=segtree[row][--ry];
84             ly >>= 1; ry >>= 1;
85         }
86         return s;
87     }
88 }
89 };

```

6.2 Bit Trie

```

1  class Trie{
2  private:
3      Trie *children[2];
4      bool term_st; // is_end
5
6      Trie* searchPrefix(int x){
7          Trie* node = this;
8
9          for(int i=29; i>=0; i--){
10             int bit = (x>>i)&1;
11             if(!node->children[bit]) return nullptr;
12             node = node->children[bit];
13         }

```

```

14         return node;
15     }
16 }
17
18 public:
19     Trie() : term_st(false) {
20         for(int i=0; i<2; i++)
21             children[i] = nullptr;
22     }
23
24     void insert(int x) {
25         Trie* node = this;
26         for(int i=29; i>=0; i--){
27             int bit = (x>>i)&1;
28             if(!node->children[bit]) node->children[bit] = new Trie
29                 ();
30             node = node->children[bit];
31         }
32         node->term_st = true;
33     }
34
35     bool search(int x) {
36         Trie* node = searchPrefix(x);
37         return node != nullptr && node->term_st;
38     }
39
40     bool startsWith(int x) {
41         Trie* node = searchPrefix(x);
42         return node != nullptr;
43     }
44
45     int maxXor(int x){
46         Trie* node = this;
47         int arns = 0;
48         for(int i=29; i>=0; i--){
49             int bit = (x>>i)&1;
50             int not_bit = 1-bit;
51             if(node->children[not_bit]){
52                 arns|=(1<<i);
53                 node = node->children[not_bit];
54             }else{
55
56                 node = node->children[bit];
57             }
58         }
59
60         return arns;
61     }
62 };

```

6.3 Disjoint Set Union

```

1 class DisjointSets{
2     private:
3         vector<int> parents;
4         vector<int> sizes;
5         int components;
6     public:
7         DisjointSets(int size) : parents(size), sizes(size,1),
8             components(size){
9             for(int i=0; i<size; i++){parents[i] = i;}
10        }
11        int find(int x) {return parents[x] == x ? x : (parents[x] =
12            find(parents[x]));}
13        bool unite(int x, int y){
14            int x_root = find(x);
15            int y_root = find(y);
16
17            if(x_root == y_root) {return false;}
18
19            if(sizes[x_root] < sizes[y_root]) {swap(x_root,y_root);}
20            sizes[x_root] += sizes[y_root];
21            parents[y_root] = x_root;
22            components--;
23            return true;
24        }
25
26        vector<int> getAllComponentSizes(){
27            map<int, int> component_sizes;
28            for (int i = 0; i < parents.size(); ++i){
29                int root = find(i);
30                if (component_sizes.find(root) == component_sizes.
31                    end()){
32                    component_sizes[root] = sizes[root];
33                }
34            }
35            vector<int> result;
36            for (auto& [root, size] : component_sizes) {
37                result.push_back(size);
38            }
39            return result;
40        }
41
42        bool connected(int x, int y) { return find(x) == find(y);}
43        int getSize(int x) {return sizes[find(x)];}
44        int getComponents() const {return components;}
45    };

```

6.4 Fenwick Tree

```

1 // Fenwick Tree (Binary Indexed Tree)
2 class FenwickTree {
3     vector<long long> tree;
4     int n;
5
6     public:
7         FenwickTree(int n) : n(n), tree(n + 1, 0) {}
8
9         void update(int idx, int delta) {
10             for (++idx; idx <= n; idx += idx & -idx) {
11                 tree[idx] += delta;
12             }
13         }
14
15         long long query(int idx) {
16             long long sum = 0;
17             for (++idx; idx > 0; idx -= idx & -idx) {
18                 sum += tree[idx];
19             }
20             return sum;
21         }
22
23         long long query(int l, int r) {
24             return query(r) - (l > 0 ? query(l - 1) : 0);
25         }
26 };

```

6.5 GP Hash Table

```

1 using namespace __gnu_pbds;
2
3 #define ll long long
4 #define pll pair<ll,ll>
5 #define fi first
6 #define sc second
7 #define u_map gp_hash_table
8
9 struct custom_hash {
10     static uint64_t splitmix64(uint64_t x) {
11         // http://xorshift.di.unimi.it/splitmix64.c
12         x += 0x9e3779b97f4a7c15;
13         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
14         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
15         return x ^ (x >> 31);
16     }
17
18     size_t operator()(uint64_t x) const {

```

```

19         static const uint64_t FIXED_RANDOM = chrono::steady_clock::
20             now().time_since_epoch().count();
21         return splitmix64(x + FIXED_RANDOM);
22     }
23 };
24 inline void solve(){
25     int n; ll t;
26     cin>>n>>t;
27     vector<ll> arr(n);
28     for(ll &x: arr) cin>>x;
29     u_map<ll, pll, custom_hash> h;
30
31     for(int i=0; i<n; i++){
32         for(int j=i+1; j<n; j++){
33             ll key = t - arr[i] - arr[j];
34             if(h.find(key) != h.end()){
35                 cout<<i+1<<" "<<j+1<<" "<<h[key].fi+1<<" "<<h[key].sc+1<<endl;
36                 return;
37             }
38         }
39
40         for(int j=0; j<i; j++) h[arr[i]+arr[j]] = {j,i};
41     }
42
43     cout<<"IMPOSSIBLE"<<endl;
44 }

```

6.6 Implicit Treap

```

1  // Implicit Treap (sequence by position)
2  struct ImplicitTreap {
3      static int nextPriority() {
4          static mt19937 rng(chrono::steady_clock::now().
5              time_since_epoch().count());
6          static uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
7          return dist(rng);
8      }
9
10     struct Node {
11         int val;
12         int prior;
13         int sz;
14         long long sum;
15         Node *l, *r;
16
17         explicit Node(int v)
18             : val(v), prior(nextPriority()), sz(1), sum(v), l(
19                 nullptr), r(nullptr) {}
20     };

```

```

19     using pNode = Node*;
20     pNode root = nullptr;
21
22     int sz(pNode t) { return t ? t->sz : 0; }
23     long long sum(pNode t) { return t ? t->sum : 0; }
24
25     void pull(pNode t) {
26         if (!t) return;
27         t->sz = 1 + sz(t->l) + sz(t->r);
28         t->sum = t->val + sum(t->l) + sum(t->r);
29     }
30
31     void split(pNode t, int leftSize, pNode &l, pNode &r) {
32         if (!t) {
33             l = r = nullptr;
34             return;
35         }
36         int curLeft = sz(t->l);
37         if (leftSize <= curLeft) {
38             split(t->l, leftSize, l, t->l);
39             r = t;
40         } else {
41             split(t->r, leftSize - curLeft - 1, t->r, r);
42             l = t;
43         }
44         pull(t);
45     }
46
47     pNode merge(pNode l, pNode r) {
48         if (!l || !r) return l ? l : r;
49         if (l->prior > r->prior) {
50             l->r = merge(l->r, r);
51             pull(l);
52             return l;
53         }
54         r->l = merge(l, r->l);
55         pull(r);
56         return r;
57     }
58
59     void insert(int pos, int val) { // insert before pos
60         pNode L, R;
61         split(root, pos, L, R);
62         root = merge(merge(L, new Node(val)), R);
63     }
64
65     void erase(int pos) {
66         pNode A, B, C;
67         split(root, pos, A, B);
68         split(B, 1, B, C); // B is erased node
69         root = merge(A, C);
70     }
71 }

```

```

72
73     long long rangeSum(int l, int r) {
74         pNode A, B, C;
75         split(root, r + 1, B, C);
76         split(B, l, A, B);
77         long long ans = sum(B);
78         root = merge(merge(A, B), C);
79         return ans;
80     }
81 };
82
83 // Example of use:
84 // ImplicitTreap it;
85 // it.insert(0, 5); it.insert(1, 7); it.insert(1, 3); // [5,3,7]
86 // long long s = it.rangeSum(0, 2); // 15

```

6.7 Iterative Segment Tree

```

1  class SegmentTree{
2  private:
3      vector<int> segtree;
4      int n;
5      int d;
6
7  public:
8
9      void build(vector<int> &a){
10         n = a.size();
11         d=1;
12         while(d<n) d<<=1;
13         segtree.assign(2*d, 0);
14         for(int i=0; i<n; i++)
15             segtree[d+i] = a[i];
16
17         for(int i=d-1; i>=1; i--)
18             segtree[i] = segtree[2*i] + segtree[2*i+1];
19     }
20
21     int find_first(int x){
22         if(segtree[1]<x) return -1;
23         int i=1;
24
25         while(i<d){
26             if(segtree[2*i] >= x) i = 2*i;
27             else i = 2*i+1;
28         }
29
30         return i-d;
31     }
32 }

```

```

33
34     int find_kth(int x){
35         //if(segtree[1]<x) return -1;
36         int i=1;
37
38         while(i<d){
39             if(segtree[2*i] >= x) i = 2*i;
40             else{
41                 x -= segtree[2*i];
42                 i = 2*i+1;
43             }
44         }
45         return i-d;
46     }
47
48
49     void update(int pos){
50         pos+=d;
51         segtree[pos] = 0;
52
53         while(pos>1){
54             pos>>=1;
55             segtree[pos] = segtree[2*pos] + segtree[2*pos+1];
56         }
57     }
58
59
60     int query(int l, int r){
61         l+=d;
62         r+=d;
63
64         int mx = INT_MIN;
65         while(l<=r){
66             if(l%2 == 1) mx = max(mx, segtree[l++]);
67             if(r%2 == 0) mx = max(mx, segtree[r--]);
68             l>>=1;
69             r>>=1;
70         }
71         return mx;
72     }
73
74 };

```

6.8 Lazy Segment Tree (Max)

```

1  enum QueryType{ MAX, NONE };
2
3  struct Query{
4      QueryType type = NONE;
5      ll val = 0;

```

```

6 };
7
8 template <typename T> class LazySegTree {
9     private:
10         const int n;
11         vector<T> tree;
12         vector<Query> lazy;
13
14         void build(int v, int l, int r, const vector<T> &a){
15             if(l==r) tree[v] = a[l];
16             else{
17                 int m = (l+r)/2;
18                 build(2*v, l, m, a);
19                 build(2*v+1, m+1, r, a);
20                 tree[v] = max(tree[2*v], tree[2*v+1]);
21             }
22         }
23
24         void apply(int v, int len, const Query &x){
25             tree[v] += x.val;
26             lazy[v].val += x.val;
27             lazy[v].type = MAX;
28         }
29
30         void push(int v, int l, int r){
31             int m = (l+r)/2;
32             apply(2*v, m-l+1, lazy[v]);
33             apply(2*v+1, r-m, lazy[v]);
34             lazy[v] = Query();
35         }
36
37         void update(int v, int l, int r, int ql, int qr, const Query
&x){
38             if(qr < l or ql > r) return;
39
40             if(ql<=l and r <= qr) apply(v,r-l+1,x);
41             else{
42                 push(v,l,r);
43                 int m = (l+r)/2;
44                 update(2*v,l,m,ql,qr,x);
45                 update(2*v+1,m+1,r,ql,qr,x);
46                 tree[v] = max(tree[2*v], tree[2*v+1]);
47             }
48         }
49
50         T query(int v, int l, int r, int ql, int qr){
51             if(qr < l || ql > r) return LLONG_MIN;
52             if(l>=ql and r<=qr) return tree[v];
53             push(v,l,r);
54             int m = (l+r)/2;
55
56             return max(query(2*v, l, m, ql, qr), query(2*v+1,m+1,r,
ql,qr));

```

```

57     }
58
59     public:
60         LazySegTree(const vector<T> &a) : n(a.size()-1), tree(8 * (a
.size()-1)), lazy(8 * (a.size()-1)){
61             build(1, 0, n, a);
62         }
63
64         void update(int ql, int qr, const Query &x){
65             update(1,0, n,ql,qr,x);
66         }
67
68         T query(int ql, int qr){ return query(1,0,n,ql,qr);};
69
70 };

```

6.9 Lazy Segment Tree (Sum)

```

1 enum QueryType{ ADD, SET, NONE };
2
3 struct Query{
4     QueryType type = NONE;
5     ll val = 0;
6 };
7
8 template <typename T> class LazySegTree {
9     private:
10         const int n;
11         vector<T> tree;
12         vector<Query> lazy;
13
14         void build(int v, int l, int r, const vector<T> &a){
15             if(l==r) tree[v] = a[l];
16             else{
17                 int m = (l+r)/2;
18                 build(2*v, l, m, a);
19                 build(2*v+1, m+1, r, a);
20                 tree[v] = tree[2*v] + tree[2*v+1];
21             }
22         }
23
24         void apply(int v, int len, const Query &x){
25             if(x.type == ADD){
26                 if(lazy[v].type != SET)
27                     lazy[v] = Query{ADD, lazy[v].val + x.val};
28                 else
29                     lazy[v] = Query{SET, lazy[v].val + x.val};
30
31                 tree[v] += x.val*len;
32             }else if(x.type == SET){

```

```

33         tree[v] = x.val*len;
34         lazy[v] = x;
35     }
36 }
37
38 void push(int v, int l, int r){
39     int m = (l+r)/2;
40     apply(2*v, m-1+1, lazy[v]);
41     apply(2*v+1, r-m, lazy[v]);
42     lazy[v] = Query();
43 }
44
45 void update(int v, int l, int r, int ql, int qr, const Query
46     &x){
47     if(qr < l or ql > r) return;
48
49     if(ql<=l and r <= qr) apply(v,r-1+1,x);
50     else{
51         push(v,l,r);
52         int m = (l+r)/2;
53         update(2*v,l,m,ql,qr,x);
54         update(2*v+1,m+1,r,ql,qr,x);
55         tree[v] = tree[2*v] + tree[2*v+1];
56     }
57 }
58
59 T query(int v, int l, int r, int ql, int qr){
60     if(qr < l || ql > r) return 0;
61     if(l>=ql and r<=qr) return tree[v];
62     push(v,l,r);
63     int m = (l+r)/2;
64
65     return query(2*v, l, m, ql, qr) + query(2*v+1,m+1,r,ql,
66         qr);
67 }
68
69 public:
70     LazySegTree(const vector<T> &a) : n(a.size()), tree(4 * n),
71         lazy(4 * n){
72         build(1, 0, n - 1, a);
73     }
74
75     void update(int ql, int qr, const Query &x){
76         update(1,0, n-1,ql,qr,x);
77     }
78
79     T query(int ql, int qr){ return query(1,0,n-1,ql,qr);};
80 };

```

6.10 Lazy Treap

```

1 // Lazy Implicit Treap (range add, range sum, range reverse)
2 struct LazyTreap {
3     static int nextPriority() {
4         static mt19937 rng(chrono::steady_clock::now().
5             time_since_epoch().count());
6         static uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
7         return dist(rng);
8     }
9
10    struct Node {
11        int val;
12        int prior;
13        int sz;
14        long long sum;
15        long long lazyAdd;
16        bool rev;
17        Node *l, *r;
18
19        explicit Node(int v)
20            : val(v), prior(nextPriority()), sz(1), sum(v), lazyAdd
21              (0), rev(false), l(nullptr), r(nullptr) {}
22    };
23
24    using pNode = Node*;
25    pNode root = nullptr;
26
27    int sz(pNode t) { return t ? t->sz : 0; }
28    long long sum(pNode t) { return t ? t->sum : 0; }
29
30    void applyAdd(pNode t, long long add) {
31        if (!t) return;
32        t->val += add;
33        t->sum += add * sz(t);
34        t->lazyAdd += add;
35    }
36
37    void applyRev(pNode t) {
38        if (!t) return;
39        t->rev ^= 1;
40        swap(t->l, t->r);
41    }
42
43    void push(pNode t) {
44        if (!t) return;
45        if (t->rev) {
46            applyRev(t->l);
47            applyRev(t->r);
48            t->rev = false;
49        }
50        if (t->lazyAdd != 0) {

```

```

49     applyAdd(t->l, t->lazyAdd);
50     applyAdd(t->r, t->lazyAdd);
51     t->lazyAdd = 0;
52 }
53 }
54
55 void pull(pNode t) {
56     if (!t) return;
57     t->sz = 1 + sz(t->l) + sz(t->r);
58     t->sum = t->val + sum(t->l) + sum(t->r);
59 }
60
61 void split(pNode t, int leftSize, pNode &l, pNode &r) {
62     if (!t) {
63         l = r = nullptr;
64         return;
65     }
66     push(t);
67     int curLeft = sz(t->l);
68     if (leftSize <= curLeft) {
69         split(t->l, leftSize, l, t->l);
70         r = t;
71     } else {
72         split(t->r, leftSize - curLeft - 1, t->r, r);
73         l = t;
74     }
75     pull(t);
76 }
77
78 pNode merge(pNode l, pNode r) {
79     if (!l || !r) return l ? l : r;
80     if (l->prior > r->prior) {
81         push(l);
82         l->r = merge(l->r, r);
83         pull(l);
84         return l;
85     }
86     push(r);
87     r->l = merge(l, r->l);
88     pull(r);
89     return r;
90 }
91
92 void insert(int pos, int val) {
93     pNode L, R;
94     split(root, pos, L, R);
95     root = merge(merge(L, new Node(val)), R);
96 }
97
98 void rangeAdd(int l, int r, long long add) {
99     pNode A, B, C;
100    split(root, r + 1, B, C);
101    split(B, l, A, B);

```

```

102    applyAdd(B, add);
103    root = merge(merge(A, B), C);
104 }
105
106 void rangeReverse(int l, int r) {
107     pNode A, B, C;
108     split(root, r + 1, B, C);
109     split(B, l, A, B);
110     applyRev(B);
111     root = merge(merge(A, B), C);
112 }
113
114 long long rangeSum(int l, int r) {
115     pNode A, B, C;
116     split(root, r + 1, B, C);
117     split(B, l, A, B);
118     long long ans = sum(B);
119     root = merge(merge(A, B), C);
120     return ans;
121 }
122 };
123
124 // Example of use:
125 // LazyTreap lt;
126 // lt.insert(0, 1); lt.insert(1, 2); lt.insert(2, 3); // [1,2,3]
127 // lt.rangeAdd(0, 1, 5); // [6,7,3]
128 // lt.rangeReverse(0, 2); // [3,7,6]
129 // long long s = lt.rangeSum(0, 2); // 16

```

6.11 Merge Sort Tree

```

1 class MergeSortTree{
2 private:
3     vector<vector<int>> segtree;
4     int n;
5     int d;
6
7 public:
8     void build(vector<int> &a){
9         n = a.size();
10        d=1;
11        while(d<n) d<=<=1;
12        segtree.assign(2*d, {0});
13        for(int i=0; i<n; i++){
14            segtree[d+i] = {a[i]};
15        }
16        for(int i=d-1; i>=1; i--){
17            segtree[i].resize(segtree[2*i].size() + segtree[2*i].
18                               size());
19            merge(segtree[2*i].begin(), segtree[2*i].end(),

```

```

19         segtree[2*i+1].begin(), segtree[2*i+1].end(),
20         segtree[i].begin());
21     }
22 }
23
24 int query(int l, int r, int x){
25     l+=d;
26     r+=d;
27
28     int arns = 0;
29     while(l<=r){
30         if(l%2 == 1){
31             arns += lower_bound(segtree[l].begin(), segtree[l].
32                                 end(), x) - segtree[l].begin();
33             l++;
34         }
35         if(r%2 == 0){
36             arns += lower_bound(segtree[r].begin(), segtree[r].
37                                 end(), x) - segtree[r].begin();
38             r--;
39         }
40         l>>=1;
41         r>>=1;
42     }
43     return arns;
44 }
45 };

```

6.12 Ordered Set

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3
4 #define pii pair<int,int>
5
6 using namespace std;
7 using namespace __gnu_pbds;
8
9 #define int long long
10
11 const int MAX = 2e5+1;
12 const int INF = LLONG_MAX;
13
14 template<typename T> using Tree = tree<T, null_type, less<T>,
15     rb_tree_tag, tree_order_statistics_node_update>;
16
17 void solve(){
18     int n,q; cin>>n>>q;

```

```

18     vector<int> v(n);
19     Tree<pii> oset;
20
21     for(int &x : v) cin>>x;
22
23     for(int i=0; i<n; i++){
24         oset.insert({v[i],i});
25     }
26
27     for(int i=0; i<q; i++){
28         char ty; cin>>ty;
29         if(ty == '!'){
30             int k, s; cin>>k>>s;
31             k--;
32             oset.erase({v[k],k});
33             v[k] = s;
34             oset.insert({v[k],k});
35         }else{
36             int a, b; cin>>a>>b;
37             cout<<oset.order_of_key({b,INF}) - oset.order_of_key({a
38                 -1,INF})<<endl;
39         }
40     }
41 }

```

6.13 Recursive Segment Tree

```

1 ll t[4*MAX];
2
3 // Shout-out to CP algo for the SegTree implementation: https://cp-
4     algorithms.com/data\_structures/segment\_tree.html#memory-
5     efficient-implementation
6
7 void buildSegTree(vector<ll> &a, int v, int tl, int tr) {
8     if (tl == tr) {
9         t[v] = a[tl];
10    } else {
11        int tm = (tl + tr) / 2;
12        buildSegTree(a, v*2, tl, tm);
13        buildSegTree(a, v*2+1, tm+1, tr);
14        t[v] = t[v*2] + t[v*2+1];
15    }
16 }
17
18 ll sum(int v, int tl, int tr, int l, int r) {
19     if (l > r)
20         return 0;
21     if (l == tl && r == tr) {

```

```

21     return t[v];
22 }
23 int tm = (t1 + tr) / 2;
24 return sum(v*2, t1, tm, l, min(r, tm))
25     + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
26 }
27
28 void update(int v, int t1, int tr, int pos, ll new_val) {
29     if (t1 == tr) {
30         t[v] = new_val;
31     } else {
32         int tm = (t1 + tr) / 2;
33         if (pos <= tm)
34             update(v*2, t1, tm, pos, new_val);
35         else
36             update(v*2+1, tm+1, tr, pos, new_val);
37         t[v] = t[v*2] + t[v*2+1];
38     }
39 }

```

6.14 Treap

```

1 // Treap (BST by key + heap by priority)
2 struct Treap {
3     static int nextPriority() {
4         static mt19937 rng(chrono::steady_clock::now().
5             time_since_epoch().count());
6         static uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
7         return dist(rng);
8     }
9
10    struct Node {
11        int key;
12        int prior;
13        int cnt;
14        Node *l, *r;
15
16        explicit Node(int k) : key(k), prior(nextPriority()), cnt(1)
17            , l(nullptr), r(nullptr) {}
18    };
19
20    using pNode = Node*;
21    pNode root = nullptr;
22
23    int cnt(pNode t) { return t ? t->cnt : 0; }
24
25    void pull(pNode t) {
26        if (!t) return;
27        t->cnt = 1 + cnt(t->l) + cnt(t->r);
28    }

```

```

27
28 void split(pNode t, int key, pNode &l, pNode &r) {
29     if (!t) {
30         l = r = nullptr;
31         return;
32     }
33     if (t->key <= key) {
34         split(t->r, key, t->r, r);
35         l = t;
36     } else {
37         split(t->l, key, l, t->l);
38         r = t;
39     }
40     pull(t);
41 }
42
43 pNode merge(pNode l, pNode r) {
44     if (!l || !r) return l ? l : r;
45     if (l->prior > r->prior) {
46         l->r = merge(l->r, r);
47         pull(l);
48         return l;
49     }
50     r->l = merge(l, r->l);
51     pull(r);
52     return r;
53 }
54
55 bool contains(pNode t, int key) {
56     while (t) {
57         if (t->key == key) return true;
58         if (key < t->key) t = t->l;
59         else t = t->r;
60     }
61     return false;
62 }
63
64 void insert(int key) {
65     if (contains(root, key)) return; // set behavior (no
66         duplicates)
67     pNode L, R;
68     split(root, key, L, R);
69     root = merge(merge(L, new Node(key)), R);
70 }
71
72 void erase(int key) {
73     pNode A, B, C;
74     split(root, key, A, C);
75     split(A, key - 1, A, B); // B contains only key (if exists)
76     // delete subtree B if needed for memory cleanup
77     root = merge(A, C);
78 }

```

```

79  int order_of_key(int key) { // count of elements < key
80      pNode t = root;
81      int ans = 0;
82      while (t) {
83          if (key <= t->key) {
84              t = t->l;
85          } else {
86              ans += 1 + cnt(t->l);
87              t = t->r;
88          }
89      }
90      return ans;
91  }
92
93  int find_by_order(int k) { // 0-indexed kth smallest
94      pNode t = root;
95      while (t) {
96          int left = cnt(t->l);
97          if (k < left) t = t->l;
98          else if (k == left) return t->key;
99          else {
100              k -= left + 1;
101              t = t->r;
102          }
103      }
104      return -1; // invalid k
105  }
106 };
107
108 // Example of use:
109 // Treap tr;
110 // tr.insert(10); tr.insert(4); tr.insert(20);
111 // int x = tr.find_by_order(1); // 10
112 // int y = tr.order_of_key(15); // 2

```

6.15 Cycle Detection Online

```

1  class CycleDetectionDSU {
2      vector<int> parent;
3      vector<int> size;
4
5  public:
6      CycleDetectionDSU(int n) : parent(n), size(n, 1) {
7          iota(parent.begin(), parent.end(), 0);
8      }
9
10     int find(int x) {
11         return parent[x] == x ? x : parent[x] = find(parent[x]);
12     }
13

```

```

14     // Returns true if adding edge u-v creates a cycle
15     bool add_edge(int u, int v) {
16         int u_root = find(u);
17         int v_root = find(v);
18         if (u_root == v_root) return true;
19
20         if (size[u_root] < size[v_root]) swap(u_root, v_root);
21         parent[v_root] = u_root;
22         size[u_root] += size[v_root];
23         return false;
24     }
25 };

```

6.16 DSU with Rollbacks

```

1  class DSU {
2  private:
3      vector<int> p, sz;
4      // stores previous unites
5      vector<pair<int &, int>> history;
6
7  public:
8      DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
9
10     int get(int x) { return x == p[x] ? x : get(p[x]); }
11
12     void unite(int a, int b) {
13         a = get(a);
14         b = get(b);
15         if (a == b) { return; }
16         if (sz[a] < sz[b]) { swap(a, b); }
17
18         // save this unite operation
19         history.push_back({sz[a], sz[a]});
20         history.push_back({p[b], p[b]});
21
22         p[b] = a;
23         sz[a] += sz[b];
24     }
25
26     int snapshot() { return history.size(); }
27
28     void rollback(int until) {
29         while (snapshot() > until) {
30             history.back().first = history.back().second;
31             history.pop_back();
32         }
33     }
34 };

```

6.17 Fenwick Lazy

```

1 // Range Update - Range Query (1-based indexing)
2 struct FenwickRURQ {
3     int n;
4     std::vector<int> B1, B2;
5
6     FenwickRURQ(int size) : n(size + 1), B1(size + 2), B2(size + 2)
7     {}
8
9     // Add val to range [l, r] (1-based)
10    void range_add(int l, int r, int val) {
11        add(B1, l, val);
12        add(B1, r + 1, -val);
13        add(B2, l, val * (l - 1));
14        add(B2, r + 1, -val * r);
15    }
16
17    // Get sum of range [l, r] (1-based)
18    int range_sum(int l, int r) {
19        return prefix_sum(r) - prefix_sum(l - 1);
20    }
21 private:
22    void add(std::vector<int>& b, int idx, int val) {
23        for(; idx < n; idx += idx & -idx)
24            b[idx] += val;
25    }
26
27    int sum(const std::vector<int>& b, int idx) {
28        int total = 0;
29        for(; idx > 0; idx -= idx & -idx)
30            total += b[idx];
31        return total;
32    }
33
34    int prefix_sum(int idx) {
35        return sum(B1, idx) * idx - sum(B2, idx);
36    }
37 };

```

6.18 Implicit Treap 2

```

1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2
3 #define ll long long
4
5 struct TreapNode{
6     ll key, pr, sz;
7     TreapNode *l, *r;

```

```

8 };
9
10 typedef TreapNode* Treap;
11
12 int getSize(Treap &t){
13     return t ? t->sz : 0;
14 }
15
16 void updateSize(Treap &t){
17     if (t) t->sz = 1 + getSize(t->l) + getSize(t->r);
18 }
19
20 void split(Treap& t, ll k, Treap &l, Treap &r){
21     if(not t) l = r = nullptr;
22
23     else if(k < t->key){
24         split(t->l, k, l, t->l);
25         r = t;
26         updateSize(r);
27     }else{
28         split(t->r, k, t->r, r);
29         l = t;
30         updateSize(l);
31     }
32 }
33
34 void insert(Treap& t, Treap a){
35     if(not t) t=a;
36     else if(a->pr > t->pr){
37         split(t, a->key, a->l, a->r);
38         t = a;
39     }else{
40         if(a->key < t-> key) insert(t->l, a);
41         else insert(t->r, a);
42     }
43     updateSize(t);
44 }
45
46 void merge(Treap &t, Treap l, Treap r){
47     if(not l) t = r;
48     else if(not r) t = l;
49
50     else if(l->pr > r->pr){
51         merge(l->r, l->r, r);
52         t=l;
53         updateSize(t);
54     }else{
55         merge(r->l, l, r->l);
56         t=r;
57         updateSize(t);
58     }
59 }
60

```

```

61 void erase(Treap &t, ll k){
62     if(not t) return;
63     if(t->key == k) merge(t,t->l, t->r);
64
65     else{
66         if(k<t->key) erase(t->l,k);
67         else erase(t->r, k);
68     }
69     updateSize(t);
70 }
71
72 bool find(Treap& t, ll k){
73     if (not t) return false;
74     if(t->key == k) return true;
75     if(k<t->key) return find(t->l,k);
76     return find(t->r,k);
77 }
78
79 void insertValue(Treap &t, ll k){
80     if(not find(t,k)){
81         Treap new_node = new TreapNode {k, rng(), 0, nullptr, nullptr};
82         insert(t, new_node);
83     }
84 }
85
86 ll getKth(Treap &t, int k){
87     if(!t || k<=0 || k>getSize(t)) return 0;
88     int leftSize = getSize(t->l);
89     if(k == leftSize+1) return t->key;
90     if(k <= leftSize) return getKth(t->l,k);
91     return getKth(t->r, k-leftSize-1);
92 }

```

6.19 Online Bipartiteness

```

1 void make_set(int v) {
2     parent[v] = make_pair(v, 0);
3     rank[v] = 0;
4     bipartite[v] = true;
5 }
6
7 pair<int, int> find_set(int v) {
8     if (v != parent[v].first) {
9         int parity = parent[v].second;
10        parent[v] = find_set(parent[v].first);
11        parent[v].second ^= parity;
12    }
13    return parent[v];
14 }

```

```

15
16 void add_edge(int a, int b) {
17     pair<int, int> pa = find_set(a);
18     a = pa.first;
19     int x = pa.second;
20
21     pair<int, int> pb = find_set(b);
22     b = pb.first;
23     int y = pb.second;
24
25     if (a == b) {
26         if (x == y)
27             bipartite[a] = false;
28     } else {
29         if (rank[a] < rank[b])
30             swap (a, b);
31         parent[b] = make_pair(a, x^y^1);
32         bipartite[a] ^= bipartite[b];
33         if (rank[a] == rank[b])
34             ++rank[a];
35     }
36 }
37
38 bool is_bipartite(int v) {
39     return bipartite[find_set(v).first];
40 }

```

6.20 Path Copy Persistent Array

```

1 struct Node {
2     int val;
3     Node *l, *r;
4
5     Node(ll x) : val(x), l(nullptr), r(nullptr) {}
6     Node(Node *ll, Node *rr) : val(0), l(ll), r(rr) {}
7 };
8
9 int n, a[100001]; // The initial array and its size
10 Node *roots[100001]; // The persistent array's roots
11
12 Node *build(int l = 0, int r = n - 1) {
13     if (l == r) return new Node(a[l]);
14     int mid = (l + r) / 2;
15     return new Node(build(l, mid), build(mid + 1, r));
16 }
17
18 Node *update(Node *node, int val, int pos, int l = 0, int r = n - 1)
19 {
20     if (l == r) return new Node(val);
21     int mid = (l + r) / 2;

```

```

21     if (pos > mid) return new Node(node->l, update(node->r, val, pos,
22         mid + 1, r));
23     else return new Node(update(node->l, val, pos, l, mid), node->r);
24 }
25 int query(Node *node, int pos, int l = 0, int r = n - 1) {
26     if (l == r) return node->val;
27     int mid = (l + r) / 2;
28     if (pos > mid) return query(node->r, pos, mid + 1, r);
29     return query(node->l, pos, l, mid);
30 }
31
32 int get_item(int index, int time) {
33     // Gets the array item at a given index and time
34     return query(roots[time], index);
35 }
36
37 void update_item(int index, int value, int prev_time, int curr_time)
38 {
39     // Updates the array item at a given index and time
40     roots[curr_time] = update(roots[prev_time], index, value);
41 }
42
43 void init_arr(int nn, int *init) {
44     // Initializes the persistent array, given an input array
45     n = nn;
46     for (int i = 0; i < n; i++) a[i] = init[i];
47     roots[0] = build();
48 }

```

6.21 Persistent Array

```

1 vector<pair<int, int>> arr[100001]; // The persistent array
2
3 int get_item(int index, int time) {
4     // Gets the array item at a given index and time
5     auto ub =
6         upper_bound(arr[index].begin(), arr[index].end(), make_pair(
7             time, INT_MAX));
8     return prev(ub)->second;
9 }
10
11 void update_item(int index, int value, int time) {
12     // Updates the array item at a given index and time
13     // Note that this only works if the time is later than all
14     // previous
15     // update times
16     assert(arr[index].back().first < time);
17     arr[index].push_back({time, value});
18 }

```

```

17
18 void init_arr(int n, int *init) {
19     // Initializes the persistent array, given an input array
20     for (int i = 0; i < n; i++) arr[i].push_back({0, init[i]});
21 }

```

6.22 Persistent Segment Tree

```

1 using ll = long long;
2
3 class PersistentSegtree {
4     private:
5     struct Node {
6         ll sum = 0;
7         int l = 0, r = 0;
8     };
9
10    const int n;
11    vector<Node> tree;
12    int timer = 1;
13
14    Node join(int l, int r) { return Node{tree[l].sum + tree[r].sum, l
15        , r}; }
16
17    int build(int tl, int tr, const vector<int> &arr) {
18        if (tl == tr) {
19            tree[timer] = {arr[tl], 0, 0};
20            return timer++;
21        }
22
23        int mid = (tl + tr) / 2;
24        tree[timer] = join(build(tl, mid, arr), build(mid + 1, tr, arr))
25            ;
26
27        return timer++;
28    }
29
30    int set(int v, int pos, int val, int tl, int tr) {
31        if (tl == tr) {
32            tree[timer] = {val, 0, 0};
33            return timer++;
34        }
35
36        int mid = (tl + tr) / 2;
37        if (pos <= mid) {
38            tree[timer] = join(set(tree[v].l, pos, val, tl, mid), tree[v].
39                r);
40        } else {
41            tree[timer] = join(tree[v].l, set(tree[v].r, pos, val, mid +
42                1, tr));
43        }
44    }
45 }

```

```

39     }
40
41     return timer++;
42 }
43
44 ll range_sum(int v, int ql, int qr, int tl, int tr) {
45     if (qr < tl || tr < ql) { return 0ll; }
46     if (ql <= tl && tr <= qr) { return tree[v].sum; }
47
48     int mid = (tl + tr) / 2;
49     return range_sum(tree[v].l, ql, qr, tl, mid) +
50         range_sum(tree[v].r, ql, qr, mid + 1, tr);
51 }
52
53 public:
54 PersistentSegtree(int n, int MX_NODES) : n(n), tree(MX_NODES) {}
55
56 int build(const vector<int> &arr) { return build(0, n - 1, arr); }
57
58 int set(int root, int pos, int val) { return set(root, pos, val,
59     0, n - 1); }
60
61 ll range_sum(int root, int l, int r) { return range_sum(root, l, r,
62     0, n - 1); }
63
64 int add_copy(int root) {
65     tree[timer] = tree[root];
66     return timer++;
67 }
68 };

```

6.23 Prefix Suffix Segment Tree

```

1  typedef long long ll;
2  typedef vector<int> vec;
3  typedef vector<pair<int,int>> vpii;
4  const ll mod=1e9+7;
5  const int MAX=1e5+3;
6  const int limit=2e5+3;
7  const int TAM=2e5+1;
8  ll t[4*TAM];
9  ll prefix[4*TAM], suffix[4*TAM], biggest[4*TAM];
10 //ascii https://elcodigoascii.com.ar/
11 ll cero=0;
12 void build(int root, int l, int r, vector<ll> &arr)
13 {
14     if (l==r)
15     {
16         t[root]=arr[l];
17         suffix[root]=max(t[root], cero);

```

```

18         prefix[root]=max(t[root], cero);
19         biggest[root]=max(t[root], cero);
20         return;
21     }
22     int mid=(l+r)/2;
23     build(2*root, l, mid, arr);
24     build(2*root+1, mid+1, r, arr);
25     t[root]=t[2*root]+t[2*root+1];
26     biggest[root]=max(biggest[2*root],
27         max(biggest[2*root+1], suffix[2*root]+prefix[2*root+1]));
28     prefix[root]=max(prefix[2*root], t[2*root]+prefix[2*root+1]);
29     suffix[root]=max(suffix[2*root+1], t[2*root+1]+suffix[2*root]);
30 }
31
32 void update(int root, int l, int r, int pos, ll val)
33 {
34     if (l==r)
35     {
36         t[root]=val;
37         suffix[root]=max(cero, t[root]);
38         prefix[root]=max(cero, t[root]);
39         biggest[root]=max(t[root], cero);
40         return;
41     }
42     int mid=(l+r)/2;
43     if (pos>mid)
44     {
45         update(2*root+1, mid+1, r, pos, val);
46     }
47     else{
48         update(2*root, l, mid, pos, val);
49     }
50     t[root]=t[2*root]+t[2*root+1];
51     biggest[root]=max(biggest[2*root],
52         max(biggest[2*root+1], suffix[2*root]+prefix[2*root+1]));
53     prefix[root]=max(prefix[2*root], t[2*root]+prefix[2*root+1]);
54     suffix[root]=max(suffix[2*root+1], t[2*root+1]+suffix[2*root]);
55 }
56
57 ll consult(int root, int l, int r, int a, int b)
58 {
59     if (a>b) return 0;
60     if (l==a && r==b) return t[root];
61     int mid=(l+r)/2;
62     return consult(2*root, l, mid, a, min(b, mid))+
63         consult(2*root+1, mid+1, r, max(mid+1, a), b);
64 }
65

```

6.24 Wicho Lazy Segment Tree

```

1 typedef long long ll;
2 typedef vector<int> vec;
3 typedef vector<pair<int,int>> vpri;
4 const ll mod=1e9+7;
5 const int MAX=1e5+3;
6 const int limit=2e5+3;
7 const int TAM=2e5+1;
8 ll t[4*TAM];
9 ll op[4*TAM];
10 int type[4*TAM];
11 //ascii https://elcodigoascii.com.ar/
12
13 void propagate(int root,int l,int r)
14 {
15     if(type[root]==1)
16     {
17         t[root]+=op[root]*(r+1-l);
18         if(l!=r){
19             op[2*root]+=op[root];
20             op[2*root+1]+=op[root];
21             type[2*root+1]=max(1,type[2*root+1]);
22             type[2*root]=max(1,type[2*root]);
23         }
24     }
25     else
26     {
27         if(type[root]==2){
28             t[root]=op[root]*(r+1-l);
29             if(l!=r){
30                 op[2*root]=op[root];
31                 op[2*root+1]=op[root];
32                 type[2*root+1]=2;
33                 type[2*root]=2;
34             }
35         }
36     }
37     op[root]=0;
38     type[root]=0;
39 }
40
41 void build(int root,int l,int r,vector<ll> &arr)
42 {
43     if(l==r)
44     {
45         t[root]=arr[l];
46         op[root]=0;
47         type[root]=0;
48         return;
49     }
50     int mid=(l+r)/2;
51     build(2*root,l,mid,arr);
52     build(2*root+1,mid+1,r,arr);

```

```

53     t[root]=t[2*root]+t[2*root+1];
54     op[root]=0;
55     type[root]=0;
56 }
57
58 void sum(int root,int l,int r,int a,int b,ll val)
59 {
60     propagate(root,l,r);
61     if(a>b) return;
62     if(l==a && r==b)
63     {
64         op[root]=val;
65         type[root]=1;
66         propagate(root,l,r);
67         return;
68     }
69     int mid=(l+r)/2;
70     sum(2*root,l,mid,a,min(b,mid),val);
71     sum(2*root+1,mid+1,r,max(mid+1,a),b,val);
72     t[root]=t[2*root]+t[2*root+1];
73 }
74
75 void setR(int root,int l,int r,int a,int b,ll val)
76 {
77     propagate(root,l,r);
78     if(a>b) return;
79     if(l==a && r==b)
80     {
81         op[root]=val;
82         type[root]=2;
83         propagate(root,l,r);
84         return;
85     }
86     int mid=(l+r)/2;
87     setR(2*root,l,mid,a,min(b,mid),val);
88     setR(2*root+1,mid+1,r,max(mid+1,a),b,val);
89     t[root]=t[2*root]+t[2*root+1];
90 }
91
92 ll consult(int root,int l,int r, int a,int b)
93 {
94     propagate(root,l,r);
95     if(a>b) return 0;
96     if(l==a && r==b){
97         return t[root];
98     }
99     int mid=(l+r)/2;
100     return consult(2*root,l,mid,a,min(b,mid))+
101     consult(2*root+1,mid+1,r,max(mid+1,a),b);
102 }

```

6.25 Li Chao Tree

```

1 typedef long long ll;
2 typedef complex<ll> pt;
3 typedef vector<int> vec;
4
5 const ll mod=1e9+7;
6 const ll MAX=1e5+3;
7 const ll INF=1e18;
8
9 pt lines[4*MAX];
10
11 //Li Chao Tree
12
13 ll dot(pt a,pt b){
14     return a.X*b.X+a.Y*b.Y;
15 }
16
17 ll f(pt line,ll val){
18     return dot(line,pt{val,1});
19 }
20
21 void addLine(pt cur,int root=1,int l=0,int r=MAX)
22 {
23
24     int m=(l+r)/2;
25     bool left_border=f(cur,l)>f(lines[root],l);
26     bool mid=f(cur,m)>f(lines[root],m);
27
28     if(mid){
29         swap(lines[root],cur);
30     }
31     if(r-l==1) return;
32     else if(left_border!=mid){
33         addLine(cur,2*root,l,m);
34     }
35     else{
36         addLine(cur,2*root+1,m,r);
37     }
38 }
39
40 ll maximum(int x,int root=1,int l=0,int r=MAX)
41 {
42     int mid=(l+r)/2;
43
44     if(r-l==1){
45         return f(lines[root],x);
46     }
47     if(x<mid){
48         return max(f(lines[root],x),maximum(x,2*root,l,mid));
49     }
50     return max(f(lines[root],x),maximum(x,2*root+1,mid,r));

```

```

51 }
52
53 void addSegment(pt cur,int a,int b,int root=1,int l=0,int r=MAX)
54 {
55     if(a>b) return;
56
57     if(a==l && b==r){
58         addLine(cur,root,l,r);
59         return;
60     }
61     if(r-l==1) return;
62     int mid=(l+r)/2;
63     addSegment(cur,a,min(mid,b),2*root,l,mid);
64     addSegment(cur,max(mid,a),b,2*root+1,mid,r);
65 }

```

6.26 Mo's Algorithm

```

1 // Mo's Algorithm for offline range query processing
2 // Time:  $O((N + Q) * \sqrt{N})$ 
3 // Based on KACTL: https://github.com/kth-competitive-programming/kactl
4
5 #pragma once
6
7 #include <bits/stdc++.h>
8 using namespace std;
9
10 // Mo's Algorithm for offline range query processing.
11 // T: answer/value type
12 // Add/Remove: functors or lambdas with operator(T) or add(T)/remove(T) methods
13 template <typename T>
14 struct MoQueries {
15     struct Query {
16         int l, r, idx;
17     };
18
19     vector<Query> queries;
20     int n;
21
22     explicit MoQueries(int n = 0) : n(n) {}
23
24     void add_query(int l, int r) {
25         queries.push_back({l, r, (int)queries.size()});
26     }
27
28     // Solve using provided add/remove/get operations
29     // Operations can be lambdas or functor objects with operator()
30     // add(x): add element x to current range

```

```

31 // remove(x): remove element x from current range
32 // get(): return current answer of type T
33 template <typename Array, typename Add, typename Remove,
34           typename Get>
35 vector<T> solve(const Array& a, Add add, Remove remove, Get get)
36 {
37     int q = queries.size();
38     int blk = static_cast<int>(sqrt(n));
39
40     sort(queries.begin(), queries.end(), [&](const Query& i,
41       const Query& j) {
42         int bi = i.l / blk;
43         int bj = j.l / blk;
44         if (bi != bj) return bi < bj;
45         // Alternate sorting direction per block to reduce
46         // pointer movement
47         return (bi & 1) ? i.r > j.r : i.r < j.r;
48     });
49
50     vector<T> res(q);
51     int L = 0, R = 0;
52
53     for (const auto& qu : queries) {
54         while (L > qu.l) add(a[--L]);
55         while (R < qu.r) add(a[R++]);
56         while (L < qu.l) remove(a[L++]);
57         while (R > qu.r) remove(a[--R]);
58         res[qu.idx] = get();
59     }
60     return res;
61 }

```

7 Range Queries

7.1 Sparse Table

```

1 // Sparse Table for Range Minimum Query
2 class SparseTable {
3     vector<vector<int>>> table;
4     vector<int> log;
5     int n;
6
7 public:
8     SparseTable(vector<int>& arr) {
9         n = arr.size();
10        int maxLog = log2(n) + 1;
11        table.assign(n, vector<int>(maxLog));
12        log.assign(n + 1, 0);

```

```

13
14 // Precompute logarithms
15 for (int i = 2; i <= n; i++) {
16     log[i] = log[i/2] + 1;
17 }
18
19 // Build sparse table
20 for (int i = 0; i < n; i++) {
21     table[i][0] = arr[i];
22 }
23
24 for (int j = 1; j < maxLog; j++) {
25     for (int i = 0; i + (1 << j) <= n; i++) {
26         table[i][j] = min(table[i][j-1],
27           table[i + (1 << (j-1))][j-1]);
28     }
29 }
30
31 int query(int l, int r) {
32     int j = log[r - l + 1];
33     return min(table[l][j], table[r - (1 << j) + 1][j]);
34 }
35 }
36 };

```

8 Mathematics

8.1 Formula Sheet (Contest)

8.1.1 Combinatorics

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$P(n, k) = \frac{n!}{(n-k)!} \quad (\text{ordered, no repetition})$$

$$\text{Combinations with repetition} = \binom{n+k-1}{k}$$

$$\text{Permutations with repetition} = \frac{n!}{a_1!a_2!\cdots a_m!}$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad (\text{Pascal identity})$$

Multinomial coefficient (split n items into groups $k_1, \dots, k_m$):

$$\binom{n}{k_1, k_2, \dots, k_m} = \frac{n!}{k_1! k_2! \dots k_m!}, \quad \sum_i k_i = n$$

Catalan numbers (balanced parentheses, rooted full binary trees, non-crossing matchings):

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \quad C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$$

- **Valid Parentheses:** Count of balanced parentheses expressions with n pairs.
- **Full Binary Trees:** Structurally unique full binary trees with $n+1$ leaves.
- **Polygon Triangulation:** Ways to triangulate a convex $(n+2)$ -gon.
- **Dyck Paths:** Paths from $(0,0)$ to $(2n,0)$ that never dip below the x-axis.
- **Non-Crossing Partitions:** Ways to connect $2n$ points on a circle without crossing chords.
- **Stack Permutations:** Valid stack-sortable permutations of length n .
- **Mountain Ranges:** Sequences of $2n$ up/down steps forming valid mountain ranges.
- **Unique BSTs:** Number of distinct binary search trees with n keys.
- **Diagonal-Avoiding Paths:** Paths in a grid from $(0,0)$ to (n,n) without crossing the diagonal.

8.2 Cayley's Formula

Number of labeled trees of n vertices: n^{n-2} .

Number of rooted forest of n vertices is: $(n+1)^{n-1}$

8.3 Geometric series

Finite:

$$\sum_{k=0}^n ar^k = \begin{cases} a \frac{1-r^{n+1}}{1-r} & \text{if } r \neq 1, \\ a(n+1) & \text{if } r = 1. \end{cases}$$

Infinite:

$$\sum_{k=0}^{\infty} ar^k = \frac{a}{1-r} \quad (\text{converges iff } |r| < 1)$$

8.4 Divisors

The number of divisors of any number n is:

$$\begin{cases} \approx 100 & n < 5 \times 10^4 \\ \approx 500 & n < 1 \times 10^7 \\ \approx 2000 & n < 1 \times 10^{10} \\ \approx 200000 & n < 1 \times 10^{19} \end{cases}$$

8.5 Number of primes between 1 and n

$$\frac{n}{\ln(n)}$$

8.6 Pythagorean triplets

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2)$$

With $m > n > 0$, $k \geq 1$, $m \perp n$, and either m or n even.

8.7 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \lfloor \frac{n!}{e} + \frac{1}{2} \rfloor$$

Pascal triangle facts:

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = 0 \quad (n \geq 1)$$

8.7.1 Sums

Gaussian sums (first powers to fourth powers):

$$\begin{aligned} \sum_{i=1}^n i &= \frac{n(n+1)}{2} \\ \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6} \\ \sum_{i=1}^n i^3 &= \left(\frac{n(n+1)}{2} \right)^2 \\ \sum_{i=1}^n i^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} \end{aligned}$$

Useful finite progressions:

$$\sum_{i=0}^{n-1} ar^i = a \frac{1-r^n}{1-r} \quad (r \neq 1), \quad \sum_{i=0}^{n-1} (a+id) = \frac{n}{2} (2a + (n-1)d)$$

Rare progressions:

$$\sum_{i=1}^{n-1} i \cdot (n-i) = \frac{n(n^2-1)}{6}$$

8.7.2 Geometry

For points $A(x_1, y_1), B(x_2, y_2)$:

$$\begin{aligned} \text{Euclidean}(A, B) &= \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \\ \text{Manhattan}(A, B) &= |x_1 - x_2| + |y_1 - y_2| \\ \text{Chebyshev}(A, B) &= \max\{|x_1 - x_2|, |y_1 - y_2|\} \end{aligned}$$

For vectors $\vec{a} = (a_x, a_y), \vec{b} = (b_x, b_y)$:

$$\begin{aligned} \vec{a} \cdot \vec{b} &= a_x b_x + a_y b_y \\ \vec{a} \times \vec{b} &= a_x b_y - a_y b_x \end{aligned}$$

Note: In 2D, $\text{sign}(\vec{a} \times \vec{b})$ gives orientation (left/right turn), and $|\vec{a} \times \vec{b}|/2$ is the triangle area.

8.7.3 Probability

$$\begin{aligned} \mathbb{E}[X] &= \sum_x x P(X = x) \\ \mathbb{E}[aX + b] &= a \mathbb{E}[X] + b \\ \mathbb{E}\left[\sum_{i=1}^m X_i\right] &= \sum_{i=1}^m \mathbb{E}[X_i] \quad (\text{linearity, no independence needed}) \end{aligned}$$

Indicator trick (very useful): If I_A is 1 when event A happens and 0 otherwise, then $\mathbb{E}[I_A] = P(A)$. Count expected objects by summing indicators.

8.7.4 Number Theory and Counting Essentials

$$\begin{aligned} \gcd(a, b) \cdot \text{lcm}(a, b) &= ab \\ \varphi(n) &= n \prod_{p|n} \left(1 - \frac{1}{p}\right) \\ a^{\varphi(m)} &\equiv 1 \pmod{m} \quad (\gcd(a, m) = 1) \end{aligned}$$

Inclusion-exclusion for sets $A_1, \dots, A_m$:

$$\left| \bigcup_i A_i \right| = \sum_i |A_i| - \sum_{i < j} |A_i \cap A_j| + \sum_{i < j < k} |A_i \cap A_j \cap A_k| - \dots$$

Stars and bars (nonnegative solutions of $x_1 + \dots + x_k = n$):

$$\# \text{solutions} = \binom{n+k-1}{k-1}$$

8.8 Manhattan Distance on a Grid

For the sum of Manhattan distances between all pairs of points in an $n \times m$ grid:

Compress the $n \times m$ grid along the x axis, leaving a row of m points where each column has n points. The horizontal contribution is:

$$D_x = n^2 \frac{m(m^2 - 1)}{6}$$

Do the same along the y axis:

$$D_y = m^2 \frac{n(n^2 - 1)}{6}$$

Total pairwise Manhattan distance in the grid:

$$D = n^2 \frac{m(m^2 - 1)}{6} + m^2 \frac{n(n^2 - 1)}{6}$$

For the external Manhattan distances between K disjoint rectangular blocks:

NOTE: Each block must have a constant height (same number of rows).

$$D_{\text{external}} = \sum_{i=1}^{K-1} \sum_{j=i+1}^K A_i A_j C_{ij}$$

Where A_i is the area of block i , and C_{ij} is the Manhattan distance between the centers of blocks i and j projected onto one axis. Repeat the same formula for the y axis and add the two contributions. To get a block's center, average the x and y coordinates of its corners; it doesn't have to be an integer.

8.9 Binary Exponentiation

```
1 const int MAX = 2e5+1;
2 const int MOD = 1e9+7;
3 int bnpow(int a, int b, int m){
4     int arns = 1;
5     a%=m;
```

```
6     while(b>0){
7         if(b&1) arns = (arns*a)%m;
8         a = (a*a)%m;
9         b>>=1;
10    }
11    return arns%m;
12 }
```

8.10 Binomial Coefficients

```
1 vector<ll> fact(MAX);
2 vector<ll> i_fact(MAX);
3
4 ll bnpow(ll a, ll b){
5     ll arns = 1;
6     a%=MOD;
7     while(b>0){
8         if(b&1)
9             arns = (arns*a)%MOD;
10            a = (a*a)%MOD;
11            b>>=1;
12        }
13        return arns;
14    }
15
16 ll modInv(ll a, ll m=MOD){
17     return bnpow(a,MOD-2);
18 }
19
20 void factorial(){
21     fact[0] = fact[1] = 1;
22     for(int i=2; i<MAX; i++){
23         fact[i] = (fact[i-1] * i)%MOD;
24     }
25
26     i_fact[MAX-1] = modInv(fact[MAX-1]);
27     for(int i=MAX-2; i>=0; i--){
28         i_fact[i] = (i_fact[i+1]*(i+1))%MOD;
29     }
30 }
31
32
33 ll bn_coef(ll n, ll k){
34     return fact[n] * i_fact[n-k]%MOD * i_fact[k]%MOD;
35 }
```

8.11 Extended Euclidean

```

1 // Extended Euclidean Algorithm
2 // Returns gcd(a, b) and finds x, y such that ax + by = gcd(a, b)
3 int extgcd(int a, int b, int &x, int &y) {
4     if (b == 0) {
5         x = 1;
6         y = 0;
7         return a;
8     }
9     int x1, y1;
10    int g = extgcd(b, a % b, x1, y1);
11    x = y1;
12    y = x1 - (a / b) * y1;
13    return g;
14 }

```

8.12 FFT

```

1 // FFT works with sizes that are powers of 2.
2 // The size should be >= n+m-1 where n,m are input sizes.
3
4 typedef complex<double> cd;
5 double PI = acos(-1);
6
7 // My favorite black box <3
8 vector<cd> fft(vector<cd> a, int d = 1) {
9     int n = a.size();
10    vector<cd> r(n);
11
12    for (int k = 0; k < n; k++) {
13        int b = 0;
14        for (int z = 1; z < n; z *= 2) {
15            b *= 2;
16            if (k & z)
17                b++;
18        }
19        r[b] = a[k];
20    }
21
22    for (int m = 2; m <= n; m *= 2) {
23        cd wm = exp(cd{0, d * 2 * PI / m});
24
25        for (int k = 0; k < n; k += m) {
26            cd w = 1;
27            for (int j = 0; j < m / 2; j++) {
28                cd u = r[k + j];
29                cd t = w * r[k + j + m / 2];
30                r[k + j] = u + t;
31                r[k + j + m / 2] = u - t;
32                w = w * wm;

```

```

33    }
34    }
35 }
36
37 if (d == -1) {
38     for (int i = 0; i < n; i++)
39         r[i] /= n;
40 }
41
42 return r;
43 }
44
45 void solve() {
46     int n, m;
47     cin >> n >> m;
48
49     vector<int> s(n), msk(m);
50
51     for (int i = 0; i < n; i++) {
52         cin >> s[i];
53     }
54     for (int i = 0; i < m; i++) {
55         cin >> msk[i];
56     }
57
58     reverse(msk.begin(), msk.end());
59
60     int sz = 1;
61     while (sz < n + m - 1) sz <= 1;
62     vector<cd> a(sz, cd(0,0)), b(sz, cd(0,0));
63
64     for (int i=0; i<n; i++){
65         a[i] += cd(s[i],0);
66     }
67
68     for (int i=0; i<m; i++){
69         b[i] += cd(msk[i],0);
70     }
71
72     auto A = fft(a);
73     auto B = fft(b);
74     vector<cd> C(sz);
75
76     for (int i = 0; i < sz; i++) {
77         C[i] = A[i] * B[i];
78     }
79
80     auto conv = fft(C, -1);
81
82     for (int i = 0; i < n+m-1; i++) {
83         cout << (ll)round(conv[i].real()) << " ";
84     }
85 }

```

```

86     cout << endl;
87 }

```

```

46 }

```

8.13 Big Integer

```

1 static const __int128 MOD = (__int128)10000000000000000031ULL;
2 static const __int128 g = (__int128)42ULL;
3
4 ostream &operator<<(ostream &os, const __int128 &value){
5     char buffer[64];
6     char *pos = end(buffer)-1;
7     *pos = '\0';
8     __int128 tmp = value < 0 ? -value : value;
9
10    do{
11        --pos;
12        *pos = tmp%10 + '0';
13        tmp/=10;
14    }while(tmp!=0);
15    if(value < 0){
16        --pos;
17        *pos = '-';
18    }
19
20    return os << pos;
21 }
22
23 inline __int128 mulm(__int128 a, __int128 b){
24     __uint128_t z = ( (__uint128_t)a * (__uint128_t)b );
25     return (__int128)( z % MOD );
26 }
27 __int128 binpow(__int128 a, __int128 e){
28     __int128 r = 1;
29     while (e > 0){
30         if (e & 1) r = mulm(r, a);
31         a = mulm(a, a);
32         e >>= 1;
33     }
34     return r;
35 }
36
37 void solve(){
38     int n; cin>>n;
39
40     __int128 p = 300;
41     __int128 cnt = 1;
42     for(11 i = 1000000LL;i>n;i--){
43         p = binpow(g,p);
44     }
45     cout << p << '\n';

```

8.14 NTT

```

1 // Number Theoretic Transform (NTT)
2 // Usage: exact polynomial convolution under modulo arithmetic (no
3 // floating-point precision issues).
4 // Good for multiplying polynomials / counting ways where answers
5 // are required modulo MOD.
6 // Constraints: n must be a power of 2 and n <= ROOT_PW for this
7 // modulus/root setup.
8
9 const int MOD = 7340033;           // 7 * 2^20 + 1
10 const int ROOT = 5;               // primitive root for MOD
11 const int ROOT_INV = 4404020;     // inverse(ROOT) mod MOD
12 const int ROOT_PW = 1 << 20;     // max power-of-two supported
13
14 int binpow(int a, int b, int m = MOD) {
15     int arns = 1;
16     while (b > 0) {
17         if (b & 1) arns = (int)(1LL * arns * a % m);
18         a = (int)(1LL * a * a % m);
19         b >>= 1;
20     }
21     return arns;
22 }
23
24 void ntt(vector<int> &a, bool invert) {
25     int n = (int)a.size();
26
27     // Bit-reversal permutation.
28     for (int i = 1, j = 0; i < n; i++) {
29         int bit = n >> 1;
30         while (j & bit) {
31             j ^= bit;
32             bit >>= 1;
33         }
34         j ^= bit;
35         if (i < j) swap(a[i], a[j]);
36     }
37
38     for (int len = 2; len <= n; len <= 1) {
39         int wlen = invert ? ROOT_INV : ROOT;
40         for (int i = len; i < ROOT_PW; i <= 1) {
41             wlen = (int)(1LL * wlen * wlen % MOD);
42         }
43
44         for (int i = 0; i < n; i += len) {
45             int w = 1;
46             for (int j = 0; j < len / 2; j++) {

```

```

44     int u = a[i + j];
45     int v = (int)(1LL * a[i + j + len / 2] * w % MOD);
46
47     a[i + j] = u + v;
48     if (a[i + j] >= MOD) a[i + j] -= MOD;
49
50     a[i + j + len / 2] = u - v;
51     if (a[i + j + len / 2] < 0) a[i + j + len / 2] +=
        MOD;
52
53     w = (int)(1LL * w * wlen % MOD);
54 }
55 }
56 }
57
58 if (invert) {
59     int n_inv = binpow(n, MOD - 2, MOD);
60     for (int &x : a) {
61         x = (int)(1LL * x * n_inv % MOD);
62     }
63 }
64 }
65
66 // Polynomial multiplication modulo MOD with NTT.
67 vector<int> multiply_ntt(vector<int> a, vector<int> b) {
68     int n = 1;
69     while (n < (int)a.size() + (int)b.size() - 1) n <= 1;
70
71     // Must fit this NTT configuration. Use another modulus/CRT if
72     // larger needed.
73     if (n > ROOT_PW) return {};
74
75     a.resize(n);
76     b.resize(n);
77
78     ntt(a, false);
79     ntt(b, false);
80
81     for (int i = 0; i < n; i++) {
82         a[i] = (int)(1LL * a[i] * b[i] % MOD);
83     }
84
85     ntt(a, true);
86     return a;
87 }
88
89 void solve() {
90     // Example usage:
91     // Input two polynomials A (size n) and B (size m), output
92     // coefficients of A*B mod MOD.
93     int n, m;
94     cin >> n >> m;
95     vector<int> a(n), b(m);

```

```

94     for (int &x : a) cin >> x;
95     for (int &x : b) cin >> x;
96
97     vector<int> c = multiply_ntt(a, b);
98     if (c.empty()) {
99         // This means size exceeded ROOT_PW with current modulus
100        configuration.
101        return;
102    }
103
104    for (int i = 0; i < n + m - 1; i++) {
105        cout << c[i] << " ";
106    }
107    cout << endl;

```

8.15 Graph Paths

```

1 vector<vector<int>> mat_mult(vector<vector<int>> &A, vector<vector<
2 int>> &B, int n, int m, int p){
3     vector<vector<int>> arns(n,vector<int> (p));
4     for(int i=0; i<n; i++){
5         for(int j=0; j<m; j++){
6             for(int k=0; k<p; k++){
7                 arns[i][k] += A[i][j] * B[j][k];
8                 arns[i][k]%=MOD;
9             }
10        }
11    }
12    return arns;
13 }
14
15 vector<vector<int>> mat_pow(vector<vector<int>> &A, int n){
16     int sz = A.size();
17     vector<vector<int>> arns(sz,vector<int> (sz,0));
18     for(int i=0; i<sz; i++)
19         arns[i][i] = 1;
20     while(n>0){
21         if(n&1) arns = mat_mult(arns,A,sz,sz,sz);
22         A = mat_mult(A,A,sz,sz,sz);
23         n>>=1;
24     }
25     return arns;
26 }
27
28 void solve(){
29     int n, m, k; cin>>n>>m>>k;
30     vector<vector<int>> adj(n,vector<int>(n,0));
31     for(int i=0; i<m; i++){

```

```

32     int a,b; cin>>a>>b;
33     a--; b--;
34     adj[a][b]++;
35 }
36 vector<vector<int>> arns = mat_pow(adj,k);
37 cout<<arns[0][n-1]<<endl;
38 }

```

```

40     for(int i=0; i<m; i++){
41         int a,b,c; cin>>a>>b>>c;
42         a--; b--;
43         adj[a][b]=min(adj[a][b], c);
44     }
45     vector<vector<int>> arns = mat_pow(adj,k);
46     if(arns[0][n-1] == INF) cout<<"-1"<<endl;
47     else cout<<arns[0][n-1]<<endl;
48 }

```

8.16 Graph Paths Min-Plus

```

1  const int MAX = 2e6+1;
2  const int MOD = 1e9+7;
3  const int INF = 4e18;
4
5  int modInv(int num){
6      if(num == 1) return 1;
7      return (MOD - ((MOD/num)*modInv(MOD%num))%MOD+MOD)%MOD;
8  }
9
10 vector<vector<int>> mat_zum(vector<vector<int>> &A, vector<vector<
11 int>> &B, int n, int m, int p){
12     vector<vector<int>> arns(n,vector<int> (p, INF));
13     for(int i=0; i<n; i++){
14         for(int j=0; j<m; j++){
15             for(int k=0; k<p; k++){
16                 if (A[i][j] < INF && B[j][k] < INF) {
17                     arns[i][k] = min(arns[i][k], A[i][j] + B[j][k]);
18                 }
19             }
20         }
21     }
22     return arns;
23 }
24
25 vector<vector<int>> mat_pow(vector<vector<int>> &A, int n){
26     int sz = A.size();
27     vector<vector<int>> arns(sz,vector<int> (sz,INF));
28     for(int i = 0; i < sz; i++) arns[i][i] = 0;
29     while(n>0){
30         if(n&1) arns = mat_zum(arns,A,sz,sz,sz);
31         A = mat_zum(A,A,sz,sz,sz);
32         n>>=1;
33     }
34     return arns;
35 }
36
37 void solve(){
38     int n, m, k; cin>>n>>m>>k;
39     vector<vector<int>> adj(n,vector<int>(n,INF));

```

8.17 Log Fibonacci

```

1  const int MAX = 2e6+1;
2  const int MOD = 1e9+7;
3
4  vector<vector<int>> mat_mult(vector<vector<int>> &A, vector<vector<
5  int>> &B, int n, int m, int p){
6      vector<vector<int>> arns(n,vector<int> (p));
7      for(int i=0; i<n; i++){
8          for(int j=0; j<m; j++){
9              for(int k=0; k<p; k++){
10                 arns[i][k] += A[i][j] * B[j][k];
11                 arns[i][k]%=MOD;
12             }
13         }
14     }
15     return arns;
16 }
17
18 vector<vector<int>> mat_pow(vector<vector<int>> &A, int n){
19     vector<vector<int>> arns = {{1,0},{0,1}};
20     while(n>0){
21         if(n&1) arns = mat_mult(arns,A,2,2,2);
22         A = mat_mult(A,A,2,2,2);
23         n>>=1;
24     }
25     return arns;
26 }
27
28 void solve(){
29     int n; cin>>n;
30     vector<vector<int>> a = {{1,1},{1,0}};
31     vector<vector<int>> arns = mat_pow(a,n);
32
33     cout<<arns[0][1]<<endl;
34 }

```

8.18 Matrix Determinant

```

1  const double EPS = 1E-9;
2
3  double matrix_determinant(vector<vector<double>> a) {
4      int n = a.size();
5      double det = 1;
6      for (int i=0; i<n; ++i) {
7          int k = i;
8          for (int j=i+1; j<n; ++j)
9              if (abs (a[j][i]) > abs (a[k][i]))
10                 k = j;
11          if (abs (a[k][i]) < EPS) {
12              det = 0;
13              break;
14          }
15          swap (a[i], a[k]);
16          if (i != k)
17              det = -det;
18          det *= a[i][i];
19          for (int j=i+1; j<n; ++j)
20              a[i][j] /= a[i][i];
21          for (int j=0; j<n; ++j)
22              if (j != i && abs (a[j][i]) > EPS)
23                  for (int k=i+1; k<n; ++k)
24                      a[j][k] -= a[i][k] * a[j][i];
25      }
26      return det;
27 }

```

8.19 Matrix Rank

```

1  const double EPS = 1E-9;
2
3  int compute_rank(vector<vector<double>> A) {
4      int n = A.size();
5      int m = A[0].size();
6
7      int rank = 0;
8      vector<bool> row_selected(n, false);
9      for (int i = 0; i < m; ++i) {
10         int j;
11         for (j = 0; j < n; ++j) {
12             if (!row_selected[j] && abs(A[j][i]) > EPS)
13                 break;
14         }
15
16         if (j != n) {
17             ++rank;
18             row_selected[j] = true;

```

```

19         for (int p = i + 1; p < m; ++p)
20             A[j][p] /= A[j][i];
21         for (int k = 0; k < n; ++k) {
22             if (k != j && abs(A[k][i]) > EPS) {
23                 for (int p = i + 1; p < m; ++p)
24                     A[k][p] -= A[j][p] * A[k][i];
25             }
26         }
27     }
28     return rank;
29 }
30 }

```

8.20 Gauss-Jordan

```

1  const double EPS = 1e-9;
2  const int INF = 2; // it doesn't actually have to be infinity or a
   big number
3
4  int gauss (vector < vector<double> > a, vector<double> & ans) {
5      int n = (int) a.size();
6      int m = (int) a[0].size() - 1;
7
8      vector<int> where (m, -1);
9      for (int col=0, row=0; col<m && row<n; ++col) {
10         int sel = row;
11         for (int i=row; i<n; ++i)
12             if (abs (a[i][col]) > abs (a[sel][col]))
13                 sel = i;
14         if (abs (a[sel][col]) < EPS)
15             continue;
16         for (int i=col; i<=m; ++i)
17             swap (a[sel][i], a[row][i]);
18         where[col] = row;
19
20         for (int i=0; i<n; ++i)
21             if (i != row) {
22                 double c = a[i][col] / a[row][col];
23                 for (int j=col; j<=m; ++j)
24                     a[i][j] -= a[row][j] * c;
25             }
26         ++row;
27     }
28
29     ans.assign (m, 0);
30     for (int i=0; i<m; ++i)
31         if (where[i] != -1)
32             ans[i] = a[where[i]][m] / a[where[i]][i];
33     for (int i=0; i<n; ++i) {
34         double sum = 0;

```

```

35     for (int j=0; j<m; ++j)
36         sum += ans[j] * a[i][j];
37     if (abs (sum - a[i][m]) > EPS)
38         return 0;
39 }
40
41 for (int i=0; i<m; ++i)
42     if (where[i] == -1)
43         return INF;
44 return 1;
45 }

```

8.21 Miller-Rabin (Primality Test)

```

1 // Deterministic Miller-Rabin primality test for unsigned 64-bit
2 // integers.
3 // Purpose: fast primality checks for big n (up to 1e18 / uint64
4 // range).
5 // Complexity per query:  $O(k * \log n)$ , where  $k$  is the number of
6 // bases (here constant 7).
7 // Typical CP usage: primality checks in factorization, number
8 // theory tasks, random tests.
9
10 using u64 = uint64_t;
11 using u128 = __uint128_t;
12
13 u64 mod_mul(u64 a, u64 b, u64 mod) {
14     return (u128)a * b % mod;
15 }
16
17 u64 mod_pow(u64 a, u64 e, u64 mod) {
18     u64 r = 1;
19     while (e > 0) {
20         if (e & 1) r = mod_mul(r, a, mod);
21         a = mod_mul(a, a, mod);
22         e >>= 1;
23     }
24     return r;
25 }
26
27 bool isPrime(u64 n) {
28     if (n < 2) return false;
29
30     // Fast small-prime filter.
31     for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL,
32         23ULL, 29ULL, 31ULL, 37ULL}) {
33         if (n % p == 0) return n == p;
34     }
35
36     // Write  $n - 1 = d * 2^s$  with  $d$  odd.

```

```

32 u64 d = n - 1;
33 int s = 0;
34 while ((d & 1) == 0) {
35     d >>= 1;
36     s++;
37 }
38
39 // Deterministic bases for testing all uint64 values.
40 // Valid for  $n < 2^{64}$ .
41 for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504
42     ULL, 1795265022ULL}) {
43     if (a % n == 0) continue;
44
45     u64 x = mod_pow(a % n, d, n);
46     if (x == 1 || x == n - 1) continue;
47
48     bool witness = true;
49     for (int r = 1; r < s; r++) {
50         x = mod_mul(x, x, n);
51         if (x == n - 1) {
52             witness = false;
53             break;
54         }
55     }
56     if (witness) return false;
57 }
58
59 return true;
60 }
61
62 // Usage example:
63 // u64 n; cin >> n;
64 // cout << (isPrime(n) ? "YES" : "NO") << '\n';

```

8.22 Linear Sieve

```

1 const int N = 10000000;
2 vector<int> lp(N+1);
3 vector<int> pr;
4
5 for (int i=2; i <= N; ++i) {
6     if (lp[i] == 0) {
7         lp[i] = i;
8         pr.push_back(i);
9     }
10    for (int j = 0; i * pr[j] <= N; ++j) {
11        lp[i * pr[j]] = pr[j];
12        if (pr[j] == lp[i]) {
13            break;

```

```

14     }
15 }
16 }

```

8.23 Euler Totient Function

```

1 void phi_1_to_n(int n) {
2     vector<int> phi(n + 1);
3     for (int i = 0; i <= n; i++)
4         phi[i] = i;
5
6     for (int i = 2; i <= n; i++) {
7         if (phi[i] == i) {
8             for (int j = i; j <= n; j += i)
9                 phi[j] -= phi[j] / i;
10        }
11    }
12 }

```

8.24 Fast Totient Function

```

1 int prime[MAX_PRIMES];
2 int SPF[MAX_SIZE];
3 int et[MAX_SIZE];
4 int idx=0;
5
6
7 ll bin_pow(ll x, ll k){
8     ll res=1;
9     while(k){
10        if(k&1){
11            res=(res*x)%MOD;
12        }
13        x=(x*x)%MOD;
14        k/=2;
15    }
16    return res;
17 }
18
19 ll inv(ll x){ return bin_pow(x, MOD-2); }
20
21 void manipulated_seive(int N)
22 {
23
24     for (int i=2; i<N ; i++)
25     {
26

```

```

27         if (SPF[i]==0)
28         {
29             prime[idx++]=i;
30             SPF[i] = i;
31             et[i]=i-1;
32         }
33         else{
34             //et[i]=(SPF[i]-1)*et[i/SPF[i]];
35         }
36         for (int j=0;
37             j < idx &&
38             i*prime[j] < N && prime[j] <= SPF[i];
39             j++)
40         {
41
42
43             if(prime[j]==SPF[i]){
44                 et[i*prime[j]]=prime[j]*et[i];
45             }
46             else{
47                 et[i*prime[j]]=(prime[j]-1)*et[i];
48             }
49             SPF[i*prime[j]] = prime[j] ;
50         }
51     }
52 }
53
54 // driver program to test above function
55 int main()
56 {
57     ios_base::sync_with_stdio(false);
58     cin.tie(NULL);
59     int N,q; cin>>N>>q;
60     //memset(isprime,1,sizeof(isprime));
61     manipulated_seive(N+1);
62
63     ll Q=inv(N-1);
64     for(int i=0;i<q;i++){
65         int x; cin>>x;
66         ll P=(N-x)+et[x]-1;
67         //cout<<et[x]<<endl;
68         cout<<(P*Q)%MOD<<"\n";
69     }
70     return 0;
71 }

```

8.25 Diophantine Equations

```

1 void shift_solution(int & x, int & y, int a, int b, int cnt) {
2     x += cnt * b;

```

```

3   y -= cnt * a;
4 }
5
6 int find_all_solutions(int a, int b, int c, int minx, int maxx, int
7   miny, int maxy) {
8   int x, y, g;
9   if (!find_any_solution(a, b, c, x, y, g))
10    return 0;
11   a /= g;
12   b /= g;
13
14   int sign_a = a > 0 ? +1 : -1;
15   int sign_b = b > 0 ? +1 : -1;
16
17   shift_solution(x, y, a, b, (minx - x) / b);
18   if (x < minx)
19     shift_solution(x, y, a, b, sign_b);
20   if (x > maxx)
21     return 0;
22   int lx1 = x;
23
24   shift_solution(x, y, a, b, (maxx - x) / b);
25   if (x > maxx)
26     shift_solution(x, y, a, b, -sign_b);
27   int rx1 = x;
28
29   shift_solution(x, y, a, b, -(miny - y) / a);
30   if (y < miny)
31     shift_solution(x, y, a, b, -sign_a);
32   if (y > maxy)
33     return 0;
34   int lx2 = x;
35
36   shift_solution(x, y, a, b, -(maxy - y) / a);
37   if (y > maxy)
38     shift_solution(x, y, a, b, sign_a);
39   int rx2 = x;
40
41   if (lx2 > rx2)
42     swap(lx2, rx2);
43   int lx = max(lx1, lx2);
44   int rx = min(rx1, rx2);
45
46   if (lx > rx)
47     return 0;
48   return (rx - lx) / abs(b) + 1;
49 }

```

8.26 Mod Struct

```

1 typedef long long ll;
2
3 struct llm {
4   static const ll MOD=1e9+7;
5   ll v;
6   llm& operator+=(const llm& o){v=(v+o.v)%MOD;return *this;}
7   llm& operator-=(const llm& o){v=(v-o.v+MOD)%MOD;return *this;}
8   llm& operator*=(const llm& o){v=v*o.v%MOD;return *this;}
9   static ll pw(ll x,ll k){ll r=1;while(k){if(k&1)r=r*x%MOD;x=x%
10    MOD;k/=2;}return r;}
11   llm& operator/=(const llm& o){v=v*pw(o.v,MOD-2)%MOD;return *this
12   ;}
13   friend llm operator+(llm l,const llm r){return l+=r;}
14   friend llm operator-(llm l,const llm r){return l-=r;}
15   friend llm operator*(llm l,const llm r){return l*=r;}
16   friend llm operator/(llm l,const llm r){return l/=r;}
17 };

```

8.27 Factorials

```

1 void precompute(ll n = MAXN - 1) {
2   factorial[0] = factorial[1] = 1;
3
4   // Compute factorials
5   for (ll i = 2; i <= n; i++) {
6     factorial[i] = (factorial[i - 1] * i) % MOD;
7   }
8
9   // Compute inverse factorials efficiently
10  inv_factorial[n] = modInv(factorial[n]);
11  for (ll i = n - 1; i >= 0; i--) {
12    inv_factorial[i] =
13      (inv_factorial[i + 1] * (i + 1)) % MOD;
14  }
15 }

```

8.28 Number of Divisors

8.29 Modular Inverse

```

1 ll modInv(ll a, ll mod = MOD) {
2   return power(a, mod - 2, mod);
3 }

```

8.30 Discrete Logarithm

```

1 // Returns minimum x for which  $a^x \bmod m = b \bmod m$ .
2 int solve(int a, int b, int m) {
3     a %= m, b %= m;
4     int k = 1, add = 0, g;
5     while ((g = gcd(a, m)) > 1) {
6         if (b == k)
7             return add;
8         if (b % g)
9             return -1;
10        b /= g, m /= g, ++add;
11        k = (k * 111 * a / g) % m;
12    }
13
14    int n = sqrt(m) + 1;
15    int an = 1;
16    for (int i = 0; i < n; ++i)
17        an = (an * 111 * a) % m;
18
19    unordered_map<int, int> vals;
20    for (int q = 0, cur = b; q <= n; ++q) {
21        vals[cur] = q;
22        cur = (cur * 111 * a) % m;
23    }
24
25    for (int p = 1, cur = k; p <= n; ++p) {
26        cur = (cur * 111 * an) % m;
27        if (vals.count(cur)) {
28            int ans = n * p - vals[cur] + add;
29            return ans;
30        }
31    }
32    return -1;
33 }

```

8.31 Segmented Sieve

```

1 vector<char> segmentedSieve(long long L, long long R) {
2     // generate all primes up to sqrt(R)
3     long long lim = sqrt(R);
4     vector<char> mark(lim + 1, false);
5     vector<long long> primes;
6     for (long long i = 2; i <= lim; ++i) {
7         if (!mark[i]) {
8             primes.emplace_back(i);
9             for (long long j = i * i; j <= lim; j += i)
10                mark[j] = true;
11        }
12    }

```

```

13
14    vector<char> isPrime(R - L + 1, true);
15    for (long long i : primes)
16        for (long long j = max(i * i, (L + i - 1) / i * i); j <= R;
17             j += i)
18            isPrime[j - L] = false;
19    if (L == 1)
20        isPrime[0] = false;
21    return isPrime;

```

8.32 Sieve of Eratosthenes

```

1 vector<bool> sieve_eratosthenes(int n) {
2     vector<bool> is_prime(n+1, true);
3     is_prime[0] = is_prime[1] = false;
4     for (int i = 2; i * i <= n; i++) {
5         if (is_prime[i]) {
6             for (int j = i * i; j <= n; j += i)
7                 is_prime[j] = false;
8         }
9     }
10    return is_prime;
11 }
12
13 int count_primes(int n) {
14     const int S = 10000;
15
16     vector<int> primes;
17     int nsqrt = sqrt(n);
18     vector<char> is_prime(nsqrt + 2, true);
19     for (int i = 2; i <= nsqrt; i++) {
20         if (is_prime[i]) {
21             primes.push_back(i);
22             for (int j = i * i; j <= nsqrt; j += i)
23                 is_prime[j] = false;
24         }
25     }
26
27     int result = 0;
28     vector<char> block(S);
29     for (int k = 0; k * S <= n; k++) {
30         fill(block.begin(), block.end(), true);
31         int start = k * S;
32         for (int p : primes) {
33             int start_idx = (start + p - 1) / p;
34             int j = max(start_idx, p) * p - start;
35             for (; j < S; j += p)
36                 block[j] = false;
37         }

```

```

38     if (k == 0)
39         block[0] = block[1] = false;
40     for (int i = 0; i < S && start + i <= n; i++) {
41         if (block[i])
42             result++;
43     }
44 }
45 return result;
46 }

```

8.33 Simpson Method

```

1 const int N = 1000 * 1000; // number of steps (already multiplied by
2     2)
3 double simpson_integration(double a, double b){
4     double h = (b - a) / N;
5     double s = f(a) + f(b); // a = x_0 and b = x_2n
6     for (int i = 1; i <= N - 1; ++i) { // Refer to final Simpson's
7         formula
8         double x = a + h * i;
9         s += f(x) * ((i & 1) ? 4 : 2);
10    }
11    s *= h / 3;
12    return s;
13 }

```

8.34 Precompute Pascal Triangle

```

1 int binomialCoeff(int n, int k) {
2     vector<int> dp(k + 1);
3
4     // nC0 is 1
5     dp[0] = 1;
6
7     for (int i = 1; i <= n; i++) {
8
9         // Compute next row of pascal triangle using
10        // the previous row
11        for (int j = min(i, k); j > 0; j--)
12            dp[j] = dp[j] + dp[j - 1];
13    }
14    return dp[k];
15 }

```

8.35 Sqrt and Inverse Sqrt

```

1 double sqrt_newton(double n) {
2     const double eps = 1E-15;
3     double x = 1;
4     for (;;) {
5         double nx = (x + n / x) / 2;
6         if (abs(x - nx) < eps)
7             break;
8         x = nx;
9     }
10    return x;
11 }
12
13 int isqrt_newton(int n) {
14     int x = 1;
15     bool decreased = false;
16     for (;;) {
17         int nx = (x + n / x) >> 1;
18         if (x == nx || nx > x && decreased)
19             break;
20         decreased = nx < x;
21         x = nx;
22     }
23     return x;
24 }

```

8.36 Sum of Divisors

```

1 long long SumOfDivisors(long long num) {
2     long long total = 1;
3
4     for (int i = 2; (long long)i * i <= num; i++) {
5         if (num % i == 0) {
6             int e = 0;
7             do {
8                 e++;
9                 num /= i;
10            } while (num % i == 0);
11
12            long long sum = 0, pow = 1;
13            do {
14                sum += pow;
15                pow *= i;
16            } while (e-- > 0);
17            total *= sum;
18        }
19    }
20    if (num > 1) {
21        total *= (1 + num);
22    }
23 }

```

```

22     }
23     return total;
24 }

```

8.37 Ternary Search

```

1 double ternary_search(double l, double r) {
2     double eps = 1e-9;           //set the error limit here
3     while (r - l > eps) {
4         double m1 = l + (r - l) / 3;
5         double m2 = r - (r - l) / 3;
6         double f1 = f(m1);       //evaluates the function at m1
7         double f2 = f(m2);       //evaluates the function at m2
8         if (f1 < f2)
9             l = m1;
10        else
11            r = m2;
12    }
13    return f(l);                 //return the maximum of f(x) in
14    [l, r]

```

8.38 Trivial Factorization

```

1 vector<long long> trial_division2(long long n) {
2     vector<long long> factorization;
3     while (n % 2 == 0) {
4         factorization.push_back(2);
5         n /= 2;
6     }
7     for (long long d = 3; d * d <= n; d += 2) {
8         while (n % d == 0) {
9             factorization.push_back(d);
10            n /= d;
11        }
12    }
13    if (n > 1)
14        factorization.push_back(n);
15    return factorization;
16 }

```

8.39 Chinese Remainder Theorem

```

1 #include <bits/stdc++.h>

```

```

2 using namespace std;
3 typedef long long ll;
4
5 // Extended GCD returning gcd and setting x, y such that ax + by =
6 // gcd(a, b)
7 ll extgcd(ll a, ll b, ll &x, ll &y) {
8     if (b == 0) {
9         x = 1;
10        y = 0;
11        return a;
12    }
13    ll x1, y1;
14    ll g = extgcd(b, a % b, x1, y1);
15    x = y1;
16    y = x1 - (a / b) * y1;
17    return g;
18 }
19
20 // Chinese Remainder Theorem
21 // Given x == a[i] (mod m[i]) for pairwise coprime m[i], returns (
22 // x_mod, lcm)
23 // where x is the unique solution modulo lcm = product of m[i]
24 pair<ll, ll> crt(const vector<ll>& a, const vector<ll>& m) {
25     ll x = 0;
26     ll M = 1;
27
28     for (size_t i = 0; i < a.size(); i++) {
29         ll ai = ((a[i] % m[i]) + m[i]) % m[i];
30         ll mi = m[i];
31
32         // Solve: M * t == (ai - x) (mod mi)
33         ll diff = (ai - x) % mi;
34         if (diff < 0) diff += mi;
35
36         ll t, y;
37         ll g = extgcd(M, mi, t, y);
38         // M and mi are coprime, so g == 1
39         t = (t % mi + mi) % mi;
40         t = (__int128)t * diff % mi;
41
42         x = x + (__int128)M * t;
43         M = (__int128)M * mi / g;
44
45         x %= M;
46         if (x < 0) x += M;
47     }
48
49     return {x, M};

```

8.40 Garner's Algorithm

```

1 // Garner's Algorithm for Chinese Remainder Theorem with non-coprime
  moduli
2 // Solves  $x \equiv a_i \pmod{m_i}$  for arbitrary moduli
3 // Returns  $x \pmod{\text{lcm}(m_i)}$ , or -1 if no solution
4
5 #include <bits/stdc++.h>
6 using namespace std;
7 using ll = long long;
8
9 ll egcd(ll a, ll b, ll &x, ll &y) {
10     if (b == 0) { x = 1; y = 0; return a; }
11     ll x1, y1;
12     ll g = egcd(b, a % b, x1, y1);
13     x = y1;
14     y = x1 - (a / b) * y1;
15     return g;
16 }
17
18 // Extended modular inverse that works for non-coprime case
19 // Returns inverse of a modulo mod, or -1 if doesn't exist
20 // When  $\text{gcd}(a, \text{mod}) = d > 1$ , the inverse of  $(a/d)$  modulo  $(\text{mod}/d)$  is
  returned
21 // and the gcd is returned in g
22 ll modInvExt(ll a, ll mod, ll &g) {
23     ll x, y;
24     g = egcd(a, mod, x, y);
25     if (g != 1) return -1;
26     return (x % mod + mod) % mod;
27 }
28
29 // Check if system  $x \equiv a[i] \pmod{m[i]}$  is consistent
30 bool checkConsistency(const vector<ll> &a, const vector<ll> &m) {
31     int n = a.size();
32     for (int i = 0; i < n; i++) {
33         for (int j = i + 1; j < n; j++) {
34             ll g = std::gcd(m[i], m[j]);
35             if ((a[j] - a[i]) % g != 0) return false;
36         }
37     }
38     return true;
39 }
40
41 // Compute lcm of all moduli
42 ll lcmAll(const vector<ll> &m) {
43     ll res = 1;
44     for (ll mod : m) {
45         res = std::lcm(res, mod);
46     }
47     return res;
48 }

```

```

49
50 // Garner's algorithm for arbitrary (not necessarily coprime) moduli
51 // Returns solution  $x \pmod{\text{lcm}(m_i)}$ , or -1 if no solution exists
52 // Uses Garner's recurrence with precomputed inverses for coprime
  components
53 ll garner(const vector<ll> &a, const vector<ll> &m) {
54     int n = a.size();
55     if (n == 0) return 0;
56
57     // Check consistency of the system
58     if (!checkConsistency(a, m)) return -1;
59
60     // Build solution iteratively using Garner's recurrence
61     // At step i: we have cur satisfying congruences  $0..i-1$ 
62     // We want to find t such that:  $\text{cur} + Q * t \equiv a[i] \pmod{m[i]}$ 
63     // where  $Q = \text{lcm}(m[0], \dots, m[i-1])$ 
64     //
65     // This is equivalent to  $Q * t \equiv (a[i] - \text{cur}) \pmod{m[i]}$ 
66     // Let  $d = \text{gcd}(Q, m[i])$ . For solution to exist,  $d \mid (a[i] - \text{cur})$ 
67     // Then:  $(Q/d) * t \equiv (a[i] - \text{cur})/d \pmod{m[i]/d}$ 
68     // Since  $\text{gcd}(Q/d, m[i]/d) = 1$ , we can find inverse
69
70     ll cur = a[0] % m[0];
71     if (cur < 0) cur += m[0];
72     ll Q = m[0]; // lcm of processed moduli
73
74     for (int i = 1; i < n; i++) {
75         ll diff = a[i] - cur;
76         ll mi = m[i];
77
78         //  $d = \text{gcd}(Q, m_i)$ 
79         ll d = std::gcd(Q, mi);
80
81         // Check divisibility: solution exists iff  $d \mid \text{diff}$ 
82         if (diff % d != 0) return -1; // Should not happen if
          consistency check passed
83
84         // Reduce:  $(Q/d) * t \equiv (\text{diff}/d) \pmod{m_i/d}$ 
85         ll Qd = Q / d;
86         ll mid = mi / d;
87         ll diffd = diff / d;
88
89         // Find inverse of Qd modulo mid
90         ll inv, g;
91         inv = modInvExt(Qd % mid, mid, g);
92         if (inv == -1) return -1; // Should not happen
93
94         //  $t = (\text{diffd} * \text{inv}) \pmod{\text{mid}}$ 
95         ll t = (diffd % mid * inv) % mid;
96         if (t < 0) t += mid;
97
98         // Update cur and Q
99         cur = cur + Q * t;

```

```

100     Q = std::lcm(Q, mi);
101
102     // Normalize cur to be within [0, Q)
103     cur %= Q;
104     if (cur < 0) cur += Q;
105 }
106
107 return cur % lcmAll(m);
108 }

```

8.41 Polynomial Operations

```

1 // Polynomial Operations class
2 // Supports +, -, *, /, %, eval, and interpolate over arbitrary type
3 // Uses __int128 for safe multiplication and reduces mod 1e9+7 when
4 // needed
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 typedef long long ll;
9 const ll MOD = 1e9 + 7;
10
11 template<typename T>
12 struct Poly {
13     vector<T> a;
14
15     // Constructors
16     Poly() {}
17     Poly(const vector<T>& v) : a(v) {}
18     Poly(const T& scalar) : a(1, scalar) {}
19
20     // Degree, -1 for zero polynomial
21     int deg() const {
22         return (int)a.size() - 1;
23     }
24
25     // Ensure no trailing zeros
26     void normalize() {
27         while (!a.empty() && a.back() == T(0)) a.pop_back();
28     }
29
30     // Coefficient access
31     T& operator[](int i) { return a[i]; }
32     const T& operator[](int i) const { return a[i]; }
33
34     // Addition
35     Poly operator+(const Poly& o) const {
36         Poly res;

```

```

37         res.a.resize(max(size(), o.size()));
38         for (size_t i = 0; i < res.a.size(); i++) {
39             T x = i < size() ? a[i] : T(0);
40             T y = i < o.size() ? o.a[i] : T(0);
41             res.a[i] = x + y;
42         }
43         res.normalize();
44         return res;
45     }
46
47     // Subtraction
48     Poly operator-(const Poly& o) const {
49         Poly res;
50         res.a.resize(max(size(), o.size()));
51         for (size_t i = 0; i < res.a.size(); i++) {
52             T x = i < size() ? a[i] : T(0);
53             T y = i < o.size() ? o.a[i] : T(0);
54             res.a[i] = x - y;
55         }
56         res.normalize();
57         return res;
58     }
59
60     // Scalar multiplication
61     Poly operator*(const T& scalar) const {
62         Poly res;
63         res.a.resize(size());
64         for (size_t i = 0; i < size(); i++) {
65             res.a[i] = a[i] * scalar;
66         }
67         res.normalize();
68         return res;
69     }
70
71     // Polynomial multiplication using __int128 for intermediate
72     // results
73     Poly operator*(const Poly& o) const {
74         if (deg() < 0 || o.deg() < 0) return Poly();
75
76         Poly res;
77         res.a.resize(size() + o.size() - 1);
78         for (size_t i = 0; i < size(); i++) {
79             for (size_t j = 0; j < o.size(); j++) {
80                 __int128 prod = (__int128)a[i] * (__int128)o.a[j];
81                 ll prod_ll = (ll)(prod % MOD);
82                 res.a[i + j] = (T)((ll)res.a[i + j] + prod_ll);
83                 if (sizeof(T) != sizeof(ll)) {
84                     // Keep values bounded for non-ll types
85                     ll val = (ll)res.a[i + j] % MOD;
86                     if (val < 0) val += MOD;
87                     res.a[i + j] = (T)val;
88                 }
89             }
90         }
91         res.normalize();
92         return res;
93     }

```

```

89     }
90     res.normalize();
91     return res;
92 }
93
94 // Division with remainder ( Euclidean division )
95 // Returns quotient, remainder is stored in this object
96 Poly divmod(const Poly& o, Poly& remainder) const {
97     if (deg() < 0) {
98         remainder = Poly();
99         return Poly();
100     }
101
102     if (o.deg() < 0) {
103         // Division by zero - should not happen in well-formed
104         // code
105         remainder = *this;
106         return Poly();
107     }
108
109     int n = deg();
110     int m = o.deg();
111     if (n < m) {
112         remainder = *this;
113         return Poly();
114     }
115
116     Poly q;
117     q.a.resize(n - m + 1);
118     remainder = *this;
119
120     while (remainder.deg() >= m && remainder.deg() >= 0) {
121         int d = remainder.deg() - m;
122         T coef = remainder.a.back() / o.a.back();
123
124         for (int i = 0; i <= m; i++) {
125             remainder.a[d + i] = remainder.a[d + i] - coef * o.a[i];
126         }
127         q.a[d] = coef;
128         remainder.normalize();
129     }
130
131     q.normalize();
132     remainder.normalize();
133     return q;
134 }
135
136 // Division operator (quotient only)
137 Poly operator/(const Poly& o) const {
138     Poly rem;
139     return divmod(o, rem);

```

```

140
141 // Modulo operator (remainder only)
142 Poly operator%(const Poly& o) const {
143     Poly rem;
144     divmod(o, rem);
145     return rem;
146 }
147
148 // Evaluate polynomial at point x using Horner's method
149 T eval(T x) const {
150     if (deg() < 0) return T(0);
151     T result = a.back();
152     for (int i = deg() - 1; i >= 0; i--) {
153         result = result * x + a[i];
154     }
155     return result;
156 }
157
158 // Interpolate: given xs and ys, find polynomial P such that P[
159 // xs[i]] = ys[i]
160 // Uses Lagrange interpolation formula
161 static Poly interpolate(const vector<T>& xs, const vector<T>& ys
162 ) {
163     int n = xs.size();
164     if (n == 0) return Poly();
165     if (n == 1) return Poly(ys[0]);
166
167     // Build Lagrange basis polynomials efficiently
168     // Precompute prefix and suffix products
169     vector<Poly> pref(n), suff(n);
170     for (int i = 0; i < n; i++) {
171         pref[i] = Poly({-xs[i], T(1)});
172         suff[i] = Poly({-xs[i], T(1)});
173     }
174     for (int i = 1; i < n; i++) {
175         pref[i] = pref[i] * pref[i - 1];
176     }
177     for (int i = n - 2; i >= 0; i--) {
178         suff[i] = suff[i] * suff[i + 1];
179     }
180
181     Poly result;
182     for (int i = 0; i < n; i++) {
183         Poly num = (i == 0 ? Poly(T(1)) : pref[i - 1]) *
184             (i == n - 1 ? Poly(T(1)) : suff[i + 1]);
185
186         // Compute denominator: prod_{j != i} (xs[i] - xs[j])
187         T den = T(1);
188         for (int j = 0; j < n; j++) {
189             if (i != j) {
190                 den = den * (xs[i] - xs[j]);
191             }
192         }

```

```

191 // L_i(x) = num(x) / den
192 // We need to handle division - for integers mod MOD,
193 // use modular inverse
194 // For general T, we assume den divides num evenly or T
195 // supports division
196 Poly term = num * (ys[i] / den);
197 result = result + term;
198
199 result.normalize();
200 return result;
201 }
202
203 // Size helper
204 size_t size() const { return a.size(); }
205 };
206
207 // Explicit template instantiations for common types
208 template struct Poly<ll>;
209 template struct Poly<int>;
210 template struct Poly<long double>;
211 template struct Poly<double>;
212
213 void solve() {
214 // Example usage:
215 // Polynomial addition, subtraction, multiplication
216 Poly<ll> p1({1, 2, 3}); // 1 + 2x + 3x^2
217 Poly<ll> p2({1, 1}); // 1 + x
218
219 auto p3 = p1 + p2; // 2 + 3x + 3x^2
220 auto p4 = p1 - p2; // 0 + 1x + 3x^2
221 auto p5 = p1 * p2; // (1 + 2x + 3x^2)(1 + x)
222
223 // Division: p1 / p2 = quotient and remainder
224 Poly<ll> rem;
225 Poly<ll> q = p1.divmod(p2, rem);
226
227 // Evaluation
228 (void)p1.eval(11(2)); // p1(2) = 1 + 4 + 12 = 17
229
230 // Interpolation
231 vector<ll> xs = {0, 1, 2};
232 vector<ll> ys = {1, 3, 7}; // P(0)=1, P(1)=3, P(2)=7 -> P(x)
233 // = 1+2x+x^2
234 Poly<ll> interp = Poly<ll>::interpolate(xs, ys);

```

```

1 // Linear Recurrence solver using Berlekamp-Massey + Kitamasa
2 // O(k^3 log n) where k is the recurrence order
3 // Mod 1e9+7
4
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 using ll = long long;
9 const ll MOD = 1e9 + 7;
10
11 ll modpow(ll a, ll e) {
12     ll r = 1;
13     while (e) {
14         if (e & 1) r = r * a % MOD;
15         a = a * a % MOD;
16         e >>= 1;
17     }
18     return r;
19 }
20
21 // Berlekamp-Massey: find minimal linear recurrence relation
22 // Returns coefficients c where sequence s satisfies:
23 // s[n] = c[0]*s[n-1] + c[1]*s[n-2] + ... + c[k-1]*s[n-k]
24 vector<ll> berlekampMassey(const vector<ll> &s) {
25     vector<ll> c, oldC;
26     int f = -1;
27     for (int i = 0; i < (int)s.size(); i++) {
28         ll delta = s[i] % MOD;
29         for (int j = 1; j <= (int)c.size(); j++) {
30             delta = (delta - c[j - 1] * s[i - j]) % MOD;
31         }
32         if (delta == 0) continue;
33         if (f == -1) {
34             c.assign(i + 1, 0);
35             c[i] = 1;
36             f = i;
37         } else {
38             vector<ll> d = oldC;
39             for (ll &x : d) x = (MOD - x) % MOD;
40             d.insert(d.begin(), 1);
41             ll df1 = 0;
42             for (int j = 1; j <= (int)d.size(); j++) {
43                 df1 = (df1 + d[j - 1] * s[f + 1 - j]) % MOD;
44             }
45             ll coef = delta * modpow(df1, MOD - 2) % MOD;
46             for (ll &x : d) x = x * coef % MOD;
47             d.insert(d.end(), i - f - 1, 0);
48             vector<ll> temp = c;
49             c.resize(max(c.size(), d.size()));
50             for (int j = 0; j < (int)d.size(); j++)
51                 c[j] = (c[j] + d[j]) % MOD;
52             if (i - (int)temp.size() > f - (int)oldC.size()) {
53                 oldC = temp;

```

8.42 Linear Recurrence

```

54         f = i;
55     }
56 }
57 }
58 // Normalize: leading coefficient should be 1
59 if (!c.empty() && c.back() != 1) {
60     ll inv = modpow(c.back(), MOD - 2);
61     for (ll &x : c) x = x * inv % MOD;
62 }
63 return c;
64 }
65
66 // Kitamasa: combine two polynomials modulo the recurrence relation
67 vector<ll> combine(const vector<ll> &a, const vector<ll> &b, const
68 vector<ll> &trans) {
69     int k = trans.size();
70     vector<ll> res(2 * k);
71     for (int i = 0; i < k; i++) {
72         for (int j = 0; j < k; j++) {
73             res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
74         }
75     }
76     // Reduce degree >= k terms using the recurrence
77     for (int i = 2 * k - 1; i >= k; i--) {
78         for (int j = 1; j <= k; j++) {
79             res[i - j] = (res[i - j] + res[i] * trans[j - 1]) % MOD;
80         }
81     }
82     res.resize(k);
83     return res;
84 }
85 // Kitamasa: compute coefficients for a^n
86 vector<ll> kitamasa(ll n, const vector<ll> &trans) {
87     int k = trans.size();
88     vector<ll> res(k), coef(k);
89     res[0] = 1;
90     coef[1 % k] = 1; // Kitamasa(1) in the semiring: [0, 1, 0, ...]
91     while (n > 0) {
92         if (n & 1) res = combine(res, coef, trans);
93         coef = combine(coef, coef, trans);
94         n >>= 1;
95     }
96     return res;
97 }
98
99 // Compute the n-th term of a linear recurrence
100 // init: first k terms of the sequence
101 // trans: recurrence coefficients [c0, c1, ..., c_{k-1}]
102 // meaning s[n] = c0*s[n-1] + c1*s[n-2] + ... + c_{k-1}*s[n-k]
103 ll linear_recurrence(vector<ll> init, vector<ll> trans, ll n) {
104     int k = trans.size();

```

```

105     if (k == 0) return 0;
106     if (n < (ll)init.size()) return init[n] % MOD;
107     vector<ll> coef = kitamasa(n, trans);
108     ll res = 0;
109     for (int i = 0; i < k; i++) {
110         res = (res + coef[i] * init[i]) % MOD;
111     }
112     return res;
113 }

```

8.43 Matrix Exponentiation

```

1 // Matrix exponentiation: A^n in O(k^3 log n) for k×k matrix.
2 // Templated on element type (ll, modint, etc.).
3
4 #include <vector>
5
6 template<class T>
7 struct Matrix {
8     int n;
9     std::vector<std::vector<T>> &a;
10
11     Matrix() {}
12     Matrix(int _n, int _m, T val = T{}) : n(_n), a(_n, std::vector<T>
13         >(_m, val)) {}
14
15     static Matrix identity(int n) {
16         Matrix res(n, n);
17         for (int i = 0; i < n; i++)
18             res.a[i][i] = T{1};
19         return res;
20     }
21
22     Matrix operator+(const Matrix& o) const {
23         Matrix res(n, o.n);
24         for (int i = 0; i < n; i++)
25             for (int j = 0; j < n; j++)
26                 res.a[i][j] = a[i][j] + o.a[i][j];
27         return res;
28     }
29
30     Matrix operator*(const Matrix& o) const {
31         int m = o.n;
32         Matrix res(n, m);
33         for (int i = 0; i < n; i++)
34             for (int k = 0; k < n; k++) if (a[i][k] != T{})
35                 for (int j = 0; j < m; j++)
36                     res.a[i][j] += a[i][k] * o.a[k][j];
37         return res;
38     }
39 }

```

```

38
39 Matrix pow(long long e) const {
40     Matrix res = identity(n);
41     Matrix base = *this;
42     while (e > 0) {
43         if (e & 1) res = res * base;
44         base = base * base;
45         e >>= 1;
46     }
47     return res;
48 }
49 };

```

8.44 Modular Square Root

```

1 // Returns one x such that x^2 == a (mod p), where p is prime.
2 // The other root is p - x. Returns -1 if no square root exists.
3
4 typedef long long ll;
5
6 static ll mod_pow(ll a, ll e, ll mod) {
7     ll r = 1;
8     a %= mod;
9     while (e > 0) {
10         if (e & 1) r = r * a % mod;
11         a = a * a % mod;
12         e >>= 1;
13     }
14     return r;
15 }
16
17 ll mod_sqrt(ll a, ll p) {
18     a %= p;
19     if (a < 0) a += p;
20
21     if (a == 0) return 0;
22     if (p == 2) return a;
23
24     // Legendre symbol: a^((p-1)/2) mod p == 1 -> quadratic residue
25     //                               == p-1 -> non-residue
26     if (mod_pow(a, (p - 1) / 2, p) != 1) return -1;
27
28     // p == 3 (mod 4): simple closed form
29     if (p % 4 == 3) return mod_pow(a, (p + 1) / 4, p);
30
31     // p == 1 (mod 4): Tonelli-Shanks
32     // Factor p-1 = q * 2^s with q odd
33     ll q = p - 1;
34     int s = 0;
35     while ((q & 1) == 0) {

```

```

36         q >>= 1;
37         ++s;
38     }
39
40     // Find a quadratic non-residue z
41     ll z = 2;
42     while (mod_pow(z, (p - 1) / 2, p) != p - 1) ++z;
43
44     ll c = mod_pow(z, q, p); // c = z^q mod p
45     ll x = mod_pow(a, (q + 1) / 2, p); // x = a^{(q+1)/2} mod p
46     ll t = mod_pow(a, q, p); // t = a^q mod p
47     int m = s;
48
49     for (int i = 1; i < m; ++i) {
50         ll tp = t;
51         int d = 1;
52         while (d < m - i) {
53             tp = tp * tp % p;
54             d <<= 1;
55         }
56         if (tp != 1) {
57             x = x * mod_pow(c, 1 << (m - i - 1), p) % p;
58             c = c * c % p;
59             t = t * c % p;
60         }
61     }
62
63     return x;
64 }

```

8.45 Simplex

```

1 /**
2  * Simplex algorithm for linear programming.
3  * Solves max c*x subject to Ax <= b, x >= 0.
4  * Uses two-phase simplex with Bland's rule for guaranteed
5  * termination.
6  *
7  * Usage:
8  *     Simplex simplex(n); // n = number of variables
9  *     simplex.set_objective(c); // c = objective coefficients
10 *     simplex.add_constraint(a, b); // Ax <= b
11 *     auto [max_val, solution] = simplex.solve();
12 */
13 #include <bits/stdc++.h>
14 using namespace std;
15
16 const double INF = 1e100;
17 const double EPS = 1e-9;

```

```

66 for (int i = 0; i < m; i++) obj[n + m + i] = 1.0;
67
68 // Basic variables are artificial vars
69 vector<int> basic(M);
70 for (int i = 0; i < M; i++) basic[i] = n + m + i;
71
72 // Eliminate artificial vars from objective row
73 for (int i = 0; i < M; i++) {
74     if (basic[i] >= n + m) {
75         for (int j = 0; j <= N; j++) obj[j] -= T[i][j];
76     }
77 }
78
79 // Run simplex on phase 1
80 bool infeasible = !simplex_iterate(T, obj, basic, N, M);
81
82 // Check feasibility: artificial vars must be zero
83 double phase1_val = -obj[N];
84 for (int i = 0; i < M; i++) phase1_val -= obj[basic[i]] * T[
    i][N];
85
86 if (phase1_val > EPS || infeasible) {
87     return {-INF, {}};
88 }
89
90 // Phase 2: build new tableau without artificial variables
91 // Keep current BFS by carrying over basic variables
92 int N2 = n + m;
93 vector<vector<double>> T2(M, vector<double>(N2 + 1, 0.0));
94
95 for (int i = 0; i < M; i++) {
96     for (int j = 0; j < n; j++) T2[i][j] = A[i][j];
97     T2[i][n + i] = 1.0;
98     T2[i][N2] = b[i];
99 }
100
101 // Determine new basic variables
102 vector<int> basic2(M, -1);
103 for (int i = 0; i < M; i++) {
104     if (basic[i] < n + m) {
105         basic2[i] = basic[i];
106     }
107 }
108
109 // Pivot out artificial variables that are still basic
110 for (int i = 0; i < M; i++) {
111     if (basic2[i] == -1) {
112         // Find a non-artificial variable to pivot in
113         for (int j = 0; j < N2; j++) {
114             if (abs(T2[i][j]) > EPS) {
115                 pivot(T2, basic2, i, j, M, N2);
116                 break;
117             }

```

```

118     }
119 }
120
121 // Set up phase 2 objective
122 vector<double> obj2(N2 + 1, 0.0);
123 for (int j = 0; j < n; j++) obj2[j] = c[j];
124
125 // Eliminate basic variables from objective
126 for (int i = 0; i < M; i++) {
127     if (basic2[i] != -1 && basic2[i] < N2) {
128         double coeff = obj2[basic2[i]];
129         if (abs(coeff) > EPS) {
130             for (int j = 0; j <= N2; j++) obj2[j] -= coeff *
131                 T2[i][j];
132         }
133     }
134 }
135
136 // Run simplex on phase 2
137 bool unbounded = !simplex_iterate(T2, obj2, basic2, N2, M);
138
139 if (unbounded) {
140     return {INF, {}};
141 }
142
143 // Extract solution
144 vector<double> x(n, 0.0);
145 for (int i = 0; i < M; i++) {
146     if (basic2[i] >= 0 && basic2[i] < n) {
147         x[basic2[i]] = T2[i][N2];
148     }
149 }
150
151 double max_val = -obj2[N2];
152 for (int i = 0; i < M; i++) {
153     if (basic2[i] >= 0) max_val -= obj2[basic2[i]] * T2[i][
154         N2];
155 }
156
157 return {max_val, x};
158 }
159 private:
160 void pivot(vector<vector<double>>& T, vector<int>& basic, int
161     row, int col, int m, int n) {
162     double piv = T[row][col];
163     if (abs(piv) < EPS) return;
164
165     for (int j = 0; j <= n; j++) T[row][j] /= piv;
166     for (int i = 0; i < m; i++) {
167         if (i != row && abs(T[i][col]) > EPS) {
168             double factor = T[i][col];

```

```

168         for (int j = 0; j <= n; j++) T[i][j] -= factor * T[
169             row][j];
170     }
171     basic[row] = col;
172 }
173
174 bool simplex_iterate(vector<vector<double>>& T, vector<double>&
175     obj, vector<int>& basic, int n, int m) {
176     while (true) {
177         // Bland's rule: entering variable is smallest index
178         // with negative reduced cost
179         int entering = -1;
180         for (int j = 0; j < n; j++) {
181             if (obj[j] < -EPS) {
182                 entering = j;
183                 break;
184             }
185         }
186         if (entering == -1) break;
187
188         // Minimum ratio test with Bland's rule for leaving
189         // variable
190         int leaving = -1;
191         double min_ratio = INF;
192         for (int i = 0; i < m; i++) {
193             if (T[i][entering] > EPS) {
194                 double ratio = T[i][n] / T[i][entering];
195                 if (ratio < min_ratio - EPS || (abs(ratio -
196                     min_ratio) < EPS && basic[i] < basic[leaving
197                         ])) {
198                     min_ratio = ratio;
199                     leaving = i;
200                 }
201             }
202         }
203         if (leaving == -1) return false; // unbounded
204
205         pivot(T, basic, leaving, entering, m, n);
206
207         double coeff = obj[entering];
208         if (abs(coeff) > EPS) {
209             for (int j = 0; j <= n; j++) obj[j] -= coeff * T[
210                 leaving][j];
211         }
212     }
213     return true;
214 }
215 };

```

9 String Algorithms

9.1 Aho-Corasick

```

1 // Aho-Corasick Automaton (lowercase a-z)
2 struct AhoCorasick {
3     static const int ALPH = 26;
4
5     struct Node {
6         int next[ALPH];
7         int link;
8         vector<int> out; // pattern ids ending here
9
10        Node() : link(0) {
11            fill(next, next + ALPH, -1);
12        }
13    };
14
15    vector<Node> t;
16    vector<int> patLen;
17
18    AhoCorasick() { t.push_back(Node()); }
19
20    int addString(const string& s) {
21        int v = 0;
22        for (char ch : s) {
23            int c = ch - 'a';
24            if (t[v].next[c] == -1) {
25                t[v].next[c] = (int)t.size();
26                t.push_back(Node());
27            }
28            v = t[v].next[c];
29        }
30        int id = (int)patLen.size();
31        patLen.push_back((int)s.size());
32        t[v].out.push_back(id);
33        return id;
34    }
35
36    void build() {
37        queue<int> q;
38
39        for (int c = 0; c < ALPH; c++) {
40            if (t[0].next[c] == -1) {
41                t[0].next[c] = 0;
42            } else {
43                int u = t[0].next[c];
44                t[u].link = 0;
45                q.push(u);
46            }
47        }

```

```

48
49        while (!q.empty()) {
50            int v = q.front();
51            q.pop();
52
53            for (int c = 0; c < ALPH; c++) {
54                int u = t[v].next[c];
55                if (u == -1) {
56                    t[v].next[c] = t[t[v].link].next[c];
57                    continue;
58                }
59
60                t[u].link = t[t[v].link].next[c];
61                for (int id : t[t[u].link].out) t[u].out.push_back(id);
62                q.push(u);
63            }
64        }
65    }
66
67    // Returns all matches as {ending_index_in_text, pattern_id}
68    vector<pair<int, int>> findAll(const string& text) const {
69        vector<pair<int, int>> ans;
70        int v = 0;
71        for (int i = 0; i < (int)text.size(); i++) {
72            int c = text[i] - 'a';
73            v = t[v].next[c];
74            for (int id : t[v].out) {
75                ans.push_back({i, id});
76            }
77        }
78        return ans;
79    }
80 };
81
82 // Example of use:
83 // Count occurrences of each pattern in text.
84 vector<int> countPatternOccurrences(const string& text, const vector
85 <string>& patterns) {
86     AhoCorasick ac;
87     for (const string& p : patterns) ac.addString(p);
88     ac.build();
89
90     vector<int> cnt(patterns.size(), 0);
91     auto matches = ac.findAll(text);
92     for (auto [endIdx, id] : matches) {
93         (void)endIdx;
94         cnt[id]++;
95     }
96     return cnt;
97 }

```

9.2 DP on Trie

```

1 #ifndef MOD
2 #define MOD 1000000007
3 #endif
4
5 class Trie{
6 private:
7     Trie *children[26];
8     bool term_st; // is_end
9
10    Trie* searchPrefix(const string &p){
11        Trie* node = this;
12
13        for(char c:p){
14            int pos = c - 'a';
15            if(!node->children[pos]) return nullptr;
16            node = node->children[pos];
17        }
18        return node;
19    }
20
21 public:
22    Trie() : term_st(false) {
23        for(int i=0; i<26; i++)
24            children[i] = nullptr;
25    }
26
27    void insert(string word) {
28        Trie* node = this;
29        for(char c:word){
30            int pos = c - 'a';
31            if(!node->children[pos]) node->children[pos] = new Trie
32                ();
33            node = node->children[pos];
34        }
35        node->term_st = true;
36    }
37
38    bool search(string word) {
39        Trie* node = searchPrefix(word);
40        return node != nullptr && node->term_st;
41    }
42
43    bool startsWith(string prefix) {
44        Trie* node = searchPrefix(prefix);
45        return node != nullptr;
46    }
47
48    int arnsDP(string s, int n){
49        vector<int> dp(n+1, 0);

```

```

50        // dp[i] -> ways to form the prefix s[0..i-1]
51        dp[0] = 1;
52
53        for(int i = 0; i < n; i++){
54            Trie *it = this; // Iterate the Trie from the root
55            for(int j = i; j < n; j++){
56                int pos = s[j] - 'a';
57                if(!it->children[pos]) break;
58                it = it->children[pos];
59
60                if(it->term_st)
61                    dp[j+1] = (dp[j+1] + dp[i]) % MOD;
62            }
63        }
64        return dp[n];
65    }
66 }
67 };

```

9.3 KMP

```

1 // Knuth-Morris-Pratt Algorithm
2 vector<int> computeLPS(string pattern) {
3     int m = pattern.length();
4     vector<int> lps(m);
5     int len = 0;
6     lps[0] = 0;
7
8     for (int i = 1; i < m; ) {
9         if (pattern[i] == pattern[len]) {
10             len++;
11             lps[i] = len;
12             i++;
13         } else {
14             if (len != 0) {
15                 len = lps[len - 1];
16             } else {
17                 lps[i] = 0;
18                 i++;
19             }
20         }
21     }
22     return lps;
23 }
24
25 vector<int> KMP(string text, string pattern) {
26     vector<int> lps = computeLPS(pattern);
27     vector<int> matches;
28     int n = text.length();
29     int m = pattern.length();

```

```

30     int i = 0, j = 0;
31
32     while (i < n) {
33         if (text[i] == pattern[j]) {
34             i++;
35             j++;
36         }
37
38         if (j == m) {
39             matches.push_back(i - j);
40             j = lps[j - 1];
41         } else if (i < n && text[i] != pattern[j]) {
42             if (j != 0) {
43                 j = lps[j - 1];
44             } else {
45                 i++;
46             }
47         }
48     }
49     return matches;
50 }

```

```

25     while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i +
26         k]) k++;
27     d[i] = k;
28     if (i + k - 1 > r) {
29         l = i - k;
30         r = i + k - 1;
31     }
32     return d;
33 }
34
35 // Returns array of max palindrome radius at each position.
36 // Result[i] = max palindrome half-length at position i (odd-length
37 // uses d1, even-length uses d2).
38 vector<int> manacher(string s) {
39     string t;
40     for (char c : s) {
41         t += string("#") + c;
42     }
43     t += "#";
44     vector<int> d1 = manacher_odd(t);
45     vector<int> d2 = manacher_even(t);
46     int n = (int)s.size();
47     vector<int> res(n);
48     for (int i = 0; i < n; i++) {
49         int pos = 2 * i + 1; // position in transformed string
50         int odd = d1[pos] - 1; // radius - 1 = original odd
51         // palindrome half-length contribution
52         int even = 0;
53         if (pos - 1 >= 0) even = d2[pos - 1]; // even palindrome
54         // radius at same center
55         res[i] = max(odd, even);
56     }
57     return res;
58 }

```

9.4 Manacher Algorithm

```

1 // Odd-length manacher: d1[i] = radius of palindrome centered at i (
2 // including center)
3 vector<int> manacher_odd(const string& s) {
4     int n = (int)s.size();
5     vector<int> d(n);
6     int l = 0, r = -1;
7     for (int i = 0; i < n; i++) {
8         int k = (i > r) ? 1 : min(d[l + r - i], r - i + 1);
9         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
10        d[i] = k;
11        if (i + k - 1 > r) {
12            l = i - k + 1;
13            r = i + k - 1;
14        }
15    }
16    return d;
17 }
18
19 // Even-length manacher: d2[i] = radius of palindrome centered
20 // between i-1 and i
21 vector<int> manacher_even(const string& s) {
22     int n = (int)s.size();
23     vector<int> d(n);
24     int l = 0, r = -1;
25     for (int i = 0; i < n; i++) {
26         int k = (i > r) ? 0 : min(d[l + r - i + 1], r - i + 1);

```

9.5 Palindromic Tree

```

1 // Palindromic Tree (Eertree)
2 struct PalindromicTree {
3     struct Node {
4         int len;
5         int link;
6         int next[26];
7         int occ; // occurrences as palindrome suffix while adding
8         // characters
9     };
10    Node(int l = 0) : len(l), link(0), occ(0) {
11        fill(next, next + 26, -1);
12    }

```

```

12     };
13
14     vector<Node> tree;
15     string s;
16     int suff; // node of the longest suffix-palindrome of current
17               // string
18
19     PalindromicTree() { init(); }
20
21     void init() {
22         tree.clear();
23         tree.push_back(Node(-1)); // root 0: odd length root
24         tree.push_back(Node(0));  // root 1: even length root
25         tree[0].link = 0;
26         tree[1].link = 0;
27         s.clear();
28         suff = 1;
29     }
30
31     int getLink(int v, int pos) const {
32         while (true) {
33             int curLen = tree[v].len;
34             if (pos - 1 - curLen >= 0 && s[pos - 1 - curLen] == s[
35                 pos]) {
36                 return v;
37             }
38             v = tree[v].link;
39         }
40     }
41
42     void addChar(char ch) {
43         int c = ch - 'a';
44         s.push_back(ch);
45         int pos = (int)s.size() - 1;
46
47         int cur = getLink(suff, pos);
48
49         if (tree[cur].next[c] != -1) {
50             suff = tree[cur].next[c];
51             tree[suff].occ++;
52             return;
53         }
54
55         int newNode = (int)tree.size();
56         tree.push_back(Node(tree[cur].len + 2));
57         tree[cur].next[c] = newNode;
58
59         if (tree[newNode].len == 1) {
60             tree[newNode].link = 1;
61         } else {
62             int linkCand = getLink(tree[cur].link, pos);
63             tree[newNode].link = tree[linkCand].next[c];
64         }
65     }

```

```

63
64         suff = newNode;
65         tree[suff].occ++;
66     }
67
68     void build(const string& str) {
69         init();
70         for (char ch : str) addChar(ch);
71     }
72
73     // Number of distinct palindromic substrings.
74     int distinctPalindromes() const {
75         return max(0, (int)tree.size() - 2);
76     }
77 };
78
79 // Example of use:
80 int countDistinctPalindromes(const string& s) {
81     PalindromicTree pt;
82     pt.build(s);
83     return pt.distinctPalindromes();
84 }

```

9.6 Polynomial Hash

```

1 #define ll long long
2 #define all(x) x.begin(), x.end()
3
4 mt19937 rng((uint32_t) chrono::steady_clock::now().time_since_epoch
5   ()).count());
6 const int MOD = 1e9 + 9;
7 const ll BASE = uniform_int_distribution<ll>(0, MOD-1)(rng);
8
9 vector<ll> pows = {1};
10 vector<ll> p_hash;
11
12 void hash_str(const string &s){
13     p_hash.assign(s.size()+1, 0);
14
15     while(pows.size() <= s.size()){
16         pows.push_back((pows.back()* BASE)%MOD);
17     }
18
19     p_hash[0] = 0;
20     for(int i=0; i<s.size(); i++){
21         p_hash[i+1] = ((p_hash[i]* BASE)% MOD + s[i])%MOD;
22     }
23 }
24
25 ll get_hash(int l, int r){

```

```

25     ll raw = (p_hash[r+1] - (p_hash[l] * pows[r-l+1]));
26     return (raw % MOD + MOD) % MOD;
27 }
28
29 inline void solve(){
30     string s; cin>>s;
31     int n = s.size();
32
33     hash_str(s);
34
35     for(int i=0; i<n-1; i++){
36         if(get_hash(0,i) == get_hash(n-i-1, n-1)){
37             cout<<i+1<<" ";
38         }
39     }
40     cout<<endl;
41 }

```

9.7 Rabin-Karp

```

1  // Rabin-Karp (single pattern) with double mod hash.
2  // Finds all occurrences of pattern in text in O(n + m).
3
4  using ll = long long;
5
6  const ll MOD1 = 1000000007LL;
7  const ll MOD2 = 1000000009LL;
8  const ll BASE = 911382323LL;
9
10 vector<int> rabinKarp(const string &text, const string &pattern) {
11     int n = (int)text.size();
12     int m = (int)pattern.size();
13     vector<int> arns;
14
15     if (m == 0 || m > n) return arns;
16
17     vector<ll> p1(n + 1, 1), p2(n + 1, 1);
18     vector<ll> h1(n + 1, 0), h2(n + 1, 0);
19
20     for (int i = 1; i <= n; i++) {
21         p1[i] = (p1[i - 1] * BASE) % MOD1;
22         p2[i] = (p2[i - 1] * BASE) % MOD2;
23
24         h1[i] = (h1[i - 1] * BASE + (unsigned char)text[i - 1] + 1)
25             % MOD1;
26         h2[i] = (h2[i - 1] * BASE + (unsigned char)text[i - 1] + 1)
27             % MOD2;
28     }
29
30     ll ph1 = 0, ph2 = 0;

```

```

29     for (char c : pattern) {
30         ph1 = (ph1 * BASE + (unsigned char)c + 1) % MOD1;
31         ph2 = (ph2 * BASE + (unsigned char)c + 1) % MOD2;
32     }
33
34     auto getHash = [&](int l, int r) {
35         ll x1 = (h1[r + 1] - (h1[l] * p1[r - l + 1]) % MOD1 + MOD1)
36             % MOD1;
37         ll x2 = (h2[r + 1] - (h2[l] * p2[r - l + 1]) % MOD2 + MOD2)
38             % MOD2;
39         return pair<ll, ll>{x1, x2};
40     };
41
42     for (int i = 0; i + m - 1 < n; i++) {
43         if (getHash(i, i + m - 1) == pair<ll, ll>{ph1, ph2}) {
44             arns.push_back(i);
45         }
46     }
47
48     return arns;
49 }
50
51 inline void solve() {
52     string text, pattern;
53     cin >> text >> pattern;
54
55     vector<int> pos = rabinKarp(text, pattern);
56     cout << pos.size() << endl;
57     for (int i : pos) cout << i << " ";
58     cout << endl;
59 }

```

9.8 Suffix Array + LCP

```

1  // Suffix Array + LCP (Kasai)
2  struct SuffixArray {
3      string s;
4      vector<int> sa;    // Suffix array: indices of suffixes in
5                          // sorted order
6      vector<int> rank;  // rank[i] = position of suffix i in sa
7      vector<int> lcp;   // lcp[i] = LCP(sa[i], sa[i-1]), lcp[0] = 0
8
9      SuffixArray() {}
10     explicit SuffixArray(const string& str) { build(str); }
11
12     void build(const string& str) {
13         s = str;
14         int n = (int)s.size();
15         sa.resize(n);
16         rank.resize(n);

```

```

16     for (int i = 0; i < n; i++) {
17         sa[i] = i;
18         rank[i] = s[i];
19     }
20
21     vector<int> tmp(n);
22     for (int k = 1;; k <= 1) {
23         auto cmp = [&](int i, int j) {
24             if (rank[i] != rank[j]) return rank[i] < rank[j];
25             int ri = (i + k < n) ? rank[i + k] : -1;
26             int rj = (j + k < n) ? rank[j + k] : -1;
27             return ri < rj;
28         };
29
30         sort(sa.begin(), sa.end(), cmp);
31         tmp[sa[0]] = 0;
32
33         for (int i = 1; i < n; i++) {
34             tmp[sa[i]] = tmp[sa[i - 1]] + (cmp(sa[i - 1], sa[i])
35                 ? 1 : 0);
36         }
37
38         rank = tmp;
39         if (rank[sa[n - 1]] == n - 1) break;
40     }
41
42     buildLCP();
43 }
44
45 void buildLCP() {
46     int n = (int)s.size();
47     lcp.assign(n, 0);
48     for (int i = 0; i < n; i++) rank[sa[i]] = i;
49
50     int k = 0;
51     for (int i = 0; i < n; i++) {
52         if (rank[i] == 0) {
53             k = 0;
54             continue;
55         }
56         int j = sa[rank[i] - 1];
57         while (i + k < n && j + k < n && s[i + k] == s[j + k]) k
58             ++;
59         lcp[rank[i]] = k;
60         if (k) k--;
61     }
62 };
63
64 // Example of use:
65 // Find the longest repeated substring in s.
66 string longestRepeatedSubstring(const string& s) {

```

```

67     if (s.empty()) return "";
68     SuffixArray sa(s);
69
70     int bestLen = 0;
71     int bestPos = 0;
72     for (int i = 1; i < (int)sa.lcp.size(); i++) {
73         if (sa.lcp[i] > bestLen) {
74             bestLen = sa.lcp[i];
75             bestPos = sa.sa[i];
76         }
77     }
78     return s.substr(bestPos, bestLen);
79 }

```

9.9 Suffix Automaton

```

1 // Suffix Automaton
2 struct SuffixAutomaton {
3     struct State {
4         int link;
5         int len;
6         int next[26];
7
8         State() : link(-1), len(0) {
9             fill(next, next + 26, -1);
10         }
11     };
12
13     vector<State> st;
14     int last;
15
16     SuffixAutomaton() {
17         st.reserve(1);
18         st.push_back(State());
19         last = 0;
20     }
21
22     void extend(char ch) {
23         int c = ch - 'a';
24
25         int cur = (int)st.size();
26         st.push_back(State());
27         st[cur].len = st[last].len + 1;
28
29         int p = last;
30         while (p != -1 && st[p].next[c] == -1) {
31             st[p].next[c] = cur;
32             p = st[p].link;
33         }
34     }

```

```

35     if (p == -1) {
36         st[cur].link = 0;
37     } else {
38         int q = st[p].next[c];
39         if (st[p].len + 1 == st[q].len) {
40             st[cur].link = q;
41         } else {
42             int clone = (int)st.size();
43             st.push_back(st[q]);
44             st[clone].len = st[p].len + 1;
45
46             while (p != -1 && st[p].next[c] == q) {
47                 st[p].next[c] = clone;
48                 p = st[p].link;
49             }
50
51             st[q].link = st[cur].link = clone;
52         }
53     }
54
55     last = cur;
56 }
57
58 void build(const string& s) {
59     st.clear();
60     st.push_back(State());
61     last = 0;
62     for (char ch : s) extend(ch);
63 }
64
65 // Number of distinct substrings of the built string.
66 long long countDistinctSubstrings() const {
67     long long ans = 0;
68     for (int v = 1; v < (int)st.size(); v++) {
69         ans += st[v].len - st[st[v].link].len;
70     }
71     return ans;
72 }
73
74 bool contains(const string& t) const {
75     int v = 0;
76     for (char ch : t) {
77         int c = ch - 'a';
78         if (st[v].next[c] == -1) return false;
79         v = st[v].next[c];
80     }
81     return true;
82 }
83 };
84
85 // Example of use:
86 long long distinctSubstringsIn(const string& s) {
87     SuffixAutomaton sam;

```

```

88     sam.build(s);
89     return sam.countDistinctSubstrings();
90 }

```

9.10 Suffix Automata 2

```

1 struct state {
2     int len, link;
3     map<char, int> next;
4 };
5
6 const int MAXLEN = 100000;
7 state st[MAXLEN * 2];
8 int sz, last;
9
10 void sa_init() {
11     st[0].len = 0;
12     st[0].link = -1;
13     sz++;
14     last = 0;
15 }
16
17 void sa_extend(char c) {
18     int cur = sz++;
19     st[cur].len = st[last].len + 1;
20     int p = last;
21     while (p != -1 && !st[p].next.count(c)) {
22         st[p].next[c] = cur;
23         p = st[p].link;
24     }
25     if (p == -1) {
26         st[cur].link = 0;
27     } else {
28         int q = st[p].next[c];
29         if (st[p].len + 1 == st[q].len) {
30             st[cur].link = q;
31         } else {
32             int clone = sz++;
33             st[clone].len = st[p].len + 1;
34             st[clone].next = st[q].next;
35             st[clone].link = st[q].link;
36             while (p != -1 && st[p].next[c] == q) {
37                 st[p].next[c] = clone;
38                 p = st[p].link;
39             }
40             st[q].link = st[cur].link = clone;
41         }
42     }
43     last = cur;
44 }

```

```

45
46 long long get_diff_strings(){
47     long long tot = 0;
48     for(int i = 1; i < sz; i++) {
49         tot += st[i].len - st[st[i].link].len;
50     }
51     return tot;
52 }
53
54 long long get_tot_len_diff_substings() {
55     long long tot = 0;
56     for(int i = 1; i < sz; i++) {
57         long long shortest = st[st[i].link].len + 1;
58         long long longest = st[i].len;
59
60         long long num_strings = longest - shortest + 1;
61         long long cur = num_strings * (longest + shortest) / 2;
62         tot += cur;
63     }
64     return tot;
65 }
66
67 string lcs (string S, string T) {
68     sa_init();
69     for (int i = 0; i < S.size(); i++)
70         sa_extend(S[i]);
71
72     int v = 0, l = 0, best = 0, bestpos = 0;
73     for (int i = 0; i < T.size(); i++) {
74         while (v && !st[v].next.count(T[i])) {
75             v = st[v].link ;
76             l = st[v].len;
77         }
78         if (st[v].next.count(T[i])) {
79             v = st [v].next[T[i]];
80             l++;
81         }
82         if (l > best) {
83             best = l;
84             bestpos = i;
85         }
86     }
87     return T.substr(bestpos - best + 1, best);
88 }

```

9.11 String Stream

```

1 // Create a stringstream object
2 stringstream ss(fullString);
3 // Extract the strings

```

```

4 ss >> a >> colon >> b >> comma >> c;

```

9.12 XOR Hash

```

1 // XOR Hash for strings (very fast, collision-prone compared to
2 // polynomial hash).
3 // Good for randomized checks, grouping, and quick fingerprints.
4 using ull = unsigned long long;
5
6 mt19937_64 rng_xor((uint64_t)chrono::steady_clock::now().
7     time_since_epoch().count());
8 array<ull, 256> H;
9
10 void initXorHash() {
11     for (int i = 0; i < 256; i++) H[i] = rng_xor();
12 }
13
14 // Order-sensitive XOR hash with position mixing.
15 ull xorHash(const string &s) {
16     ull arns = 0;
17     for (int i = 0; i < (int)s.size(); i++) {
18         ull x = H[(unsigned char)s[i]];
19         arns ^= (x + 0x9e3779b97f4a7c15ULL + (arns << 6) + (arns >>
20             2) + (ull)(i + 1));
21     }
22     return arns;
23 }
24
25 inline void solve() {
26     initXorHash();
27
28     int n; cin >> n;
29     unordered_map<ull, int> freq;
30
31     for (int i = 0; i < n; i++) {
32         string s; cin >> s;
33         freq[xorHash(s)]++;
34     }
35
36     // Number of distinct hashes (not guaranteed distinct strings
37     // due to collisions).
38     cout << (int)freq.size() << endl;
39 }

```

9.13 Z Algorithm

```

1 // Z-Algorithm
2 vector<int> z_function(string s) {
3     int n = s.length();
4     vector<int> z(n);
5     int l = 0, r = 0;
6
7     for (int i = 1; i < n; i++) {
8         if (i <= r) {
9             z[i] = min(r - i + 1, z[i - l]);
10        }
11        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
12            z[i]++;
13        }
14        if (i + z[i] - 1 > r) {
15            l = i;
16            r = i + z[i] - 1;
17        }
18    }
19    return z;
20 }
21
22 // Pattern matching using Z-algorithm
23 vector<int> patternMatch(string text, string pattern) {
24     string combined = pattern + "$" + text;
25     vector<int> z = z_function(combined);
26     vector<int> matches;
27     int m = pattern.length();
28
29     for (int i = m + 1; i < combined.length(); i++) {
30         if (z[i] == m) {
31             matches.push_back(i - m - 1);
32         }
33     }
34     return matches;
35 }

```

10 Computational Geometry

10.1 Convex Hull

```

1 const ll mod=1e9+7;
2 const ll limit=4e9;
3 //ascii https://elcodigoascii.com.ar/
4
5 int orientation(point a,point b,point c){
6     ll ori=(b.y-c.y)*(b.x-a.x)-(b.y-a.y)*(b.x-c.x);
7     if(ori==0) return 0;
8     if(ori>0) return 1;

```

```

9     return 2;
10 }
11
12 void getLastTwo(point &a,point &b,stack<point> &s)
13 {
14     a=s.top();
15     s.pop();
16     b=s.top();
17     s.pop();
18 }
19
20 void show(point a){
21     cout<<a.x<<" "<<a.y<<endl;
22 }
23
24 //Graham scan
25
26 void solve(){
27     int n; cin>>n;
28     vector<point> puntos(n);
29     FO(i,n){
30         ll a,b; cin>>a>>b;
31         puntos[i]=make_pair(a,b);
32     }
33     sort(all(puntos));
34     //Lower Part
35     stack<point> lower;
36     FO(i,n)
37     {
38         if(lower.size()<2){
39             lower.push(puntos[i]);
40             continue;
41         }
42         point a,b;
43         getLastTwo(a,b,lower);
44         if(orientation(a,b,puntos[i])<2)
45         {
46             lower.push(b);
47             lower.push(a);
48             lower.push(puntos[i]);
49         }
50         else{
51             lower.push(b);
52             i--;
53         }
54     }
55     stack<point> upper;
56     for(int i=n-1;i>=0;i--)
57     {
58         if(upper.size()<2){
59             upper.push(puntos[i]);
60             continue;
61         }

```

```

62     point a,b;
63     getLastTwo(a,b,upper);
64     if(orientation(a,b,puntos[i])<2)
65     {
66         upper.push(b);
67         upper.push(a);
68         upper.push(puntos[i]);
69     }
70     else{
71         upper.push(b);
72         i++;
73     }
74 }
75
76 set<point> res;
77
78 while(!lower.empty()){
79     res.insert(lower.top());
80     lower.pop();
81 }
82 while(!upper.empty()){
83     res.insert(upper.top());
84     upper.pop();
85 }
86 cout<<res.size()<<endl;
87 for(auto c:res) show(c);
88 }

```

10.2 Complex Point

```

1  typedef long long ll;
2  typedef vector<int> vec;
3  const ll mod=1e9+7;
4  const int MAX=2e5+3;
5
6  //ascii https://elcodigoascii.com.ar/
7
8  T norma(pt a){return a.x*a.x+a.y*a.y;}
9
10
11 int sgn(T X){
12     return (T(0)<X)-(T(0)>X);
13 }
14
15
16 pt translate(pt a,pt v){return a+v;}
17 pt scale(pt p,pt c,T factor){return c+(p-c)*factor;}
18 pt rot(pt p,T a){return p*polar(1.0,a);}
19 pt perp(pt p){return pt({-p.y,p.x});}
20 pt linearFunc(pt p,pt q,pt r,pt fp,pt fq){

```

```

21     return fp+(r-p)*(fq-fp)/(q-p);
22 }
23 T dot(pt v,pt w){ return v.x*w.x+v.y*w.y;}
24 T cross(pt v,pt w){ return v.x*w.y-v.y*w.x;}
25
26 bool isperp(pt a,pt b){return dot(a,b)==0;}
27
28 double angle(pt v,pt w){
29     return acos(clamp(dot(v,w)/abs(v)/abs(w),-1.0,-1.0));
30 }
31
32 T orientation(pt a,pt b,pt c){return cross(b-a,c-a);}
33
34 bool inAngle(pt a,pt b,pt c,pt p){
35     if(orientation(a,b,c)<0) swap(b,c);
36     return sgn(orientation(a,b,p))*sgn(orientation(a,c,p))<=0;
37 }
38
39 bool isconvex(vector<pt> p){
40     bool hasPos=false,hasNeg=false;
41     for(int i=0,n=p.size();i<n;i++){
42         int o=orientation(p[i],p[(i+1)%n],p[(i+2)%n]);
43         if(o>0) hasPos=true;
44         if(o<0) hasNeg=true;
45     }
46     return !(hasPos && hasNeg);
47 }
48
49 inline void solve()
50 {
51     pt p{3,-4};
52     p+=pt({1,2});
53     cout<<p<<endl;
54     cout<<norma(p)<<endl;
55 }

```

10.3 Line Segment Intersection

```

1  \begin{lstlisting}[style=compactcpp]
2  // BeginCodeSnip{Point Class}
3  struct Point {
4      int x, y;
5      Point(int a = 0, int b = 0) : x(a), y(b) {}
6
7      friend istream &operator>>(istream &in, Point &p) {
8          int x, y;
9          in >> p.x >> p.y;
10         return in;
11     }
12 };

```

```

13 // EndCodeSnip
14
15 int sign(long long num) {
16     if (num < 0) {
17         return -1;
18     } else if (num == 0) {
19         return 0;
20     } else {
21         return 1;
22     }
23 }
24
25 long long trigonometric_sense(Point p, Point p1, Point p2) {
26     return sign(1LL * (p1.x - p.x) * (p2.y - p.y) -
27               1LL * (p2.x - p.x) * (p1.y - p.y));
28 }
29
30 // Check if the rectangles with [P1, P2] and [P3, P4] as diagonals
   intersect
31 bool quick_check(Point p1, Point p2, Point p3, Point p4) {
32     int x1, x2, x3, x4, y1, y2, y3, y4;
33     x1 = min(p1.x, p2.x), x2 = max(p1.x, p2.x);
34     y1 = min(p1.y, p2.y), y2 = max(p1.y, p2.y);
35     x3 = min(p3.x, p4.x), x4 = max(p3.x, p4.x);
36     y3 = min(p3.y, p4.y), y4 = max(p3.y, p4.y);
37     return x2 < x3 || x4 < x1 || y2 < y3 || y4 < y1;
38 }
39
40 bool check(Point p1, Point p2, Point p3, Point p4) {
41     if (trigonometric_sense(p1, p2, p3) * trigonometric_sense(p1, p2,
42       p4) > 0) {
43         return false;
44     }
45     if (trigonometric_sense(p3, p4, p1) * trigonometric_sense(p3, p4,
46       p2) > 0) {
47         return false;
48     }
49     // Collinear case: check if projections on x-axis overlap properly
   // (not just touch at a single endpoint)
   // First check if all 4 points are collinear by verifying p3 and
   p4 lie on line p1-p2
50     if (trigonometric_sense(p1, p2, p3) == 0 && trigonometric_sense(p1
51       , p2, p4) == 0) {
52         if (max(p1.x, p2.x) < min(p3.x, p4.x) ||
53             max(p3.x, p4.x) < min(p1.x, p2.x))
54             return false;
55     }
56     return true;
57 }
58
59 int main() {
60     int test_num;
61     cin >> test_num;

```

```

61     for (int t = 0; t < test_num; t++) {
62         Point p1, p2, p3, p4;
63         cin >> p1 >> p2 >> p3 >> p4;
64
65         if (quick_check(p1, p2, p3, p4)) {
66             cout<<"NO"<<endl;
67         } else if (check(p1, p2, p3, p4)) {
68             cout<<"YES"<<endl;
69         } else {
70             cout<<"NO"<<endl;
71         }
72     }
73 }

```

10.4 Minimum Euclidean Distance

```

1 const ll mod=1e9+7;
2 const ll MAX=8e18;
3 const ll limit=1e9+1;
4 //ascii https://elcodigoascii.com.ar/
5
6 ll distance(point a,point b){
7     return (a.X-b.X)*(a.X-b.X)+(a.Y-b.Y)*(a.Y-b.Y);
8 }
9
10 inline void solve()
11 {
12     int n; cin>>n;
13     vector<point> sortedX(n);
14     set<point> sortedY;
15     FO(i,n)
16     {
17         ll x,y; cin>>x>>y;
18         sortedX[i]=make_pair(x,y);
19     }
20     sort(all(sortedX));
21     sortedY.insert(make_pair(sortedX[0].Y,sortedX[0].X));
22     ll d,minSquare=MAX;
23     int j=0;
24     FOR(i,1,n)
25     {
26         d=ceil(sqrt(minSquare));
27         while(sortedX[i].X-sortedX[j].X>d)
28         {
29             sortedY.erase(make_pair(sortedX[j].Y,sortedX[j].X));
30             j++;
31         }
32         auto lower=sortedY.lower_bound(make_pair(sortedX[i].Y-d,0));
33         auto upper=sortedY.upper_bound(make_pair(sortedX[i].Y+d,0));
34         for(auto pointer=lower;pointer!=upper;pointer++)

```

```

35     {
36         minSquare=min(minSquare,distance(*pointer,make_pair(
37             sortedX[i].Y,sortedX[i].X)));
38     }
39     sortedY.insert(make_pair(sortedX[i].Y,sortedX[i].X));
40 }
41 cout<<minSquare<<endl;
42 }

```

10.5 Minimum Manhattan Distance

```

1 pt rot(pt a)
2 {
3     return {a.x+a.y,a.y-a.x};
4 }
5
6 ll max_coord(ll a,set<ll>& v)
7 {
8     ll min_val=*v.begin();
9     ll max_val=*std::prev(v.end());
10    return max(abs(a-min_val),abs(a-max_val));
11 }
12
13
14
15 inline void solve()
16 {
17     int n; cin>>n;
18     vector<pt> points(n);
19     set<ll> x_;
20     set<ll> y_;
21     ll res=0;
22     cout<<res<<endl;
23     ll a,b; cin>>a>>b;
24     pt aux=rot({a,b});
25     x_.insert(aux.x);
26     y_.insert(aux.y);
27     FO(i,n-1){
28         cin>>a>>b;
29         aux=rot({a,b});
30         res=max({res,max_coord(aux.x,x_),max_coord(aux.y,y_)});
31         cout<<res<<endl;
32         x_.insert(aux.x);
33         y_.insert(aux.y);
34     }
35 }
36 }

```

10.6 Point In Polygon

```

1 \begin{lstlisting}[style=compactcpp]
2 struct point{
3     ll x,y;
4     void show(){
5         cout<<x<<" "<<y<<endl;
6     }
7 };
8
9 int sign(ll a){
10     if(a<0) return -1;
11     if(a==0) return 0;
12     if(a>0) return 1;
13 }
14
15 int signCP(point p,point p1,point p2)
16 {
17     return sign(1LL*((p1.x-p.x)*(p2.y-p.y)-(p1.y-p.y)*(p2.x-p.x)));
18 }
19
20 bool intersect(point n, point m,point a,point b)
21 {
22     if(signCP(n,a,b)*signCP(m,a,b)>0) return false;
23     if(signCP(a,n,m)*signCP(b,n,m)>0) return false;
24     return true;
25 }
26
27 bool inside(point a,point b,point c){
28     return a.x>=min(b.x,c.x) && a.x<=max(b.x,c.x) && a.y>=min(b.y,c.
29         y)
30         && a.y<=max(b.y,c.y);
31 }
32
33 inline void solve()
34 {
35     int n,m; cin>>n>>m;
36     vector<point> vertices(n);
37     FO(i,n)
38     {
39         cin>>vertices[i].x>>vertices[i].y;
40     }
41     point query,par,init,first,second;
42     int counter;
43     int resta=0;
44     FO(i,m)
45     {
46         resta=0;
47         counter=0;
48         cin>>query.x>>query.y;
49         par.x=query.x;
50         par.y=-MAX-1;

```

```

50 init.x=vertices[0].x;
51 init.y=vertices[0].y;
52 first.x=init.x;
53 first.y=init.y;
54 bool ver=false;
55 for(int j=1;j<=n;j++)
56 {
57     second.x=vertices[j%n].x;
58     second.y=vertices[j%n].y;
59     point AB,u;
60     AB.x=second.x-first.x;
61     AB.y=second.y-first.y;
62     u.x=second.x-query.x;
63     u.y=second.y-query.y;
64     if((AB.x*u.y-AB.y*u.x)==0 && inside(query,first,second))
65     {
66         cout<<"BOUNDARY"<<endl;
67         ver=true;
68         break;
69     }
70     if(intersect(query,par,first,second) && first.x<=query.x
71         && query.x<second.x)
72     {
73         counter++;
74     }
75     if(intersect(query,par,first,second) && second.x<=query.
76         x && query.x<first.x){
77         counter++;
78     }
79     first.x=second.x;
80     first.y=second.y;
81 }
82 point AB,u;
83 AB.x=init.x-first.x;
84 AB.y=init.y-first.y;
85 u.x=init.x-query.x;
86 u.y=init.y-query.y;
87 if(!ver){
88     //if(intersect(query,par,first,init)) counter++;
89     if((counter)&1) cout<<"INSIDE";
90     else cout<<"OUTSIDE";
91     cout<<endl;
92 }
93 }

```

```

3 };
4
5 struct Vector{
6     double a=0,b=0;
7     void getVector(point p1,point p2){
8         a=p2.x-p1.x;
9         b=p2.y-p1.y;
10    }
11
12    double getModulo(){
13        return pow(a*a+b*b,0.5);
14    }
15
16    Vector getUnitarian(){
17        Vector x;
18        x.a=a/getModulo();
19        x.b=b/getModulo();
20        //cout<<x.a<<" "<<x.b<<endl;
21        return x;
22    }
23 };
24
25 double dotProduct(Vector x,Vector y)
26 {
27     return x.a*y.a+x.b*y.b;
28 }
29
30 double CrossProduct(Vector x,Vector y)
31 {
32     return x.a*y.b-x.b*y.a;
33 }
34
35 inline void solve()
36 {
37
38
39     point p1,p2,p3,p4;
40     cin>>p1.x>>p1.y>>p2.x>>p2.y>>p3.x>>p3.y;
41     Vector u,v,t;
42     u.getVector(p1,p3);
43     //cout<<u.a<<" "<<u.b<<endl;
44     v.getVector(p2,p3);
45     if(CrossProduct(u,v)>0) cout<<"LEFT"<<endl;
46     else if(CrossProduct(u,v)<0) cout<<"RIGHT"<<endl;
47     else cout<<"TOUCH"<<endl;
48
49 }

```

10.7 Point Location Test

```

1 struct point{
2     double x,y;

```

10.8 Polar Sort

```

1 #define x real()
2 #define y imag()
3
4 typedef long long ll;
5 typedef double T;
6 typedef complex<T> pt;
7 typedef vector<int> vec;
8 const ll mod=1e9+7;
9 const int MAX=2e5+3;
10
11 T cross(pt v,pt w){ return v.x*w.y-v.y*w.x;}
12 T norma(pt a){return a.x*a.x+a.y*a.y;}
13 //ascii https://elcodigoascii.com.ar/
14
15 bool half(pt p){
16     assert(p.x!=0 || p.y!=0);
17     return p.y>0 || (p.y==0 && p.x<0);
18 }
19
20 void polarSort(vector<pt> &v){
21     sort(all(v),[](pt v,pt w){
22         return make_tuple(half(v),0)<make_tuple(half(w),cross(v,w));
23     });
24 }
25
26 void polarSortNorm(vector<pt> &v){
27     sort(all(v),[](pt v,pt w){
28         return make_tuple(half(v),0,norma(v))<make_tuple(half(w),
29             cross(v,w),norma(w));
30     });
31 }

```

10.9 Polygon Area

```

1 struct point{
2     ll x,y;
3 };
4
5 ll CrossP(point a,point b){
6     return a.x*b.y-a.y*b.x;
7 }
8
9 inline void solve()
10 {
11     int n; cin>>n;
12     ll res=0;
13     point p1,p2,p3;
14     cin>>p3.x>>p3.y;
15     p1.x=p3.x;
16     p1.y=p3.y;

```

```

17     F0(i,n-1)
18     {
19         cin>>p2.x>>p2.y;
20         res+=CrossP(p1,p2);
21         p1.x=p2.x;
22         p1.y=p2.y;
23     }
24     res+=CrossP(p1,p3);
25     cout<<abs(res)<<endl;
26 }

```

10.10 Closest Pair of Points

```

1 #include <utility>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 typedef long long ll;
7 const ll MAX = 8e18;
8
9 inline ll dist(const pair<ll,ll>& a, const pair<ll,ll>& b){
10     ll dx = a.first - b.first;
11     ll dy = a.second - b.second;
12     return dx*dx + dy*dy;
13 }
14
15 pair<pair<ll,ll>, pair<ll,ll>> bruteForce(const vector<pair<ll,ll>>&
16     pts, int left, int right){
17     ll bestd = MAX;
18     pair<pair<ll,ll>, pair<ll,ll>> best = {pts[left], pts[left+1]};
19     for(int i = left; i < right; i++){
20         for(int j = i + 1; j < right; j++){
21             ll d = dist(pts[i], pts[j]);
22             if(d < bestd){
23                 bestd = d;
24                 best = {pts[i], pts[j]};
25             }
26         }
27     }
28     return best;
29 }
30
31 struct cmpY {
32     bool operator()(const pair<ll,ll>& a, const pair<ll,ll>& b)
33     const {
34         return a.second < b.second;
35     }
36 };

```

```

36 pair<pair<ll,ll>, pair<ll,ll>> closestPairRecursive(const vector<
    pair<ll,ll>>& pts, int left, int right){
37     if(right - left <= 3){
38         return bruteForce(pts, left, right);
39     }
40     int mid = left + (right - left) / 2;
41     ll midX = pts[mid].first;
42
43     auto leftPair = closestPairRecursive(pts, left, mid);
44     auto rightPair = closestPairRecursive(pts, mid, right);
45     ll d = min(dist(leftPair.first, leftPair.second), dist(rightPair
        .first, rightPair.second));
46     pair<pair<ll,ll>, pair<ll,ll>> best = leftPair;
47     if(dist(rightPair.first, rightPair.second) < dist(best.first,
        best.second)){
48         best = rightPair;
49     }
50
51     vector<pair<ll,ll>> strip;
52     strip.reserve(right - left);
53     for(int i = left; i < right; i++){
54         if(abs(pts[i].first - midX) < d){
55             strip.push_back(pts[i]);
56         }
57     }
58     sort(strip.begin(), strip.end(), cmpY());
59
60     for(size_t i = 0; i < strip.size(); i++){
61         for(size_t j = i + 1; j < strip.size() && (strip[j].second -
            strip[i].second) < d; j++){
62             ll d2 = dist(strip[i], strip[j]);
63             if(d2 < d){
64                 d = d2;
65                 best = {strip[i], strip[j]};
66             }
67         }
68     }
69     return best;
70 }
71
72 pair<pair<ll,ll>, pair<ll,ll>> closest_pair(vector<pair<ll,ll>>& pts
    ){
73     sort(pts.begin(), pts.end());
74     return closestPairRecursive(pts, 0, (int)pts.size());
75 }

```

```

3
4 typedef long long ll;
5 const long double EPS = 1e-12;
6
7 struct Point {
8     ll x, y;
9     Point() {}
10    Point(ll _x, ll _y) : x(_x), y(_y) {}
11    bool operator==(const Point& o) const { return x == o.x && y ==
        o.y; }
12    bool operator!=(const Point& o) const { return !(*this == o); }
13 };
14
15 Point operator+(const Point& a, const Point& b) { return Point(a.x +
    b.x, a.y + b.y); }
16 Point operator-(const Point& a, const Point& b) { return Point(a.x -
    b.x, a.y - b.y); }
17 Point operator*(const Point& a, ll k) { return Point(a.x * k, a.y *
    k); }
18
19 ll cross(const Point& a, const Point& b) { return a.x * b.y - a.y *
    b.x; }
20 ll dot(const Point& a, const Point& b) { return a.x * b.x + a.y * b.
    y; }
21
22 // Half-plane: the left side of directed line (p, p+v) is the valid
    region
23 struct HalfPlane {
24     Point p; // point on the line
25     Point v; // direction vector
26     long double a; // angle for sorting
27
28     HalfPlane() {}
29     HalfPlane(const Point& _p, const Point& _v) : p(_p), v(_v) {
30         a = atan2((long double)v.y, (long double)v.x);
31     }
32
33     // Positive if q is in the half-plane (left of line)
34     ll eval(const Point& q) const {
35         return cross(v, q - p);
36     }
37
38     bool contains(const Point& q) const {
39         return eval(q) >= 0;
40     }
41 };
42
43 // Angle comparison for sorting
44 bool hpAngleCmp(const HalfPlane& hp1, const HalfPlane& hp2) {
45     return hp1.a < hp2.a;
46 }
47
48 // Check if two half-planes are parallel

```

10.11 Half-Plane Intersection

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

```

49 bool parallel(const HalfPlane& hp1, const HalfPlane& hp2) {
50     return cross(hp1.v, hp2.v) == 0;
51 }
52
53 // Line intersection:  $p1 + t \cdot v1 = p2 + s \cdot v2$ 
54 Point lineIntersect(const Point& p1, const Point& v1, const Point&
55     p2, const Point& v2) {
56     Point u = p2 - p1;
57     ll d = cross(v1, v2);
58     if (d == 0) return Point(0, 0); // parallel - caller should
59         handle
60     long double t = (long double)cross(u, v2) / (long double)d;
61     return Point(p1.x + (ll)(v1.x * t), p1.y + (ll)(v1.y * t));
62 }
63
64 // Intersection of two half-plane boundary lines
65 Point hpIntersect(const HalfPlane& hp1, const HalfPlane& hp2) {
66     return lineIntersect(hp1.p, hp1.v, hp2.p, hp2.v);
67 }
68
69 // S&I Algorithm:  $O(N \log N)$  half-plane intersection
70 vector<Point> half_plane_intersection(vector<HalfPlane>& hps) {
71     if (hps.empty()) return {};
72
73     // Sort by angle
74     sort(hps.begin(), hps.end(), hpAngleCmp);
75
76     // Remove half-planes with duplicate angles (keep the more
77         restrictive one)
78     vector<HalfPlane> filtered;
79     for (size_t i = 0; i < hps.size(); i++) {
80         if (i + 1 < hps.size() && fabs(hps[i].a - hps[i + 1].a) <
81             EPS) {
82             // Same angle: keep the one that contains the other's
83                 point
84             if (hps[i].contains(hps[i + 1].p)) {
85                 continue; // hps[i+1] is redundant
86             }
87         }
88         filtered.push_back(hps[i]);
89     }
90     hps.swap(filtered);
91
92     if (hps.empty()) return {};
93
94     // Deque for the algorithm
95     deque<HalfPlane> dq;
96
97     for (const auto& hp : hps) {
98         // Skip if parallel and on same side as last
99         if (!dq.empty() && parallel(dq.back(), hp)) {
100             if (dq.back().contains(hp.p)) {
101                 continue; // hp is redundant

```

```

102         } else {
103             dq.pop_back(); // hp is more restrictive
104         }
105     }
106
107     // Remove from back while intersection with new hp is
108         outside
109     while (dq.size() >= 2) {
110         Point inter = hpIntersect(dq[dq.size() - 2], dq.back());
111         if (hp.contains(inter)) break;
112         dq.pop_back();
113     }
114
115     // Remove from front while intersection with new hp is
116         outside
117     while (dq.size() >= 2) {
118         Point inter = hpIntersect(dq[0], dq[1]);
119         if (hp.contains(inter)) break;
120         dq.pop_front();
121     }
122
123     dq.push_back(hp);
124 }
125
126 // Final cleanup: remove last half-plane if it becomes redundant
127 while (dq.size() > 2) {
128     Point inter = hpIntersect(dq[0], dq[dq.size() - 2]);
129     if (dq.back().contains(inter)) {
130         dq.pop_back();
131     } else {
132         break;
133     }
134 }
135
136 // Remove first if redundant
137 while (dq.size() > 2) {
138     Point inter = hpIntersect(dq[dq.size() - 1], dq[1]);
139     if (dq[0].contains(inter)) {
140         dq.pop_front();
141     } else {
142         break;
143     }
144 }
145
146 if (dq.size() < 3) return {}; // Empty or unbounded
147
148 // Extract polygon vertices
149 vector<Point> result;
150 for (size_t i = 0; i < dq.size(); i++) {
151     size_t j = (i + 1) % dq.size();
152     result.push_back(hpIntersect(dq[i], dq[j]));
153 }

```

```

148     return result;
149 }
150
151 // Legacy compatibility
152 struct point {
153     ll x, y;
154     point() {}
155     point(ll _x, ll _y) : x(_x), y(_y) {}
156     explicit operator Point() const { return Point(x, y); }
157 };

```

10.12 Smallest Enclosing Circle

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Point {
5     double x, y;
6     Point() {}
7     Point(double _x, double _y) : x(_x), y(_y) {}
8 };
9
10 double dist(Point a, Point b) {
11     double dx = a.x - b.x, dy = a.y - b.y;
12     return sqrt(dx * dx + dy * dy);
13 }
14
15 pair<Point, double> trivial(const vector<Point>& R) {
16     if (R.empty()) return {Point(0, 0), 0};
17     if (R.size() == 1) return {R[0], 0};
18     if (R.size() == 2) {
19         Point c{(R[0].x + R[1].x) / 2, (R[0].y + R[1].y) / 2};
20         return {c, dist(R[0], R[1]) / 2};
21     }
22     // 3 points - circumcircle
23     double d = 2 * (R[0].x * (R[1].y - R[2].y) +
24                     R[1].x * (R[2].y - R[0].y) +
25                     R[2].x * (R[0].y - R[1].y));
26     if (fabs(d) < 1e-12) return {Point(0, 0), 0}; // degenerate,
27     // shouldn't happen
28     double ux = ((R[0].x * R[0].x + R[0].y * R[0].y) * (R[1].y - R
29 [2].y) +
30                 (R[1].x * R[1].x + R[1].y * R[1].y) * (R[2].y - R
31 [0].y) +
32                 (R[2].x * R[2].x + R[2].y * R[2].y) * (R[0].y - R
33 [1].y)) / d;
34     double uy = ((R[0].x * R[0].x + R[0].y * R[0].y) * (R[2].x - R
35 [1].x) +
36                 (R[1].x * R[1].x + R[1].y * R[1].y) * (R[0].x - R
37 [2].x) +
38                 (R[2].x * R[2].x + R[2].y * R[2].y) * (R[1].x - R
39 [0].x)) / d;
40     return {Point(ux, uy), dist(R[0], Point(ux, uy))};
41 }

```

```

32         (R[2].x * R[2].x + R[2].y * R[2].y) * (R[1].x - R
33 [0].x)) / d;
34     return {Point(ux, uy), dist(R[0], Point(ux, uy))};
35 }
36
37 pair<Point, double> welzl(const vector<Point>& pts_input) {
38     vector<Point> pts = pts_input;
39     shuffle(pts.begin(), pts.end(), mt19937{});
40     vector<Point> R;
41
42     function<pair<Point, double>(int)> rec = [&](int n) -> pair<
43 Point, double> {
44     if (n == 0 || R.size() == 3) {
45         return trivial(R);
46     }
47     auto d = rec(n - 1);
48     if (dist(pts[n - 1], d.first) <= d.second + 1e-9) {
49         return d;
50     }
51     R.push_back(pts[n - 1]);
52     auto ans = rec(n - 1);
53     R.pop_back();
54     return ans;
55 };
56
57 return rec(pts.size());
58 }

```

10.13 Rotating Calipers

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5
6 struct point{
7     ll x,y;
8 };
9
10 ll dist2(point a, point b){
11     ll dx = a.x - b.x;
12     ll dy = a.y - b.y;
13     return dx*dx + dy*dy;
14 }
15
16 ll cross(point a, point b){
17     return a.x*b.y - a.y*b.x;
18 }
19
20 point operator-(point a, point b){

```

```

21     return {a.x - b.x, a.y - b.y};
22 }
23
24 pair<point, point> hull_diameter(const vector<point>& hull){
25     int n = hull.size();
26     if (n == 1) return {hull[0], hull[0]};
27
28     ll bestd = 0;
29     pair<point, point> best = {hull[0], hull[0]};
30
31     int j = 1;
32     for (int i = 0; i < n; i++){
33         // edge i -> (i+1)%n
34         point edge_i = hull[(i+1)%n] - hull[i];
35         // advance j to antipodal point (maximizes area with edge i)
36         while (cross(edge_i, hull[(j+1)%n] - hull[i]) >
37               cross(edge_i, hull[j] - hull[i]))
38             j = (j + 1) % n;
39
40         ll cur = dist2(hull[i], hull[j]);
41         if (cur > bestd){
42             bestd = cur;
43             best = {hull[i], hull[j]};
44         }
45     }
46     return best;
47 }

```

10.14 Pick's Theorem

```

1  #include <vector>
2  #include <utility>
3  #include <cmath>
4  using namespace std;
5  #define ll long long
6
7  struct Point {
8      ll x, y;
9  };
10
11 ll CrossP(Point a, Point b) {
12     return a.x * b.y - a.y * b.x;
13 }
14
15 ll gcd(ll a, ll b) {
16     if (a < 0) a = -a;
17     if (b < 0) b = -b;
18     while (b) {
19         ll t = a % b;
20         a = b;

```

```

21         b = t;
22     }
23     return a;
24 }
25
26 // Pick's Theorem: A = I + B/2 - 1
27 // Returns (interior, boundary)
28 pair<long long, long long> lattice_points(const vector<Point>& poly)
29 {
30     int n = poly.size();
31     if (n < 3) return {0, 0};
32
33     // Compute signed area via shoelace
34     ll area2 = 0;
35     for (int i = 0; i < n; i++) {
36         Point p1 = poly[i];
37         Point p2 = poly[(i + 1) % n];
38         area2 += CrossP(p1, p2);
39     }
40     ll area = llabs(area2) / 2;
41
42     // Compute boundary points B = sum of gcd(|dx|, |dy|) for each edge
43     ll B = 0;
44     for (int i = 0; i < n; i++) {
45         Point p1 = poly[i];
46         Point p2 = poly[(i + 1) % n];
47         ll dx = p2.x - p1.x;
48         ll dy = p2.y - p1.y;
49         B += gcd(dx, dy);
50     }
51
52     // I = area - B/2 + 1
53     ll I = area - B / 2 + 1;
54
55     return {I, B};
56 }

```

11 Dynamic Programming

11.1 Coin Problem

```

1 void solve(){
2     int n, x; cin>>n>>x;
3     vector<ll> coins(n);
4     for(int i=0; i<n; i++){
5         cin>>coins[i];
6     }
7

```

```

8   vector<ll> dp(x+1,LLONG_MAX);
9   dp[0] = 0;
10  for(int i=0; i<=x; i++){
11      for(int j=0; j<n; j++){
12          if(i-coins[j]>=0){
13              dp[i] = min(dp[i], dp[i-coins[j]]+1);
14          }
15      }
16  }
17
18  if(dp[x] != LLONG_MAX){
19      cout<<dp[x]<<endl;
20  }else{
21      cout<<"-1"<<endl;
22  }
23
24
25
26 }

```

```

2   int n, x; cin>>n>>x;
3   vector<ll> coins(n);
4   for(int i=0; i<n; i++){
5       cin>>coins[i];
6   }
7
8   int dp[102][1000005];
9   dp[0][0] = 1;
10  for(int i=1; i<=n; i++){
11      for(int j=0; j<=x; j++){
12          dp[i][j] = dp[i-1][j];
13          int l = j-coins[i-1];
14          if(l>=0){
15              dp[i][j] += (dp[i][l])%MOD;
16              dp[i][j]%=MOD;
17          }
18      }
19  }
20
21
22  cout<<dp[n][x]%MOD<<endl;
23 }

```

11.2 Coin Problem (Min Coins)

```

1 void solve(){
2     int n, x; cin>>n>>x;
3     vector<ll> coins(n);
4     for(int i=0; i<n; i++){
5         cin>>coins[i];
6     }
7
8     vector<ll> dp(x+1,0);
9     dp[0] = 1;
10    for(int i=0; i<=x; i++){
11        for(int j=0; j<n; j++){
12            if(i-coins[j]>=0){
13                dp[i] = (dp[i] + dp[i-coins[j]]);
14                dp[i]%=MOD;
15            }
16        }
17    }
18
19    cout<<dp[x]<<endl;
20 }
21

```

11.3 Coin Problem (Sorted)

```

1 void solve(){

```

11.4 Knapsack

```

1 void solve(){
2     int n,x; cin>>n>>x;
3     vector<int> prices(n+1);
4     vector<int> pages(n+1);
5     for(int i=1; i<=n; i++){cin>>prices[i];}
6     for(int i=1; i<=n; i++){cin>>pages[i];}
7
8     vector<int> dp(x+1); // dp[prices[i]] -> Max pages I can buy
                          // with price i
9
10    dp[0] = 0;
11    for(int i=1; i<=n; i++){
12        for(int j = x; j>=prices[i]; j--){
13            dp[j] = max(dp[j],dp[j-prices[i]]+pages[i]);
14        }
15    }
16
17    //for(int i=1; i<=x; i++) cout<<dp[i]<<" ";
18    //cout<<endl;
19
20    cout<<dp[x]<<endl;
21 }

```

11.5 LCS

```

1 vector<int> lcs(vector<int> const& a, vector<int> const &b, int n,
2   int m){
3     vector<vector<int>> dp (n+1,vector<int> (m+1,0));
4
5     for(int i=1; i<=n; i++){
6       for(int j=1; j<=m; j++){
7         if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1]+1;
8         else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
9       }
10    }
11
12    int ind = dp[n][m];
13    // Note: if we have characters, we must set ind+1 size
14    // and set the last character as terminal '\0'
15    vector<int> arns(ind);
16    int i = n, j = m;
17
18    while(i>0 and j>0){
19      if(a[i-1] == b[j-1]){
20        arns[ind-1] = a[i-1];
21        i--;
22        j--;
23        ind--;
24      }else if(dp[i-1][j] > dp[i][j-1]) i--;
25      else j--;
26    }
27
28    return arns;

```

```

17   }
18   return ans;
19 }

```

11.7 Two Sets

```

1 void solve(){
2   int n; cin>>n;
3
4   int tw = n*(n+1)/2;
5   if(tw%2 == 1){
6     cout<<0<<endl;
7     return;
8   }
9   tw = tw/2;
10
11   vector<int> dp(tw+1,0);
12
13   dp[0] = 1;
14
15   for(int i=1; i<n; i++){
16     for(int j=tw; j>=i; j--){
17       dp[j] = (dp[j] + dp[j-i])%MOD;
18     }
19   }
20
21   cout<<dp[tw]<<endl;
22
23 }

```

11.6 LIS

```

1 int lis(vector<ll> const& a) {
2   int n = a.size();
3   const int INF = 1e9;
4   vector<int> d(n+1, INF);
5   d[0] = -INF;
6
7   for (int i = 0; i < n; i++) {
8     int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
9     if (d[l-1] < a[i] && a[i] < d[l])
10      d[l] = a[i];
11   }
12
13   int ans = 0;
14   for (int l = 0; l <= n; l++) {
15     if (d[l] < INF)
16      ans = l;

```

11.8 Bitmask DP

```

1 const int limit=20;
2 vector<pair<ll,ll>> dp((1<<limit));
3 //ascii https://elcodigoascii.com.ar/
4
5 inline void solve()
6 {
7   int n; cin>>n;
8   ll x; cin>>x;
9   vector<ll> weight(n);
10  dp[0]={1,0};
11  F0(i,n) cin>>weight[i];
12  for(ll i=1;i<(1<<n);i++)
13  {
14    dp[i]={n+1,0};
15    for(int j=0;j<n;j++)

```

```

16     {
17         if(i & (1 << j))
18         {
19             pair<ll, ll> aux = dp[i ^ (1 << j)];
20             if(aux.second + weight[j] <= x) {
21                 aux.second += weight[j];
22             }
23             else {
24                 aux.first++;
25                 aux.second = weight[j];
26             }
27             dp[i] = min(dp[i], aux);
28         }
29     }
30 }
31 cout << dp[(1 << n) - 1].first << endl;
32 }

```

```

31 {
32     if(ld || prev != i)
33     {
34         int new_ld, new_tight;
35         if(i == 0 && ld) new_ld = 1;
36         else new_ld = 0;
37         if(tight && k == i) new_tight = 1;
38         else new_tight = 0;
39         sum += mem(idx - 1, new_tight, i, new_ld, s);
40     }
41 }
42 dp[idx][prev][ld][tight] = sum;
43 return sum;
44 }

```

11.9 Digit DP

```

1  const ll mod = 1e9 + 7;
2  ll dp[20][10][2][2];
3  //ascii https://elcodigoascii.com.ar/
4
5  inline void init() {
6      for(int i = 0; i < 20; i++)
7          for(int j = 0; j < 10; j++)
8              for(int k = 0; k < 2; k++)
9                  for(int l = 0; l < 2; l++)
10                     dp[i][j][k][l] = -1;
11 }
12
13 ll mem(int idx, int tight, int prev, int ld, string s)
14 {
15     if(idx == 0)
16     {
17         return 1;
18     }
19     static bool initialized = false;
20     if(!initialized) {
21         memset(dp, -1, sizeof(dp));
22         initialized = true;
23     }
24     if(dp[idx][prev][ld][tight] != -1) {
25         return dp[idx][prev][ld][tight];
26     }
27     int k = 9;
28     if(tight) k = s[s.size() - idx] - '0';
29     ll sum = 0;
30     for(int i = 0; i <= k; i++)

```

11.10 Edit Distance

```

1  int editDistance(string &s1, string &s2) {
2
3      int m = s1.length();
4      int n = s2.length();
5
6      // Create a table to store results of subproblems
7      vector<vector<int>> dp(m + 1, vector<int>(n + 1));
8
9      // Fill the known entries in dp[][]
10     // If one string is empty, then answer
11     // is length of the other string
12     for (int i = 0; i <= m; i++)
13         dp[i][0] = i;
14     for (int j = 0; j <= n; j++)
15         dp[0][j] = j;
16
17     // Fill the rest of dp[][]
18     for (int i = 1; i <= m; i++) {
19         for (int j = 1; j <= n; j++) {
20             if (s1[i - 1] == s2[j - 1])
21                 dp[i][j] = dp[i - 1][j - 1];
22             else
23                 dp[i][j] = 1 + min({dp[i][j - 1],
24                                     dp[i - 1][j],
25                                     dp[i - 1][j - 1]});
26         }
27     }
28
29     return dp[m][n];
30 }

```

11.11 Retrieve LIS

```

1 vector<int> lis(vector<int> const& a) {
2     int n = a.size();
3     vector<int> d(n, 1), p(n, -1);
4     for (int i = 0; i < n; i++) {
5         for (int j = 0; j < i; j++) {
6             if (a[j] < a[i] && d[i] < d[j] + 1) {
7                 d[i] = d[j] + 1;
8                 p[i] = j;
9             }
10        }
11    }
12
13    int ans = d[0], pos = 0;
14    for (int i = 1; i < n; i++) {
15        if (d[i] > ans) {
16            ans = d[i];
17            pos = i;
18        }
19    }
20
21    vector<int> subseq;
22    while (pos != -1) {
23        subseq.push_back(a[pos]);
24        pos = p[pos];
25    }
26    reverse(subseq.begin(), subseq.end());
27    return subseq;
28 }

```

11.12 SOS DP

```

1 typedef long long ll;
2 typedef vector<int> vec;
3 const ll mod=1e9+7;
4 const int NMAX=2e5+3;
5 const int XMAX=1e6+1e5;
6 const int BT=20;
7
8 //ascii https://elcodigoascii.com.ar/
9
10 inline void solve()
11 {
12     int n; cin>>n;
13     vector<int> arr(n);
14     vector<ll> r(XMAX),d(XMAX),zero(XMAX);
15     vector<vector<int>> res(n,vector<int>(3));
16     int maxi=0;

```

```

18     FO(i,n){
19         cin>>arr[i];
20         r[arr[i]]++;
21         d[arr[i]]++;
22     }
23
24     for(int i=0;i<BT;i++){
25         for(int j=0;j<1<<BT;j++){
26             {
27                 if(j&(1<<i)){
28                     r[j]+=r[j^(1<<i)];
29                 }
30             }
31             for(int j=(1<<BT)-1;j>=0;j--){
32                 if(j&(1<<i)){
33                     d[j^(1<<i)]+=d[j];
34                 }
35             }
36         }
37     }
38
39     FO(i,n){
40         cout<<r[arr[i]]<<" "<<d[arr[i]]<<" "<<n-r[(1<<BT)-1-arr[i]]<<endl;
41     }
42 }
43

```

11.13 Convex Hull Trick

```

1 #include <deque>
2 #include <algorithm>
3 typedef long long ll;
4
5 const ll INF = 1e18;
6
7 // LineContainer: deque-based Convex Hull Trick for monotonic slopes
8 // Maintains y = m*x + b and supports query(x) = max y at x
9 struct Line {
10     ll m, b; // y = m*x + b
11     mutable ll p; // intersection with next line (x-coordinate)
12     // Sort by slope for insertion
13     bool operator<(const Line& o) const { return m < o.m; }
14 };
15
16 // For max query with binary search on intersection x
17 struct LineContainer : std::deque<Line> {
18     // Intersection x-coordinate of lines l1 and l2: (b2-b1)/(m1-m2)
19     // l1 has smaller slope than l2 (m1 < m2 for max hull, since m1*
20         x+b1 > m2*x+b2 for large negative x)

```

```

20 static ll intersectX(const Line& l1, const Line& l2) {
21     if (l1.m == l2.m) return l1.b > l2.b ? INF : -INF;
22     // floor division for ll, works correctly for positive/
23     // negative results
24     return (l2.b - l1.b) / (l1.m - l2.m);
25 }
26
27 // Check if middle line l2 is unnecessary between l1 and l3
28 static bool isBad(const Line& l1, const Line& l2, const Line& l3)
29 {
30     // l2 is unnecessary if intersection(l1,l3) <= intersection(
31     // l1,l2)
32     // (b3-b1)/(m1-m3) <= (b2-b1)/(m1-m2)
33     // cross-multiply (watch sign of denominator)
34     return (l3.b - l1.b) * (l1.m - l2.m) <= (l2.b - l1.b) * (l1.
35     m - l3.m);
36 }
37
38 // Add line y = m*x + b (slopes added in monotonic increasing
39 // order)
40 void add(ll m, ll b) {
41     Line line = {m, b, 0};
42     // Find position based on slope
43     auto it = std::lower_bound(begin(), end(), line);
44
45     // If same slope exists, keep the one with larger b (higher
46     // line)
47     if (it != end() && it->m == m) {
48         if (b <= it->b) return; // existing line is better
49         it->b = b;
50         it = std::next(it);
51     } else {
52         it = insert(it, line);
53     }
54
55     // Remove lines made obsolete by the new line
56     while (it != begin() && std::prev(it, 1) != end()) {
57         auto l1 = std::prev(it, 2);
58         auto l2 = std::prev(it, 1);
59         if (l1->m == l2->m || l2->m == line.m) {
60             // Same slope adjacent - remove the middle one with
61             // worse b
62             if (l2->b < line.b) {
63                 erase(l2);
64                 break;
65             }
66         }
67         if (isBad(*l1, *l2, line)) {
68             erase(l2);
69         } else {
70             break;
71         }
72     }
73 }

```

```

66 // Remove lines made obsolete by the new line to the right
67 while (std::next(it) != end()) {
68     auto l2 = it;
69     auto l3 = std::next(it, 1);
70     if (isBad(*l2, *l3, line)) {
71         erase(l3);
72     } else {
73         break;
74     }
75 }
76
77 // Update intersection points
78 if (size() >= 2) {
79     auto cur = begin();
80     auto nxt = std::next(cur);
81     cur->p = intersectX(*cur, *nxt);
82 }
83
84 // Query maximum y at x
85 ll query(ll x) {
86     if (empty()) return -INF;
87     // Binary search for rightmost line whose p >= x
88     int l = 0, r = size() - 1;
89     while (l < r) {
90         int m = (l + r + 1) / 2;
91         if (at(m).p >= x) l = m;
92         else r = m - 1;
93     }
94     Line& ln = at(l);
95     return ln.m * x + ln.b;
96 }
97
98 };
99

```

11.14 Knuth's Optimization

```

1 #include <vector>
2 #include <climits>
3
4 // Knuth's Optimization for DP with quadrangle inequality
5 // dp[i][j] = min over k of (dp[i][k] + dp[k][j]) + C[i][j]
6 // Reduces O(N^3) to O(N^2) when cost matrix C satisfies quadrangle
7 // inequality
8 // and monotonicity of optimal split points.
9
10 using namespace std;
11
12 vector<vector<long long>> knuth_dp(int n, const vector<vector<long
13     long>>& cost) {

```

```

12 vector<vector<long long>> dp(n, vector<long long>(n, 0));
13 vector<vector<int>> opt(n, vector<int>(n, 0));
14
15 // Base case: opt[i][i] = i
16 for (int i = 0; i < n; i++) {
17     opt[i][i] = i;
18 }
19
20 // Increasing interval length
21 for (int len = 2; len <= n; len++) {
22     for (int i = 0; i + len - 1 < n; i++) {
23         int j = i + len - 1;
24
25         // Search for optimal k in [opt[i][j-1], opt[i+1][j]]
26         int start = opt[i][j - 1];
27         int end = (i + 1 < n) ? opt[i + 1][j] : j;
28
29         long long best = LLONG_MAX;
30         int bestK = start;
31
32         for (int k = start; k <= end; k++) {
33             long long val = dp[i][k] + dp[k][j] + cost[i][j];
34             if (val < best) {
35                 best = val;
36                 bestK = k;
37             }
38         }
39
40         dp[i][j] = best;
41         opt[i][j] = bestK;
42     }
43 }
44
45 return dp;
46 }

```

```

13 * Time complexity:  $O(n \log n)$  instead of  $O(n^2)$ 
14 *
15 * Usage:
16 *   auto cost = [&](int k, int j) { return dp[k] + transCost(k, j);
17 *   };
18 *   vector<ll> ans = divide_conquer_dp(n, cost);
19 */
20 vector<ll> divide_conquer_dp(int n, function<ll(int k, int j)> cost)
21 {
22     vector<ll> dp(n, INF);
23     dp[0] = 0;
24
25     // Recursive D&C: compute dp[l..r] given that dp[0..l-1] is
26     // computed
27     function<void(int, int, int, int)> compute = [&](int l, int r,
28         int optL, int optR) {
29         if (l > r) return;
30         int mid = (l + r) >> 1;
31         pair<ll, int> best = {INF, -1};
32
33         // Search for optimal k in [optL, min(mid-1, optR)]
34         int upper = min(mid - 1, optR);
35         for (int k = optL; k <= upper; ++k) {
36             ll val = dp[k] + cost(k, mid);
37             if (val < best.first) {
38                 best = {val, k};
39             }
40         }
41
42         dp[mid] = best.first;
43         int optK = best.second;
44
45         // Left half: optimal k is in [optL, optK]
46         compute(l, mid - 1, optL, optK);
47         // Right half: optimal k is in [optK, optR]
48         compute(mid + 1, r, optK, optR);
49     };
50
51     compute(1, n - 1, 0, n - 1);
52     return dp;
53 }

```

11.15 Divide and Conquer DP

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 const ll INF = 1e18;
6
7 /**
8  * Divide and Conquer DP Optimization
9  *
10  * For DP of the form:  $dp[j] = \min_{k < j} (dp[k] + cost(k, j))$ 
11  * where the optimal k for each j is monotonic ( $opt[j] \leq opt[j+1]$ ).
12  *

```

12 Trees

12.1 Diameter

```

1 const int MAX = 2e5+3;
2
3 #define INF LLONG_MAX

```

```

4
5 vector<vector<int>> adj(MAX);
6 bool vis[MAX];
7 vector<int> dist(MAX);
8
9 int far;
10 void dfs(int s, int &max_count, int c){
11     vis[s] = true;
12     c++;
13     for(auto u : adj[s]){
14         if(!vis[u]){
15             if(c >= max_count){
16                 max_count = c;
17                 far = u;
18             }
19             dfs(u,max_count, c);
20         }
21     }
22 }
23
24 void resetVisited(int n){
25     for(int i=1; i<=n; i++)
26         vis[i] = false;
27 }
28
29 void solve(){
30     int n; cin>>n;
31     for(int i=1; i<=n-1; i++){
32         int u,v; cin>>u>>v;
33         adj[u].push_back(v);
34         adj[v].push_back(u);
35     }
36
37     int max_count = INT_MIN;
38     dfs(1,max_count,0);
39     resetVisited(n);
40     dfs(far,max_count,0);
41     if(max_count == INT_MIN){
42         cout<<0<<endl;
43         return;
44     }
45
46     cout<<max_count<<endl;
47 }

```

12.2 Euler Tour

```

1 vector<vector<int>> adj(MAX);
2 vector<int> in(MAX);
3 vector<int> out(MAX);

```

```

4 vector<ll> values(MAX);
5 vector<ll> flat(MAX);
6 int timer = 1;
7
8 ll t[2*MAX];
9
10 void build(int n){
11     for(int i=1; i<=n; i++)
12         t[n+i] = flat[i];
13
14     for(int i=n; i>=1; i--)
15         t[i] = t[i<<1] + t[i<<1 | 1];
16 }
17
18 void update(int idx, int v, int n){
19     for(t[idx += n] = v; idx > 1; idx >>= 1)
20         t[idx >> 1] = t[idx ^ 1] + t[idx];
21 }
22
23 // Query [l,r)
24
25 ll query(int l, int r, int n){
26     ll arns = 0;
27     for(l+=n, r+=n; l<r; l>>=1, r>>=1){
28         if (l & 1) arns += t[l++];
29         if (r & 1) arns += t[--r];
30     }
31     return arns;
32 }
33
34 void eulerTour(int s, int e){
35     in[s] = timer;
36     flat[timer++] = values[s];
37     for(auto u : adj[s]){
38         if (u == e) continue;
39         eulerTour(u,s);
40     }
41     out[s] = timer;
42 }
43
44 void solve(){
45     int n,q; cin>>n>>q;
46     for(int i=1; i<=n; i++){
47         cin>>values[i];
48     }
49
50     for(int i=1; i<=n-1; i++){
51         int u,v; cin>>u>>v;
52         adj[u].push_back(v);
53         adj[v].push_back(u);
54     }
55 }
56

```

```

57     eulerTour(1,0);
58     /*
59     for(int i=1; i<=n; i++){
60         cout<<in[i]<<endl;
61     }
62
63     for(int i=1; i<=n; i++){
64         cout<<out[i]<<endl;
65     }
66     */
67
68     build(n);
69
70     for(int i=1; i<=q; i++){
71         int a; cin>>a;
72         if(a == 1){
73             int v; int x; cin>>v>>x;
74             update(in[v],x,n);
75         }else{
76             int v; cin>>v;
77             cout<<query(in[v],out[v],n)<<endl;
78         }
79     }
80 }
81

```

```

22 }
23
24 void hash_tree1(int u, int prev, int depth){
25     h_map[u] = 1;
26     for(int v : tree[u]){
27         if(v!=prev){
28             hash_tree1(v,u,depth+1);
29             h_map[u] += mix(h_map[v]) ;
30         }
31     }
32 }
33
34 void hash_tree2(int u, int prev, int depth){
35     t2_map[u] = 1;
36     for(int v : tree2[u]){
37         if(v!=prev){
38             hash_tree2(v,u,depth+1);
39             t2_map[u] += mix(t2_map[v]);
40         }
41     }
42 }

```

12.3 Tree Isomorphism

```

1 vector<ull> h_map;
2 vector<ull> t2_map;
3 vector<ull> d; // Random variable per depth
4 vector<vector<int>> tree;
5 vector<vector<int>> tree2;
6 int arns = 0;
7
8 ull mix(ull x){
9     x += 0x9e3779b97f4a7c15;
10    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
11    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
12    return x ^ (x >> 31);
13 }
14
15 void init_rng(int depth){
16     uint64_t seed = chrono::steady_clock::now().time_since_epoch().
17         count();
18     mt19937_64 rng(seed);
19     d.resize(depth+1);
20     for(int i=0; i<=depth; i++){
21         d[i] = rng();
22     }
23 }

```

12.4 Max Distance

```

1 #define INF LLONG_MAX
2
3 vector<vector<int>> adj(MAX);
4 vector<int> f(MAX,0); // f(x) = maximum distance from node x to any
5     node of its subtree
6 vector<int> g(MAX,0); // g(x) = maximum distance from any node to x
7     outside its subtree
8 vector<int> arns(MAX,0); // arns(x) = max(f(x),g(x))
9
10 void dfsA(int s, int e){
11     for(auto u : adj[s]){
12         if(u == e) continue;
13         dfsA(u,s);
14         if(f[u]+1 > f[s]){ // Maximum distance + 1 of u > Max dist
15             of predecessor
16             g[s] = f[s]; // Maximum distance from any node outside
17                 of s subtree is f[s]
18             f[s] = f[u] + 1;
19         }else if(f[u]+1>g[s]){
20             g[s] = f[u]+1; // Maximum distance from any node outside
21                 of s subtree is f[u]+1
22         }
23     }
24 }
25
26 }
27
28 }
29
30 }
31

```

```

22 void dfsB(int s, int e, int to){
23     arns[s] = max(to, f[s]);
24     for(auto u : adj[s]){
25         if(u == e) continue;
26         if(f[u]+1 == f[s]) dfsB(u,s, max(to,g[s])+1);
27         else dfsB(u,s, arns[s]+1);
28     }
29 }
30
31 void solve(){
32     int n; cin>>n;
33     for(int i=1; i<=n-1; i++){
34         int u,v; cin>>u>>v;
35         adj[u].push_back(v);
36         adj[v].push_back(u);
37     }
38
39     dfsA(1,0);
40     dfsB(1,0,0);
41
42     for(int i=1;i<=n; i++)
43         cout<<arns[i]<<" ";
44     cout<<endl;
45 }

```

```

24 {
25     return tin[u] <= tin[v] && tout[u] >= tout[v];
26 }
27
28 int lca(int u, int v)
29 {
30     if (is_ancestor(u, v))
31         return u;
32     if (is_ancestor(v, u))
33         return v;
34     for (int i = 1; i >= 0; --i) {
35         if (!is_ancestor(up[u][i], v))
36             u = up[u][i];
37     }
38     return up[u][0];
39 }
40
41 void preprocess(int root) {
42     tin.resize(n);
43     tout.resize(n);
44     timer = 0;
45     l = ceil(log2(n));
46     up.assign(n, vector<int>(l + 1));
47     dfs(root, root);
48 }

```

12.5 Binary Lifting

```

1 int n, l;
2 vector<vector<int>> adj;
3
4 int timer;
5 vector<int> tin, tout;
6 vector<vector<int>> up;
7
8 void dfs(int v, int p)
9 {
10     tin[v] = ++timer;
11     up[v][0] = p;
12     for (int i = 1; i <= l; ++i)
13         up[v][i] = up[up[v][i-1]][i-1];
14
15     for (int u : adj[v]) {
16         if (u != p)
17             dfs(u, v);
18     }
19
20     tout[v] = ++timer;
21 }
22
23 bool is_ancestor(int u, int v)

```

12.6 Centroid Decomposition

```

1 bool v[MAX];
2 int dst[MAX];
3 int sz[MAX];
4 vector<int> adj[MAX];
5
6 // Buffer estatico global para recolectar profundidades rapidisimo
7 int depth_buffer[MAX];
8
9 class CentroidDecomposition {
10 private:
11     int k;
12     ll res=0;
13
14     void sztree(int s, int par=-1) {
15         sz[s]=1;
16         for(int ch : adj[s]){
17             if(ch==par || v[ch]) continue;
18             sztree(ch, s);
19             sz[s] += sz[ch];
20         }
21     }
22 }

```

```

23 // Un solo DFS por subarbol: guarda los resultados en el
24 // buffer continuo
25 void get_depths(int s, int par, int depth, int &tail, int &
26 // max_d) {
27     if(depth > k) return;
28     depth_buffer[tail++] = depth;
29     max_d = max(max_d, depth);
30     for(int ch : adj[s]){
31         if(ch==par || v[ch]) continue;
32         get_depths(ch, s, depth+1, tail, max_d);
33     }
34 }
35 public:
36 CentroidDecomposition(int _k): k(_k) {
37     res=0;
38 }
39 void addEdge(int a, int b){
40     adj[a].push_back(b);
41     adj[b].push_back(a);
42 }
43
44 // VERSION ITERATIVA: A prueba de Stack Overflow (evita TLE/
45 // RTE en casos borde)
46 int getCentroid(int s, int n) {
47     int par = -1;
48     bool found = true;
49     while(found) {
50         found = false;
51         for(int ch : adj[s]) {
52             if(ch != par && !v[ch] && sz[ch] > n/2) {
53                 par = s;
54                 s = ch;
55                 found = true;
56                 break;
57             }
58         }
59     }
60     return s;
61 }
62 void Decompose(int s, int par=-1) {
63     sztree(s, par);
64     s = getCentroid(s, sz[s]); // Encontramos centroide de
65     // forma segura
66     v[s] = true;
67
68     dst[0] = 1;
69     int current_max_depth = 0;
70     sort(all(adj[s]), [](int a, int b){
71         return sz[a] < sz[b];
72     });

```

```

72
73     for(int ch : adj[s]){
74         if(v[ch]) continue; // Solo nos importa no cruzar
75         // componentes visitados
76
77         int tail = 0;
78         int sub_max_depth = 0;
79
80         // 1. Recolectar (1 solo DFS)
81         get_depths(ch, s, 1, tail, sub_max_depth);
82
83         // 2. Contar caminos cruzados (bucle cache-friendly)
84         for(int i = 0; i < tail; i++) {
85             res += dst[k - depth_buffer[i]];
86         }
87
88         // 3. Agregar subarbol actual al arreglo maestro
89         for(int i = 0; i < tail; i++) {
90             dst[depth_buffer[i]]++;
91         }
92
93         current_max_depth = max(current_max_depth,
94             sub_max_depth);
95     }
96
97     // Limpieza ultra rapida usando memset implicito
98     fill(dst, dst + current_max_depth + 1, 0);
99     dst[0] = 1; // restore centroid contribution for
100     // subsequent children
101
102     // Recursion para el resto de los componentes
103     for(int ch : adj[s]){
104         if(v[ch]) continue;
105         Decompose(ch, s);
106     }
107
108     ll getRes(){ return res; }
109 };

```

12.7 Heavy-Light Decomposition

```

1 vector<int> parent, depth, heavy, head, pos;
2 int cur_pos;
3
4 int dfs(int v, vector<vector<int>> &adj) {
5     int size = 1;
6     int max_c_size = 0;
7     for (int c : adj[v]) {
8         if (c != parent[v]) {

```

```

9         parent[c] = v, depth[c] = depth[v] + 1;
10        int c_size = dfs(c, adj);
11        size += c_size;
12        if (c_size > max_c_size)
13            max_c_size = c_size, heavy[v] = c;
14    }
15    }
16    return size;
17 }
18
19 void decompose(int v, int h, vector<vector<int>> const& adj) {
20     head[v] = h, pos[v] = cur_pos++;
21     if (heavy[v] != -1)
22         decompose(heavy[v], h, adj);
23     for (int c : adj[v]) {
24         if (c != parent[v] && c != heavy[v])
25             decompose(c, c, adj);
26     }
27 }
28
29 void init(vector<vector<int>> const& adj) {
30     int n = adj.size();
31     parent = vector<int>(n);
32     depth = vector<int>(n);
33     heavy = vector<int>(n, -1);
34     head = vector<int>(n);
35     pos = vector<int>(n);
36     cur_pos = 0;
37
38     dfs(0, adj);
39     decompose(0, 0, adj);
40 }
41
42 int query(int a, int b) {
43     int res = 0;
44     for (; head[a] != head[b]; b = parent[head[b]]) {
45         if (depth[head[a]] > depth[head[b]])
46             swap(a, b);
47         int cur_heavy_path_max = segment_tree_query(pos[head[b]],
48             pos[b]);
49         res = max(res, cur_heavy_path_max);
50     }
51     if (depth[a] > depth[b])
52         swap(a, b);
53     int last_heavy_path_max = segment_tree_query(pos[a], pos[b]);
54     res = max(res, last_heavy_path_max);
55     return res;
56 }

```

```

1 struct LCA {
2     vector<int> height, euler, first, segtree;
3     vector<bool> visited;
4     int n;
5
6     LCA(vector<vector<int>> &adj, int root = 0) {
7         n = adj.size();
8         height.resize(n);
9         first.resize(n);
10        euler.reserve(n * 2);
11        visited.assign(n, false);
12        dfs(adj, root);
13        int m = euler.size();
14        segtree.resize(m * 4);
15        build(1, 0, m - 1);
16    }
17
18    void dfs(vector<vector<int>> &adj, int node, int h = 0) {
19        visited[node] = true;
20        height[node] = h;
21        first[node] = euler.size();
22        euler.push_back(node);
23        for (auto to : adj[node]) {
24            if (!visited[to]) {
25                dfs(adj, to, h + 1);
26                euler.push_back(node);
27            }
28        }
29    }
30
31    void build(int node, int b, int e) {
32        if (b == e) {
33            segtree[node] = euler[b];
34        } else {
35            int mid = (b + e) / 2;
36            build(node << 1, b, mid);
37            build(node << 1 | 1, mid + 1, e);
38            int l = segtree[node << 1], r = segtree[node << 1 | 1];
39            segtree[node] = (height[l] < height[r]) ? l : r;
40        }
41    }
42
43    int query(int node, int b, int e, int L, int R) {
44        if (b > R || e < L)
45            return -1;
46        if (b >= L && e <= R)
47            return segtree[node];
48        int mid = (b + e) >> 1;
49
50        int left = query(node << 1, b, mid, L, R);
51        int right = query(node << 1 | 1, mid + 1, e, L, R);
52        if (left == -1) return right;

```

12.8 Lowest Common Ancestor

```

53     if (right == -1) return left;
54     return height[left] < height[right] ? left : right;
55 }
56
57 int lca(int u, int v) {
58     int left = first[u], right = first[v];
59     if (left > right)
60         swap(left, right);
61     return query(1, 0, euler.size() - 1, left, right);
62 }
63 };

```

12.9 Successor DP

```

1  const int m=30;
2  int sucesorM[limit][m];
3  //ascii https://elcodigoascii.com.ar/
4
5  inline void solve()
6  {
7      int n,q; cin>>n>>q;
8      int res,aux;
9      ll k;
10     lFOR(i,n){
11         cin>>sucesorM[i][0];
12     }
13     FOR(j,1,m)
14     {
15         lFOR(i,n)
16         {
17             sucesorM[i][j]=sucesorM[sucesorM[i][j-1]][j-1];
18         }
19     }
20     FO(i,q)
21     {
22         cin>>res>>k;
23         aux=0;
24         while(k)
25         {
26             if(k%2){
27                 res=sucesorM[res][aux];
28             }
29             k/=2;
30             aux++;
31         }
32         cout<<res<<endl;
33     }
34 }

```

13 Miscellaneous

13.1 2D Prefix Sum

```

1  void solve(){
2      int n,q; cin>>n>>q;
3      char forest[n+1][n+1];
4
5      for(int i=1; i<=n; i++){
6          for(int j=1; j<=n; j++){
7              cin>>forest[i][j];
8          }
9      }
10
11     vector<vector<int>> dp(n+1, vector<int> (n+1,0));
12
13
14     for(int i=1; i<=n; i++){
15         for(int j=1; j<=n; j++){
16             dp[i][j] = dp[i][j-1] + dp[i-1][j] - dp[i-1][j-1];
17             if(forest[i][j] == '*') dp[i][j]++;
18         }
19     }
20
21
22     for(int i=0; i<q; i++){
23         int a,b,c,d;
24         cin>>a>>b>>c>>d;
25         cout<<dp[c][d] - dp[a-1][d] - dp[c][b-1] + dp[a-1][b-1]<<
26             endl;
27     }
28 }
29

```

13.2 Kadane

```

1  ll kadane(vector<int> &a, int n){
2      ll arns = a[0], zum = 0;
3
4      for (int i = 0; i < n; i++){
5          zum += a[i];
6          arns = max(arns, zum);
7          zum = max(zum, 0LL);
8      }
9
10     return arns;
11 }

```

13.3 Index Compression

```

1 sort(all(a));
2
3 auto compression = [&](int x) {
4     return (int)(lower_bound(all(a), x) - a.begin());
5 };

```

13.4 Berlekamp-Massey

```

1 vector<T> berlekampMassey(const vector<T> &s) {
2     vector<T> c; // the linear recurrence sequence we are
3         building
4     vector<T> oldC; // the best previous version of c to use (the
5         one with the rightmost left endpoint)
6     int f = -1; // the index at which the best previous version
7         of c failed on
8     for (int i=0; i<(int)s.size(); i++) {
9         // evaluate c(i)
10        // delta = s_i - \sum_{j=1}^n c_j s_{i-j}
11        // if delta == 0, c(i) is correct
12        T delta = s[i];
13        for (int j=1; j<=(int)c.size(); j++)
14            delta -= c[j-1] * s[i-j]; // c_j is one-indexed, so we
15            actually need index j - 1 in the code
16        if (delta == 0)
17            continue; // c(i) is correct, keep going
18        // now at this point, delta != 0, so we need to adjust it
19        if (f == -1) {
20            // this is the first time we're updating c
21            // s_i was the first non-zero element we encountered
22            // we make c of length i + 1 so that s_i is part of the
23            base case
24            c.assign(i + 1, T(0));
25            c[i] = T(1);
26            f = i;
27        } else {
28            // we need to use a previous version of c to improve on
29            this one
30            // apply the 5 steps to build d
31            // 1. set d equal to our chosen sequence
32            vector<T> d = oldC;
33            // 2. multiply the sequence by -1
34            for (T &x : d)
35                x = -x;
36            // 3. insert a 1 on the left
37            d.insert(d.begin(), 1);
38            // 4. multiply the sequence by delta / d(f + 1)
39            T df1 = 0; // d(f + 1)
40            for (int j=1; j<=(int)d.size(); j++)

```

```

35        df1 += d[j-1] * s[f+1-j];
36        assert(df1 != 0);
37        T coef = delta / df1; // storing this in outer
38            variable so it's O(n^2) instead of O(n^2 log MOD)
39        for (T &x : d)
40            x *= coef;
41        // 5. insert i - f - 1 zeros on the left
42        vector<T> zeros(i - f - 1);
43        zeros.insert(zeros.end(), d.begin(), d.end());
44        d = zeros;
45        // now we have our new recurrence: c + d
46        vector<T> temp = c; // save the last version of c
47            because it might have a better left endpoint
48        c.resize(max(c.size(), d.size()));
49        for (int j=0; j<(int)d.size(); j++)
50            c[j] += d[j];
51        // finally, let's consider updating oldC
52        if (i - (int) temp.size() > f - (int) oldC.size()) {
53            // better left endpoint, let's update!
54            oldC = temp;
55            f = i;
56        }
57    }
58    return c;

```

13.5 Moore's Voting

```

1 int majorityElement(vector<int> & nums) {
2     int vote = 0, r = 0;
3     for(int i=0; i<nums.size(); i++){
4         if(nums[i] == nums[r]){
5             vote++;
6         }
7         else{
8             vote--;
9             if(vote == 0){
10                r = i;
11                vote = 1;
12            }
13        }
14    }
15    int cnt = 0;
16    int goal = (nums.size())/2;
17    for(int i=0; i<nums.size(); i++){
18        if(nums[i] == nums[r]){
19            cnt++;
20            if(cnt > goal){
21                break;
22            }
23        }
24    }
25    return r;
26 }

```

```

22     }
23 }
24
25     return nums[r];
26 }

```

13.6 Compiler Optimizations

```

1  // C++ compiler optimizations for competitive programming.
2  // Use in submission files when heavy loops / math / bit operations
3  // are bottlenecks.
4  // Avoid for interactive/debug-heavy tasks because aggressive
5  // optimizations can make debugging harder.
6
7  #pragma GCC optimize("O3,unroll-loops")
8  #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
9
10 // Optional fast I/O toggle:
11 // ios::sync_with_stdio(false);
12 // cin.tie(nullptr);
13
14 // Notes:
15 // - O3 + unroll-loops: usually safe and useful in most non-
16 //   interactive problems.
17 // - target(...): can speed up bitset/popcount-heavy code, but may
18 //   fail on judges without these CPU features.
19 // - If unsure about judge hardware, remove target(...) and keep O3
20 //   only.

```

```

17     return lo;
18 }
19
20 int grundy(int state, function<vector<int>(int)> moves) {
21     static unordered_map<int, int> memo;
22     if (memo.count(state)) return memo[state];
23     vector<int> g;
24     for (int nxt : moves(state))
25         g.push_back(grundy(nxt, moves));
26     return memo[state] = mex(g);
27 }

```

13.7 Sprague-Grundy

```

1  #include <vector>
2  #include <functional>
3  #include <unordered_map>
4  using namespace std;
5
6  int mex(vector<int>& s) {
7      if (s.empty()) return 0;
8      int n = s.size();
9      int lo = 0, hi = n;
10     while (lo < hi) {
11         int mid = (lo + hi) / 2;
12         int cnt = 0;
13         for (int x : s) if (x <= mid) cnt++;
14         if (cnt > mid) hi = mid;
15         else lo = mid + 1;
16     }

```